

Master Python in 30 Modules - Complete Course Curriculum

Course Overview

Welcome to this comprehensive Python course! Using the **ToyotechICT Solutions WWWH Learning Model** (What, When, Where, How), we'll transform you from a complete beginner to a confident Python developer capable of building real-world applications.

STAGE 1:

PYTHON BASICS (Modules 1-7)

Module 1: Introduction to Python & Setup

WHAT is Python?

Python is a high-level, interpreted, dynamically typed programming language known for its readability and simplicity. Created by Guido van Rossum in 1991, it emphasizes code readability with its distinctive use of significant whitespace.

WHEN to use Python?

- **Rapid prototyping:** When you need to build something quickly
- **Data science projects:** When working with large datasets
- **Web development:** When building backend services
- **Automation:** When you need to automate repetitive tasks
- **Learning programming:** As your first programming language

WHERE is Python used?

- **Web Development:** Django, Flask frameworks

- **Data Science:** Pandas, NumPy, Scikit-learn
- **Machine Learning/AI:** TensorFlow, PyTorch
- **Automation:** Scripting, web scraping
- **Game Development:** Pygame
- **Scientific Computing:** Research institutions, NASA

HOW to set up Python?

Step 1: Download Python from python.org
Step 2: Verify installation
Open terminal/command prompt and type:
python --version

Step 3: Your first Python program
print("Hello, World!")
print("Welcome to Python Mastery Course!")
print("Let's begin this 30-day journey!")

Step 4: Understanding the Python interpreter
Python can be used in interactive mode (REPL)
>>> 2 + 2
4
*# >>> "Python" * 3*
'PythonPythonPython'

Class Activity:

- Install Python and VS Code/PyCharm
- Install code runner extension in VS Code
- Install Nodejs - nodejs.org
- Create your first Python file (*hello.py*)
- Run your first program
- Explore the Python REPL

Module 2: Variables & Data Types

WHAT are Variables?

Variables are containers for storing data values. Python is dynamically typed, meaning you don't need to declare variable types explicitly.

WHEN to use different data types?

- **Integers:** Counting, indexing, mathematical operations
- **Floats:** Scientific calculations, measurements
- **Strings:** Text processing, user input, file handling
- **Booleans:** Conditional logic, flags
- **None:** Representing absence of value

WHERE to use variables?

- Storing user input
- Tracking program state
- Temporary data storage
- Configuration settings

HOW to use variables and data types?

Numeric Types

```
age = 25          # Integer
price = 19.99     # Float
complex_num = 3 + 4j  # Complex number
```

Text Type

```
name = "Alice"    # String (can use single or double quotes)
multiline = """This is a
multiline string""" # Triple quotes for multiline
```

Boolean Type

```
is_student = True    # Boolean (True/False, case-sensitive)
is_graduated = False
```

None Type

```
result = None        # Represents null/absence of value
```

Type Checking

```
print(type(age))      # <class 'int'>
print(type(name))     # <class 'str'>
print(type(is_student)) # <class 'bool'>
```

Type Conversion (Casting)

```
age_str = str(age)     # 25 -> "25"
price_int = int(price) # 19.99 -> 19
is_active = bool(1)    # True
```

Variable Naming Rules

```
# Valid: user_name, _private, age2, UserName
# Invalid: 2name, user-name, class, my name
```

Multiple Assignment

x, y, z = 10, 20, 30

a = b = c = 100

Constants (by convention, use UPPERCASE)

PI = 3.14159

MAX_CONNECTIONS = 100

Class Activity:

- Create variables for a student profile (name, age, grades, enrollment status)
- Practice type conversion
- Explore memory usage with `id()` function

Module 3: Operators & Expressions

WHAT are Operators?

Operators are special symbols that perform operations on variables and values. An expression combines variables, values, and operators.

WHEN to use different operators?

- **Arithmetic:** Mathematical calculations
- **Comparison:** Making decisions, sorting
- **Logical:** Combining conditions
- **Assignment:** Updating values
- **Membership:** Checking existence in collections

WHERE to use operators?

- Calculating totals in e-commerce
- Validating user input
- Loop conditions
- Data filtering

HOW to use operators?

1. Arithmetic Operators

a, b = 15, 4

print(a + b) # 19 - Addition

print(a - b) # 11 - Subtraction

*print(a * b) # 60 - Multiplication*

```
print(a / b)  # 3.75 - Division (always returns float)
print(a // b) # 3 - Floor Division (integer division)
print(a % b)  # 3 - Modulus (remainder)
print(a ** b) # 50625 - Exponentiation
```

2. Comparison Operators

```
x, y = 10, 20
print(x == y) # False - Equal to
print(x != y) # True - Not equal to
print(x > y)  # False - Greater than
print(x < y)  # True - Less than
print(x >= y) # False - Greater than or equal to
print(x <= y) # True - Less than or equal to
```

3. Logical Operators

```
is_weekend = True
has_time = False
print(is_weekend and has_time) # False - AND
print(is_weekend or has_time)  # True - OR
print(not is_weekend)          # False - NOT
```

4. Assignment Operators

```
num = 10
num += 5    # num = num + 5 -> 15
num -= 3    # num = num - 3 -> 12
num *= 2    # num = num * 2 -> 24
num /= 4    # num = num / 4 -> 6.0
```

5. Identity Operators

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = list1
print(list1 is list3)    # True - same object
print(list1 is list2)    # False - different objects
print(list1 is not list2) # True
```

6. Membership Operators

```
fruits = ['apple', 'banana', 'orange']
print('apple' in fruits)    # True
print('grape' not in fruits) # True
```

7. Bitwise Operators (for advanced use)

```
p, q = 5, 3 # 5=0101, 3=0011 in binary
print(p & q) # 1 (0001) - AND
```

```

print(p | q)  # 7 (0111) - OR
print(p ^ q)  # 6 (0110) - XOR
print(~p)     # -6 - NOT
print(p << 1) # 10 (1010) - Left shift
print(p >> 1) # 2 (0010) - Right shift

```

Operator Precedence

```

result = 10 + 5 * 2 ** 2 # Exponentiation first, then multiplication, then addition
print(result) # 30 (5 * 4 = 20, 10 + 20 = 30)

```

Use parentheses for clarity

```

clear_result = 10 + (5 * (2 ** 2))
print(clear_result) # 30

```

Class Activity:

- Build a simple calculator using arithmetic operators
- Create a grade checker using comparison operators
- Implement login validation using logical operators

Module 4: Input & Output Functions

WHAT are Input/Output Functions?

Input functions allow programs to receive data from users, while output functions display information to users.

WHEN to use input/output?

- **Input:** Getting user data, configuration, interactive programs
- **Output:** Showing results, debugging, user feedback

WHERE to use I/O?

- Command-line applications
- Interactive scripts
- Data entry programs
- Progress indicators

HOW to use input/output functions?

```

# Basic Input Function
name = input("Enter your name: ")
print(f"Hello, {name}!")

```

Handling Different Data Types

age = int(input("Enter your age: ")) # Convert to integer

height = float(input("Enter your height in meters: ")) # Convert to float

Multiple Inputs

x, y = input("Enter two numbers: ").split()

x, y = int(x), int(y)

Basic Print Function

print("Simple text")

print("Text", "with", "multiple", "arguments", sep=" - ")

print("No newline", end="")

print(" on same line")

Formatted Output

name, age = "Alice", 25

f-strings (Python 3.6+)

print(f"My name is {name} and I'm {age} years old")

.format() method

print("My name is {} and I'm {} years old".format(name, age))

% formatting (older style)

print("My name is %s and I'm %d years old" % (name, age))

Formatting Numbers

price = 19.9999

print(f"Price: \${price:.2f}") # Price: \$20.00

percentage = 0.856

print(f"Success rate: {percentage:.1%}") # Success rate: 85.6%

Alignment and Width

print(f"{'Name':<10} {'Age':>5}") # Left and right alignment

print(f"{'Alice':<10} {25:>5}")

print(f"{'Bob':<10} {30:>5}")

Advanced Print Options

import sys

sys.stdout.write("This is low-level output\n") # Alternative output

Error Output

import sys

```

print("Error message", file=sys.stderr)

# Pretty Printing
import pprint
data = {"name": "Alice", "grades": [85, 92, 78], "metadata": {"year": 2024}}
pprint.pprint(data, indent=2, width=40)

# Input Validation Example
def get_valid_age():
    while True:
        try:
            age = int(input("Enter your age: "))
            if 0 <= age <= 120:
                return age
            print("Please enter a valid age (0-120)")
        except ValueError:
            print("Please enter a number")

age = get_valid_age()
print(f"Your age is {age}")

```

Class Activity:

- Create a personal information collector
- Build a simple mad libs game
- Implement a temperature converter with user input

Module 5: Strings & String Operations

WHAT are Strings?

Strings are sequences of characters used to represent text in Python. They're immutable (cannot be changed after creation).

WHEN to use string operations?

- Text processing and manipulation
- Data validation and cleaning
- Formatting output
- Parsing structured text

WHERE to use strings?

- User interfaces
- File processing

- API communication
- Database queries

HOW to work with strings?

String Creation

```
single_quotes = 'Hello'
```

```
double_quotes = "World"
```

```
multiline = """This is a
```

```
multiline string"""
```

```
raw_string = r"C:\Users\name" # Raw string (ignores escape chars)
```

String Indexing

```
text = "Python"
```

```
print(text[0])    # 'P' - first character
```

```
print(text[-1])   # 'n' - last character
```

```
print(text[1:4])  # 'yth' - slicing
```

```
print(text[::-1]) # 'nohtyP' - reverse
```

String Methods

```
text = " Hello Python World "
```

Case Conversion

```
print(text.lower()) # " hello python world "
```

```
print(text.upper()) # " HELLO PYTHON WORLD "
```

```
print(text.title()) # " Hello Python World "
```

```
print(text.swapcase()) # " hELLO pYTHON wORLD "
```

```
print(text.capitalize())# " hello python world "
```

Stripping Whitespace

```
print(text.strip()) # "Hello Python World"
```

```
print(text.lstrip()) # "Hello Python World "
```

```
print(text.rstrip()) # " Hello Python World"
```

Search and Replace

```
print(text.find("Python")) # 8 (index where found)
```

```
print(text.find("Java")) # -1 (not found)
```

```
print(text.index("Python")) # 8 (raises error if not found)
```

```
print(text.count("l")) # 3
```

```
print(text.replace("Python", "Java")) # " Hello Java World "
```

String Validation

```
print("abc123".isalnum()) # True (alphanumeric)
```

```
print("abc".isalpha()) # True (only letters)
```

```
print("123".isdigit()) # True (only digits)
```

```
print(" ".isspace())    # True (only whitespace)
print("Hello".istitle()) # True (title case)
print("HELLO".isupper()) # True (all uppercase)
```

String Splitting and Joining

```
csv_data = "apple,banana,orange"
fruits = csv_data.split(",") # ['apple', 'banana', 'orange']
print(fruits)
```

```
sentence = "Python is awesome"
words = sentence.split()    # ['Python', 'is', 'awesome']
```

Joining

```
fruit_list = ['apple', 'banana', 'orange']
joined = ", ".join(fruit_list) # "apple, banana, orange"
```

String Formatting

```
name, age, score = "Alice", 25, 85.5
```

f-strings (preferred)

```
print(f"{name} is {age} years old and scored {score:.1f}%")
```

.format()

```
print("{} is {} years old".format(name, age))
print("{0} is {1} years old, {0} scored {2}%".format(name, age, score))
```

Format specifiers

```
print(f"{age:5d}")    # right-aligned, width 5
print(f"{age:<5d}")    # left-aligned
print(f"{age:^5d}")    # center-aligned
print(f"{score:08.2f}") # zero-padded, 8 width, 2 decimals
```

Advanced String Operations

```
import string
```

String constants

```
print(string.ascii_lowercase) # 'abcdefghijklmnopqrstuvwxyz'
print(string.digits)          # '0123456789'
print(string.punctuation)     # '!"#$%&'\()*+,-./:;<=>?@[\\]^_`{|}~'
```

String concatenation

```
parts = ["Part1", "Part2", "Part3"]
result = "".join(parts) # Efficient for many concatenations
```

```

# Substring checking
if "Python" in text:
    print("Found Python!")

# Common String Operations
def is_palindrome(text):
    """Check if a string is a palindrome"""
    cleaned = "".join(char.lower() for char in text if char.isalnum())
    return cleaned == cleaned[::-1]

print(is_palindrome("A man a plan a canal Panama")) # True

```

Class Activity:

- Build a text analyzer (count characters, words, sentences)
- Create a password strength checker
- Implement a simple text-based encryption/decryption

Module 6: Lists & List Methods

WHAT are Lists?

Lists are ordered, mutable collections that can hold items of different types. They're one of Python's most versatile data structures.

WHEN to use lists?

- Storing collections of related data
- Maintaining order in data
- When you need to modify the collection
- Implementing stacks and queues

WHERE to use lists?

- Managing todo lists
- Storing user data
- Processing sequences
- Batch operations

HOW to work with lists?

```

# List Creation
empty_list = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
nested = [[1, 2], [3, 4], [5, 6]]

```

```
list_from_range = list(range(5)) # [0, 1, 2, 3, 4]
```

```
# List Comprehension
```

```
squares = [x**2 for x in range(10)] # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
evens = [x for x in range(20) if x % 2 == 0]
```

```
# Accessing Elements
```

```
fruits = ['apple', 'banana', 'orange', 'grape', 'kiwi']
```

```
print(fruits[0])      # 'apple'
```

```
print(fruits[-1])     # 'kiwi'
```

```
print(fruits[1:3])    # ['banana', 'orange']
```

```
print(fruits[:2])     # ['apple', 'orange', 'kiwi']
```

```
# Modifying Lists
```

```
fruits[0] = 'pear'     # Change element
```

```
fruits.append('mango') # Add to end
```

```
fruits.insert(1, 'berry') # Insert at index
```

```
fruits.extend(['lemon', 'lime']) # Extend with another list
```

```
# Removing Elements
```

```
fruits.remove('banana') # Remove by value (first occurrence)
```

```
popped = fruits.pop()   # Remove and return last element
```

```
popped_at = fruits.pop(2) # Remove and return at index
```

```
del fruits[0]           # Delete by index
```

```
fruits.clear()          # Empty the list
```

```
# List Methods
```

```
numbers = [3, 1, 4, 1, 5, 9, 2]
```

```
print(len(numbers))     # 7 - length
```

```
print(numbers.count(1))  # 2 - count occurrences
```

```
print(numbers.index(5))  # 4 - find index of value
```

```
numbers.sort()          # Sort in place
```

```
print(numbers)          # [1, 1, 2, 3, 4, 5, 9]
```

```
numbers.sort(reverse=True) # Sort descending
```

```
print(numbers)          # [9, 5, 4, 3, 2, 1, 1]
```

```
numbers.reverse()       # Reverse order
```

```
print(numbers)          # [1, 1, 2, 3, 4, 5, 9]
```

```
# Copying Lists (Important!)
```

```
original = [1, 2, [3, 4]]
```

```
shallow_copy = original.copy() # Shallow copy
```

```
deep_copy = original.deepcopy()      # Need import copy
```

```
# List Operations
```

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
combined = list1 + list2      # [1, 2, 3, 4, 5, 6]
```

```
repeated = list1 * 3          # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
# Membership Testing
```

```
if 'apple' in fruits:
```

```
    print("Found apple!")
```

```
# Iterating through lists
```

```
fruits = ['apple', 'banana', 'orange']
```

```
# Basic iteration
```

```
for fruit in fruits:
```

```
    print(fruit)
```

```
# With index
```

```
for i, fruit in enumerate(fruits):
```

```
    print(f"{i}: {fruit}")
```

```
# Advanced List Operations
```

```
def get_unique_elements(lst):
```

```
    """Return list with duplicates removed (preserving order)"""
```

```
    seen = set()
```

```
    return [x for x in lst if not (x in seen or seen.add(x))]
```

```
# List as Stack (LIFO)
```

```
stack = []
```

```
stack.append(1)    # Push
```

```
stack.append(2)
```

```
top = stack.pop()  # Pop -> 2
```

```
# List as Queue (FIFO) - Not efficient, use collections.deque
```

```
from collections import deque
```

```
queue = deque([1, 2, 3])
```

```
queue.append(4)    # Enqueue
```

```
front = queue.popleft() # Dequeue -> 1
```

```
# List Sorting with Custom Key
```

```
students = [
```

```
    ('Alice', 85),
```

```

    ('Bob', 92),
    ('Charlie', 78)
]
students.sort(key=lambda x: x[1], reverse=True) # Sort by grade descending

# Flattening Nested Lists
nested = [[1, 2], [3, 4], [5, 6]]
flattened = [num for sublist in nested for num in sublist]

```

Class Activity:

- Create a shopping list manager (add, remove, view, sort)
- Implement a simple grade book using lists
- Build a playlist manager with shuffle functionality

Module 7: Tuples & Sets

WHAT are Tuples and Sets?

- **Tuples:** Ordered, immutable sequences (cannot be changed after creation)
- **Sets:** Unordered collections of unique elements

WHEN to use each?

- **Tuples:** Fixed data, dictionary keys, returning multiple values
- **Sets:** Removing duplicates, membership testing, mathematical operations

WHERE to use them?

- **Tuples:** Coordinates, database records, configuration values
- **Sets:** Unique ID tracking, relationship operations, data cleaning

HOW to use tuples and sets?

```

# TUPLE BASICS
# Creation
empty_tuple = ()
single_tuple = (1,) # Note: comma is necessary
coordinates = (10, 20)
person = ("Alice", 25, "Engineer")
nested_tuple = ((1, 2), (3, 4))

# Tuple Unpacking
x, y = coordinates
name, age, job = person

```

```
print(f"{name} is {age} years old")
```

```
# Accessing Elements
```

```
print(coordinates[0]) # 10
```

```
print(person[-1]) # "Engineer"
```

```
print(person[1:3]) # (25, "Engineer")
```

```
# Tuple Methods
```

```
count_tuple = (1, 2, 3, 1, 1)
```

```
print(count_tuple.count(1)) # 3
```

```
print(count_tuple.index(3)) # 2
```

```
# Why use tuples?
```

```
# 1. Immutability (safety)
```

```
config = ("localhost", 8080, True)
```

```
# config[1] = 9090 # Error! Cannot change tuple
```

```
# 2. Memory efficient
```

```
import sys
```

```
a_list = [1, 2, 3, 4, 5]
```

```
a_tuple = (1, 2, 3, 4, 5)
```

```
print(sys.getsizeof(a_list)) # Larger
```

```
print(sys.getsizeof(a_tuple)) # Smaller
```

```
# 3. Can be used as dictionary keys
```

```
locations = {
```

```
    (40.7128, 74.0060): "New York",
```

```
    (34.0522, 118.2437): "Los Angeles"
```

```
}
```

```
# SET BASICS
```

```
# Creation
```

```
empty_set = set() # Not {}, that's a dictionary
```

```
numbers_set = {1, 2, 3, 4, 5}
```

```
mixed_set = {1, "hello", 3.14}
```

```
from_list = set([1, 2, 2, 3, 3, 3]) # {1, 2, 3} - duplicates removed
```

```
# Set Operations
```

```
set_a = {1, 2, 3, 4, 5}
```

```
set_b = {4, 5, 6, 7, 8}
```

```
# Union
```

```
union_set = set_a | set_b # {1, 2, 3, 4, 5, 6, 7, 8}
```

```
union_set = set_a.union(set_b) # Same
```

Intersection

```
intersection_set = set_a & set_b  # {4, 5}
intersection_set = set_a.intersection(set_b)
```

Difference

```
diff_set = set_a - set_b          # {1, 2, 3}
diff_set = set_a.difference(set_b)
```

Symmetric Difference

```
sym_diff = set_a ^ set_b          # {1, 2, 3, 6, 7, 8}
sym_diff = set_a.symmetric_difference(set_b)
```

Set Methods

```
fruits = {'apple', 'banana', 'orange'}
fruits.add('grape')                # Add element
fruits.remove('banana')            # Remove (error if not exists)
fruits.discard('mango')            # Remove (no error if not exists)
popped = fruits.pop()              # Remove and return arbitrary element
fruits.clear()                    # Empty set
```

Subset and Superset

```
small_set = {1, 2}
large_set = {1, 2, 3, 4}
print(small_set.issubset(large_set)) # True
print(large_set.issuperset(small_set)) # True
```

Set Comprehensions

```
squares = {x**2 for x in range(10)}    # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
even_squares = {x**2 for x in range(10) if x % 2 == 0}
```

Practical Examples

```
def remove_duplicates(lst):
    """Remove duplicates while preserving order"""
    seen = set()
    return [x for x in lst if not (x in seen or seen.add(x))]
```

Find common elements

```
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
common = set(list1) & set(list2)  # {4, 5}
```

Check if all elements are unique

```
def all_unique(lst):
```



```

    return len(lst) == len(set(lst))

# Comparing performance
import timeit

# Membership testing with set vs list
large_list = list(range(10000))
large_set = set(large_list)

# Set is O(1), List is O(n) for membership
print(9999 in large_set) # Very fast
print(9999 in large_list) # Slower

# FrozenSet (immutable set)
frozen = frozenset([1, 2, 3, 4, 5])
    # frozen.add(6) # Error! Cannot modify

```

Class Activity:

- Create a unique word counter using sets
- Build a coordinate system using tuples
- Implement a friendship network using set operations

STAGE 2:

CONTROL FLOW & FUNCTIONS (Modules 8-14)

Module 8: Conditional Statements (if/elif/else)

WHAT are Conditional Statements?

Conditional statements control the flow of program execution based on boolean conditions.

WHEN to use conditionals?

- Decision making in programs
- Input validation
- Branching logic
- Error handling

WHERE to use conditionals?

- User authentication
- Menu systems
- Data filtering
- Game logic

HOW to use conditionals?

Basic if Statement

age = 18

if age >= 18:

print("You can vote!")

if-else Statement

temperature = 25

if temperature > 30:

print("It's hot outside!")

else:

print("It's pleasant outside!")

if-elif-else Chain

score = 85

if score >= 90:

grade = 'A'

elif score >= 80:

grade = 'B'

elif score >= 70:

grade = 'C'

elif score >= 60:

grade = 'D'

else:

grade = 'F'

print(f"Grade: {grade}")

Nested Conditionals

age = 25

has_license = True

if age >= 18:

if has_license:

print("You can drive!")

else:

print("You need a license first!")

else:

print("You're too young to drive!")

Ternary Operator (Conditional Expression)

age = 20

status = "Adult" if age >= 18 else "Minor"

print(status)

Multiple Conditions with Logical Operators

year = 2024

is_leap = (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0)

print(f"{year} is leap year: {is_leap}")

Membership Conditions

user_role = "admin"

allowed_roles = ["admin", "moderator", "editor"]

if user_role in allowed_roles:

print("Access granted!")

else:

print("Access denied!")

Pattern Matching (Python 3.10+)

def http_status(code):

match code:

case 200:

return "OK"

case 404:

return "Not Found"

case 500:

return "Internal Server Error"

case _:

return "Unknown"

Complex Conditions

def can_rent_car(age, has_license, credit_score):

return (age >= 21 and

has_license and

credit_score >= 650)

Short-circuit Evaluation

Python stops evaluating once result is determined

x = 5

if x > 0 or (10/0): # Won't evaluate division because first condition is True

print("This is safe!")

Chained Comparisons

```
x = 5
if 0 < x < 10: # Equivalent to (0 < x) and (x < 10)
    print("x is between 0 and 10")
```

Class Activity:

- Build a BMI calculator with health recommendations
- Create a simple quiz system with multiple-choice questions
- Implement a rock-paper-scissors game

Module 9: Loops (for, while)

WHAT are Loops?

Loops allow repeated execution of code blocks until a condition is met or a sequence is exhausted.

WHEN to use different loops?

- **for loops:** Iterating over sequences, known number of iterations
- **while loops:** Unknown number of iterations, waiting for conditions

WHERE to use loops?

- Processing collections
- Retry mechanisms
- Game loops
- Data processing pipelines

HOW to use loops?

```
# FOR LOOPS
# Iterating over sequences
fruits = ['apple', 'banana', 'orange']
for fruit in fruits:
    print(fruit)

# Using range()
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8, 2): # 2, 4, 6
    print(i)

# Iterating backwards
for i in range(10, 0, -1):
```

```

    print(i)

# Enumerate for index and value
colors = ['red', 'green', 'blue']
for index, color in enumerate(colors):
    print(f"{index}: {color}")

# Zip for parallel iteration
names = ['Alice', 'Bob', 'Charlie']
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# Iterating over dictionaries
student = {'name': 'Alice', 'age': 25, 'grade': 'A'}
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(f"{key}: {value}")

# WHILE LOOPS
# Basic while loop
count = 0
while count < 5:
    print(count)
    count += 1

# User input validation
while True:
    user_input = input("Enter a number (or 'quit' to exit): ")
    if user_input.lower() == 'quit':
        break
    try:
        number = float(user_input)
        print(f"You entered: {number}")
    except ValueError:
        print("Invalid input!")

# Fibonacci sequence
a, b = 0, 1
while a < 100:
    print(a, end=' ')
    a, b = b, a + b

```

```

# Infinite loop with condition
import random
target = random.randint(1, 100)
guess = None
attempts = 0

while guess != target:
    guess = int(input("Guess the number (1-100): "))
    attempts += 1
    if guess < target:
        print("Too low!")
    elif guess > target:
        print("Too high!")
    print(f"Correct! You took {attempts} attempts.")

# Loop Optimization
# List comprehension vs for loop
numbers = [1, 2, 3, 4, 5]

# Traditional loop
squares = []
for n in numbers:
    squares.append(n**2)

# List comprehension (faster and cleaner)
squares = [n**2 for n in numbers]

# Nested Loops
# Multiplication table
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{i*j:4}", end=" ")
    print()

# Pattern printing
for i in range(1, 6):
    print('*' * i)

# Loop else clause (executes if loop completes without break)
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(f"{n} = {x} * {n//x}")

```

```

        break
    else:
        print(f"{n} is prime")

```

Class Activity:

- Create a number guessing game with difficulty levels
- Build a simple animation using loops and sleep
- Implement a shopping cart total calculator

Module 10: Nested Loops & Loop Control (break, continue, pass)

WHAT are Loop Control Statements?

- **break:** Exits the current loop entirely
- **continue:** Skips the rest of the current iteration
- **pass:** Does nothing (placeholder)
- **Nested loops:** Loops within loops

WHEN to use each?

- **break:** Found what you're looking for
- **continue:** Skip invalid items
- **pass:** Placeholder for future code
- **nested loops:** Working with 2D data

WHERE to use these?

- Search algorithms
- Data cleaning
- Game development
- Matrix operations

HOW to use loop control?

```

python
# BREAK STATEMENT
# Search for first occurrence
numbers = [1, 3, 5, 7, 9, 2, 4, 6, 8]
target = 4

for num in numbers:
    if num == target:
        print(f"Found {target} at position {numbers.index(num)}")
        break
    print(f"Checking {num}")

```

```

else:
    print(f"{target} not found")

# Breaking out of nested loops
found = False
for i in range(5):
    if found:
        break
    for j in range(5):
        if i == j == 2:
            found = True
            break
    print(f"({i},{j})")

# Better approach: use a flag or function
def find_in_matrix(matrix, target):
    for i, row in enumerate(matrix):
        for j, value in enumerate(row):
            if value == target:
                return i, j
    return None

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
result = find_in_matrix(matrix, 5)
if result:
    print(f"Found at {result}")

# CONTINUE STATEMENT
# Skip even numbers
for i in range(10):
    if i % 2 == 0:
        continue
    print(i, end=' ') # 1 3 5 7 9

# Skip invalid data
data = [10, -5, 0, 25, -3, 30, 0, 15]
positive_sum = 0
for num in data:
    if num <= 0:
        continue
    positive_sum += num
print(f"Sum of positive numbers: {positive_sum}")

# PASS STATEMENT

```



```

# Placeholder for future implementation
def process_data(data):
    pass # TODO: Implement data processing

class MyClass:
    pass # Placeholder class

# Nested Loops in Practice
# Matrix operations
def matrix_multiplication(A, B):
    """Multiply two matrices"""
    result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]

    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

# Finding common elements
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]

common = []
for item1 in list1:
    for item2 in list2:
        if item1 == item2:
            common.append(item1)
            break # Found it, no need to continue inner loop

# Set-based solution (more efficient)
common_set = set(list1) & set(list2)

# Optimizing nested loops
# Bad:  $O(n^2)$ 
def find_duplicates_slow(lst):
    duplicates = []
    for i in range(len(lst)):
        for j in range(i + 1, len(lst)):
            if lst[i] == lst[j] and lst[i] not in duplicates:
                duplicates.append(lst[i])
    return duplicates

# Good:  $O(n)$ 

```

```

def find_duplicates_fast(lst):
    seen = set()
    duplicates = set()
    for item in lst:
        if item in seen:
            duplicates.add(item)
        seen.add(item)
    return list(duplicates)

# Complex nested loop example: Generate all subsets
def generate_subsets(items):
    """Generate all subsets of a list (power set)"""
    subsets = [[]]
    for item in items:
        new_subsets = []
        for subset in subsets:
            new_subsets.append(subset + [item])
        subsets.extend(new_subsets)
    return subsets

print(generate_subsets([1, 2, 3]))

```

Class Activity:

- Create a matrix search game
- Build a prime number finder with optimization
- Implement a simple pattern recognition system

Module 11: Functions - Defining & Calling

WHAT are Functions?

Functions are reusable blocks of code that perform specific tasks. They promote code reuse, modularity, and maintainability.

WHEN to create functions?

- When code is repeated
- For complex operations that need organization
- To create reusable components
- For better testing and debugging

WHERE to use functions?

- Business logic encapsulation

- Utility libraries
- API endpoints
- Data processing pipelines

HOW to define and use functions?

Basic Function Definition

```
def greet():
    """Simple greeting function"""
    print("Hello, World!")
```

Calling a function

```
greet()
```

Functions with parameters

```
def greet_person(name):
    """Greet a specific person"""
    print(f"Hello, {name}!")
```

```
greet_person("Alice")
```

```
greet_person("Bob")
```

Functions with return values

```
def add(a, b):
    """Add two numbers and return result"""
    return a + b
```

```
result = add(5, 3)
```

```
print(f"5 + 3 = {result}")
```

Functions with multiple return values

```
def get_min_max(numbers):
    """Return both minimum and maximum values"""
    return min(numbers), max(numbers)
```

```
min_val, max_val = get_min_max([1, 5, 3, 9, 2])
```

```
print(f"Min: {min_val}, Max: {max_val}")
```

Default parameters

```
def greet_with_title(name, title="Mr."):
    """Greet with optional title"""
    print(f"Hello, {title} {name}")
```

```
greet_with_title("Smith")      # Uses default title
```

```
greet_with_title("Johnson", "Dr.") # Custom title
```

```

# Keyword arguments (order doesn't matter)
def introduce(name, age, city):
    print(f"I'm {name}, {age} years old from {city}")

introduce(city="New York", age=25, name="Alice")

# Variable number of arguments
def sum_all(*args):
    """Sum any number of arguments"""
    return sum(args)

print(sum_all(1, 2, 3, 4, 5)) # 15
print(sum_all(10, 20))       # 30

# Variable number of keyword arguments
def print_info(**kwargs):
    """Print any number of keyword arguments"""
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, job="Engineer")

# Combining parameter types
def complex_function(a, b, *args, c=10, **kwargs):
    """Demonstrate parameter ordering"""
    print(f"a: {a}, b: {b}")
    print(f"args: {args}")
    print(f"c: {c}")
    print(f"kwargs: {kwargs}")

complex_function(1, 2, 3, 4, 5, c=20, name="Alice", city="NYC")

# Type hints (Python 3.5+)
def calculate_area(radius: float) -> float:
    """Calculate circle area with type hints"""
    import math
    return math.pi * radius ** 2

# Documentation strings (docstrings)
def factorial(n):
    """
    Calculate factorial of a number.

```

Args:
n (int): Non-negative integer

Returns:
int: Factorial of n

Raises:
ValueError: If n is negative
"""
if n < 0:
raise ValueError("n must be non-negative")
if n == 0:
return 1
*return n * factorial(n - 1)*

Function as first-class citizens
def apply_operation(func, x, y):
"""Apply any function to two arguments"""
return func(x, y)

def multiply(a, b):
*return a * b*

result = apply_operation(multiply, 5, 3) # 15

Nested functions and closures
def outer_function(x):
def inner_function(y):
return x + y
return inner_function

add_five = outer_function(5)
print(add_five(3)) # 8
print(add_five(10)) # 15

Recursion
def fibonacci(n):
"""Recursive Fibonacci (inefficient for large n)"""
if n <= 1:
return n
return fibonacci(n-1) + fibonacci(n-2)

Better recursive Fibonacci with memoization
def fibonacci_memo(n, memo={}):

```

if n in memo:
    return memo[n]
if n <= 1:
    return n
memo[n] = fibonacci_memo(n-1, memo) + fibonacci_memo(n-2, memo)
return memo[n]

print(fibonacci_memo(10)) # 55

```

Class Activity:

- Create a calculator with separate functions for each operation
- Build a unit converter with multiple conversion functions
- Implement a function that validates email addresses

Module 12: Function Arguments (*args, **kwargs)

WHAT are *args and **kwargs?

- ***args**: Collects variable positional arguments as a tuple
- ****kwargs**: Collects variable keyword arguments as a dictionary

WHEN to use them?

- Creating flexible APIs
- Decorators and wrappers
- Function composition
- When argument count is unknown

WHERE to use them?

- Framework development
- Plugin systems
- Wrapper functions
- Configuration handling

HOW to use *args and **kwargs?

```

# *args - variable positional arguments
def print_all(*args):
    """Print all arguments passed"""
    print(f'Arguments: {args}')
    print(f'Type: {type(args)}')
    print(f'Count: {len(args)}')

    for i, arg in enumerate(args):

```

```

    print(f"arg{i}: {arg}")

print_all(1, 2, 3, "hello", [1, 2])
# Arguments: (1, 2, 3, 'hello', [1, 2])

# Using *args for flexible functions
def calculate_average(*args):
    """Calculate average of any number of numbers"""
    if not args:
        return 0
    return sum(args) / len(args)

print(calculate_average(1, 2, 3, 4, 5)) # 3.0
print(calculate_average(10, 20))       # 15.0
print(calculate_average())              # 0

# **kwargs - variable keyword arguments
def print_kwargs(**kwargs):
    """Print all keyword arguments"""
    print(f"Keyword arguments: {kwargs}")
    print(f"Type: {type(kwargs)}")

    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_kwargs(name="Alice", age=25, city="New York")
# Keyword arguments: {'name': 'Alice', 'age': 25, 'city': 'New York'}

# Creating flexible configuration functions
def configure(**kwargs):
    """Apply configuration settings"""
    config = {
        'debug': False,
        'timeout': 30,
        'max_retries': 3
    }
    config.update(kwargs)
    return config

settings = configure(debug=True, timeout=60, custom_option="value")
print(settings)

# Combining *args and **kwargs
def universal_function(*args, **kwargs):

```

```

"""Function that accepts anything"""
print(f"Positional args: {args}")
print(f"Keyword args: {kwargs}")

# Unpacking arguments
def sum_three(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
print(sum_three(*numbers)) # Unpack list -> 6

def describe_person(name, age, city):
    return f"{name} is {age} from {city}"

person_info = {'name': 'Alice', 'age': 25, 'city': 'New York'}
print(describe_person(**person_info)) # Unpack dict

# Decorators with *args and **kwargs
def timer(func):
    """Decorator that measures execution time"""
    import time

    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function(duration):
    import time
    time.sleep(duration)

slow_function(1.5)

# Practical example: Data processor
def process_data(*args, **kwargs):
    """
    Process data with flexible inputs

    Args:
        *args: Data items to process
    """

```



```

    **kwargs: Processing options
    """
    operation = kwargs.get('operation', 'sum')
    filter_min = kwargs.get('filter_min', None)

    # Convert args to list for processing
    data = list(args)

    # Filter if needed
    if filter_min is not None:
        data = [x for x in data if x >= filter_min]

    # Apply operation
    if operation == 'sum':
        return sum(data)
    elif operation == 'average':
        return sum(data) / len(data) if data else 0
    elif operation == 'count':
        return len(data)
    else:
        return data

# Different ways to call
print(process_data(1, 2, 3, 4, 5))          # 15
print(process_data(1, 2, 3, 4, 5, operation='average')) # 3.0
print(process_data(1, 2, 3, 4, 5, filter_min=3))    # 12 (3+4+5)
print(process_data(1, 2, 3, 4, 5, operation='count', filter_min=3)) # 3

# Class methods with *args, **kwargs
class FlexibleClass:
    def __init__(self, *args, **kwargs):
        self.args = args
        self.kwargs = kwargs
        print(f"Initialized with {len(args)} positional and {len(kwargs)} keyword args")

    def method(self, *args, **kwargs):
        print(f"Method called with {args} and {kwargs}")

# Function composition with *args
def compose(*functions):
    """Compose multiple functions from right to left"""
    def composed_function(x):
        result = x
        for func in reversed(functions):

```

```

        result = func(result)
    return result
return composed_function

def double(x): return x * 2
def add_one(x): return x + 1
def square(x): return x ** 2

composed = compose(square, add_one, double)
print(composed(5)) # double(5)=10, add_one(10)=11, square(11)=121

```

Class Activity:

- Create a flexible logging system using *args and **kwargs
- Build a function that can accept any type of data and format it
- Implement a configuration system using **kwargs

Module 13: Return Values & Scope

WHAT are Return Values and Scope?

- **Return values:** Data passed back from functions
- **Scope:** Region where variables are accessible (local, global, nonlocal)

WHEN to consider scope?

- Managing variable lifetime
- Preventing naming conflicts
- Creating closures
- Working with nested functions

WHERE to use scope?

- Module organization
- Function factories
- State management
- Recursive algorithms

HOW to work with return values and scope?

```

# RETURN VALUES
# Returning single value
def square(x):
    return x ** 2

# Returning multiple values (as tuple)

```

```
def get_stats(numbers):  
    return min(numbers), max(numbers), sum(numbers) / len(numbers)
```

```
min_val, max_val, avg = get_stats([1, 2, 3, 4, 5])
```

```
# Returning early (multiple returns)
```

```
def classify_number(n):  
    if n > 0:  
        return "positive"  
    if n < 0:  
        return "negative"  
    return "zero"
```

```
# Returning None (implicitly or explicitly)
```

```
def do_nothing():  
    pass # Returns None
```

```
def return_none():  
    return None # Explicit None
```

```
# Returning from loops
```

```
def find_first_even(numbers):  
    for num in numbers:  
        if num % 2 == 0:  
            return num  
    return None # No even found
```

```
# SCOPE
```

```
# Global scope  
global_var = "I'm global"
```

```
def function_scope():  
    local_var = "I'm local"  
    print(global_var) # Can access global  
    print(local_var) # Can access local
```

```
# Local variables
```

```
def modify_local():  
    x = 10 # Local variable  
    x = x + 5  
    print(f"Inside function: {x}")
```

```
x = 100 # Global variable  
modify_local()
```

```
print(f"Outside: {x}") # Still 100
```

```
# Accessing global variables
```

```
def modify_global():  
    global global_var # Declare intent to modify global  
    global_var = "Modified global"
```

```
modify_global()  
print(global_var) # "Modified global"
```

```
# Nonlocal scope (nested functions)
```

```
def outer_function():  
    outer_var = "outer"  
  
    def inner_function():  
        nonlocal outer_var # Modify variable from outer scope  
        outer_var = "modified by inner"  
        print(f"Inner: {outer_var}")  
  
    inner_function()  
    print(f"Outer after: {outer_var}")
```

```
outer_function()
```

```
# LEGB Rule (Local, Enclosing, Global, Built-in)
```

```
x = "global"
```

```
def outer():  
    x = "enclosing"
```

```
    def inner():  
        x = "local"  
        print(x) # Local
```

```
    inner()  
    print(x) # Enclosing
```

```
outer()  
print(x) # Global
```

```
# Scope with list/dict modification (no need for global)
```

```
my_list = [1, 2, 3]
```

```
def modify_list():
```

```

    my_list.append(4) # Works! Modifying, not reassigning
    # my_list = [1, 2, 3, 4] # Would create local variable

modify_list()
print(my_list) # [1, 2, 3, 4]

# Returning multiple values with named tuples
from collections import namedtuple

Person = namedtuple('Person', ['name', 'age', 'city'])
def get_person():
    return Person(name="Alice", age=25, city="New York")

person = get_person()
print(person.name, person.age)

# Return values with type hints
def process_data(data: list) -> dict:
    """Process data and return structured result"""
    return {
        'count': len(data),
        'sum': sum(data),
        'average': sum(data) / len(data) if data else 0
    }

# Advanced: Returning functions (closures)
def make_multiplier(factor):
    """Return a function that multiplies by factor"""
    def multiplier(x):
        return x * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5)) # 10
print(triple(5)) # 15

# Scope in comprehensions (Python 3)
# List comprehension creates its own scope
x = "global"
squares = [x for x in range(5)] # x in comprehension is local
print(x) # Still "global"
print(squares) # [0, 1, 2, 3, 4]

```

```

# Dictionary with return values
def safe_divide(a, b):
    """Return result and error status"""
    if b == 0:
        return None, "Division by zero"
    return a / b, None

result, error = safe_divide(10, 2)
if error:
    print(f"Error: {error}")
else:
    print(f"Result: {result}")

```

Class Activity:

- Create a function that returns multiple statistics about a list
- Build a counter using closures and nonlocal
- Implement a function that modifies global configuration safely

Module 14: Lambda Functions & Map/Filter/Reduce

WHAT are Lambda Functions?

Lambda functions are small, anonymous functions defined with the `lambda` keyword. They can have any number of arguments but only one expression.

WHEN to use lambda and functional tools?

- For short, throwaway functions
- With higher-order functions (map, filter, reduce)
- For simple transformations
- When you need a function for a short time

WHERE to use them?

- Data transformation pipelines
- Sorting with custom keys
- Event handlers
- Quick calculations

HOW to use lambda and functional programming?

```

# LAMBDA BASICS
# Regular function
def square(x):

```

```

    return x ** 2

# Lambda equivalent
square_lambda = lambda x: x ** 2

print(square(5))      # 25
print(square_lambda(5)) # 25

# Multiple arguments
add = lambda x, y: x + y
print(add(3, 5)) # 8

# Conditional expression in lambda
max_value = lambda x, y: x if x > y else y
print(max_value(10, 20)) # 20

# MAP FUNCTION
# Apply function to all items in iterable
numbers = [1, 2, 3, 4, 5]

# Square all numbers using map with lambda
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # [1, 4, 9, 16, 25]

# Map with multiple iterables
list1 = [1, 2, 3]
list2 = [10, 20, 30]
sums = list(map(lambda x, y: x + y, list1, list2))
print(sums) # [11, 22, 33]

# FILTER FUNCTION
# Filter items based on condition
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Filter even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6, 8, 10]

# Filter words with length > 3
words = ["cat", "dog", "elephant", "bird", "ant"]
long_words = list(filter(lambda w: len(w) > 3, words))
print(long_words) # ['elephant', 'bird']

# REDUCE FUNCTION

```

```
from functools import reduce
```

```
# Calculate product of all numbers
```

```
numbers = [1, 2, 3, 4, 5]
```

```
product = reduce(lambda x, y: x * y, numbers)
```

```
print(product) # 120
```

```
# Find maximum value
```

```
max_num = reduce(lambda x, y: x if x > y else y, numbers)
```

```
print(max_num) # 5
```

```
# Chain string concatenation
```

```
words = ["Hello", "World", "Python", "Rocks"]
```

```
sentence = reduce(lambda x, y: x + " " + y, words)
```

```
print(sentence) # "Hello World Python Rocks"
```

```
# PRACTICAL EXAMPLES
```

```
# 1. Data cleaning
```

```
data = [" hello ", "WORLD", "Python ", " CODE"]
```

```
cleaned = list(map(lambda s: s.strip().lower(), data))
```

```
print(cleaned) # ['hello', 'world', 'python', 'code']
```

```
# 2. Filtering and transforming combined
```

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
result = list(map(lambda x: x * 2,
```

```
                filter(lambda x: x % 2 == 0, numbers)))
```

```
print(result) # [4, 8, 12, 16, 20]
```

```
# 3. Dictionary transformations
```

```
students = [
```

```
    {'name': 'Alice', 'grade': 85},
```

```
    {'name': 'Bob', 'grade': 92},
```

```
    {'name': 'Charlie', 'grade': 78}
```

```
]
```

```
# Extract names
```

```
names = list(map(lambda s: s['name'], students))
```

```
print(names) # ['Alice', 'Bob', 'Charlie']
```

```
# Filter students with grade >= 80
```

```
top_students = list(filter(lambda s: s['grade'] >= 80, students))
```

```
print(top_students)
```

```
# 4. Sorting with lambda
```



```

products = [
    {'name': 'Laptop', 'price': 999},
    {'name': 'Mouse', 'price': 25},
    {'name': 'Monitor', 'price': 300}
]

# Sort by price
sorted_products = sorted(products, key=lambda x: x['price'])
print(sorted_products)

# Sort by name length
words = ['python', 'java', 'c', 'javascript', 'go']
sorted_words = sorted(words, key=lambda x: len(x))
print(sorted_words) # ['c', 'go', 'java', 'python', 'javascript']

# 5. Grouping data
from itertools import groupby

# Group numbers by parity
numbers = [1, 2, 3, 4, 5, 6]
grouped = groupby(sorted(numbers), key=lambda x: x % 2)
for key, group in grouped:
    print(f"{ 'Even' if key == 0 else 'Odd': } {list(group)}")

# 6. List comprehension vs map/filter
# Using list comprehension (often more readable)
squares_lc = [x**2 for x in range(10)]
evens_lc = [x for x in range(20) if x % 2 == 0]

# Using map/filter
squares_mf = list(map(lambda x: x**2, range(10)))
evens_mf = list(filter(lambda x: x % 2 == 0, range(20)))

# 7. Advanced: Partial application with lambda
def power(base):
    return lambda exponent: base ** exponent

square = power(2)
cube = power(3)

print(square(4)) # 16
print(cube(3)) # 27

# 8. Performance comparison

```

```

import timeit

# List comprehension
lc_time = timeit.timeit('[x**2 for x in range(1000)]', number=1000)

# Map with lambda
map_time = timeit.timeit('list(map(lambda x: x**2, range(1000)))', number=1000)

print(f"List comp: {lc_time:.4f}s")
print(f"Map: {map_time:.4f}s")

# 9. Complex data transformation
data = [
    {'name': 'Alice', 'grades': [85, 90, 88]},
    {'name': 'Bob', 'grades': [92, 85, 89]},
    {'name': 'Charlie', 'grades': [78, 82, 80]}
]

# Calculate averages
averages = list(map(
    lambda s: {'name': s['name'], 'avg': sum(s['grades'])/len(s['grades'])},
    data
))
print(averages)

# 10. Custom functional pipeline
def pipeline(data, *functions):
    """Apply multiple functions in sequence"""
    for func in functions:
        data = func(data)
    return data

# Usage
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
result = pipeline(
    numbers,
    lambda x: filter(lambda y: y % 2 == 0, x), # Keep evens
    lambda x: map(lambda y: y * 2, x),        # Double them
    list                                       # Convert to list
)
print(result) # [4, 8, 12, 16, 20]

```

Class Activity:

- Create a data processing pipeline for student grades using map/filter/reduce
- Build a custom sorting system using lambda functions
- Implement a text analysis tool that counts word frequencies

STAGE 3:

INTERMEDIATE PYTHON (Modules 15-21)

Module 15: Dictionaries & Dictionary Methods

WHAT are Dictionaries?

Dictionaries are unordered, mutable collections of key-value pairs. They provide $O(1)$ average access time.

WHEN to use dictionaries?

- Storing structured data
- Fast lookups by key
- Counting occurrences
- Caching/memoization

WHERE to use dictionaries?

- Configuration settings
- Database-like structures
- API responses
- JSON data handling

HOW to work with dictionaries?

```
# DICTIONARY CREATION
```

```
# Empty dictionary
```

```
empty_dict = {}
```

```
empty_dict2 = dict()
```

```
# Pre-populated dictionary
```

```
person = {
```

```
    'name': 'Alice',
```

```
    'age': 25,
```

```

    'city': 'New York'
}

# Using dict() constructor
person2 = dict(name='Bob', age=30, city='Los Angeles')

# From list of tuples
pairs = [('name', 'Charlie'), ('age', 35), ('city', 'Chicago')]
person3 = dict(pairs)

# Dictionary comprehension
squares = {x: x**2 for x in range(5)}
print(squares) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# ACCESSING VALUES
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}

# Direct access (raises KeyError if key missing)
print(person['name']) # Alice

# Using get() (returns None or default if key missing)
print(person.get('age'))      # 25
print(person.get('country'))  # None
print(person.get('country', 'USA')) # USA

# Access all keys, values, items
print(person.keys()) # dict_keys(['name', 'age', 'city'])
print(person.values()) # dict_values(['Alice', 25, 'New York'])
print(person.items()) # dict_items([('name', 'Alice'), ('age', 25), ('city', 'New York')])

# MODIFYING DICTIONARIES
# Add or update
person['email'] = 'alice@example.com' # Add new key
person['age'] = 26                    # Update existing

# Update multiple values
person.update({'city': 'Boston', 'phone': '123-456-7890'})

# Remove items
del person['phone'] # Remove by key
age = person.pop('age') # Remove and return value
last_item = person.popitem() # Remove and return last item
person.clear() # Empty dictionary

```

DICTIONARY METHODS

Set default value

```
person = {'name': 'Alice'}
person.setdefault('age', 25) # Adds if key doesn't exist
print(person) # {'name': 'Alice', 'age': 25}
```

Create dictionary with default values

```
from collections import defaultdict
```

Default dictionary

```
word_count = defaultdict(int) # Default value is 0
words = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
for word in words:
    word_count[word] += 1
print(dict(word_count)) # {'apple': 3, 'banana': 2, 'orange': 1}
```

Counter (specialized dictionary)

```
from collections import Counter
word_counter = Counter(words)
print(word_counter) # Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

Merging dictionaries (Python 3.9+)

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged = dict1 | dict2 # {'a': 1, 'b': 2, 'c': 3, 'd': 4}
dict1 |= dict2 # Update dict1 with dict2
```

NESTED DICTIONARIES

```
students = {
    'student1': {
        'name': 'Alice',
        'grades': [85, 90, 88],
        'contact': {'email': 'alice@example.com', 'phone': '123-456'}
    },
    'student2': {
        'name': 'Bob',
        'grades': [92, 85, 89],
        'contact': {'email': 'bob@example.com', 'phone': '789-012'}
    }
}
```

Access nested data

```
print(students['student1']['name']) # Alice
print(students['student2']['grades'][1]) # 85
```

ITERATING THROUGH DICTIONARIES

```
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

```
# Iterate through keys
```

```
for key in person:
```

```
    print(key, person[key])
```

```
# Iterate through values
```

```
for value in person.values():
```

```
    print(value)
```

```
# Iterate through key-value pairs
```

```
for key, value in person.items():
```

```
    print(f"{key}: {value}")
```

DICTIONARY COMPREHENSIONS

```
# Filter dictionary
```

```
grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78, 'David': 88}
```

```
high_grades = {k: v for k, v in grades.items() if v >= 85}
```

```
print(high_grades) # {'Alice': 85, 'Bob': 92, 'David': 88}
```

```
# Transform values
```

```
incremented = {k: v + 5 for k, v in grades.items()}
```

```
print(incremented) # {'Alice': 90, 'Bob': 97, 'Charlie': 83, 'David': 93}
```

```
# Swap keys and values
```

```
swapped = {v: k for k, v in grades.items()}
```

```
print(swapped) # {85: 'Alice', 92: 'Bob', 78: 'Charlie', 88: 'David'}
```

PRACTICAL EXAMPLES

```
# 1. Counting occurrences
```

```
def count_frequencies(items):
```

```
    """Count frequency of each item"""
```

```
    freq = {}
```

```
    for item in items:
```

```
        freq[item] = freq.get(item, 0) + 1
```

```
    return freq
```

```
text = "the quick brown fox jumps over the lazy dog"
```

```
words = text.split()
```

```
freq = count_frequencies(words)
```

```
print(freq)
```

2. Grouping data

```
def group_by_key(data, key_func):
    """Group items by a key function"""
    groups = {}
    for item in data:
        key = key_func(item)
        groups.setdefault(key, []).append(item)
    return groups

people = [
    {'name': 'Alice', 'age': 25},
    {'name': 'Bob', 'age': 30},
    {'name': 'Charlie', 'age': 25},
    {'name': 'David', 'age': 30}
]
grouped = group_by_key(people, lambda p: p['age'])
print(grouped) # {25: [{'name': 'Alice'}, {'name': 'Charlie'}], 30: [...]}
```

3. Cache implementation

```
class SimpleCache:
    def __init__(self, max_size=100):
        self.cache = {}
        self.max_size = max_size

    def get(self, key):
        return self.cache.get(key)

    def set(self, key, value):
        if len(self.cache) >= self.max_size:
            # Remove oldest item (simplified)
            self.cache.pop(next(iter(self.cache)))
        self.cache[key] = value
```

4. Deep dictionary merging

```
def deep_merge(dict1, dict2):
    """Recursively merge two dictionaries"""
    result = dict1.copy()
    for key, value in dict2.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, dict):
            result[key] = deep_merge(result[key], value)
        else:
            result[key] = value
    return result
```

5. JSON serialization

```
import json
```

```
# Convert dictionary to JSON
```

```
data = {'name': 'Alice', 'age': 25, 'hobbies': ['reading', 'coding']}
```

```
json_string = json.dumps(data, indent=2)
```

```
print(json_string)
```

```
# Parse JSON to dictionary
```

```
parsed_data = json.loads(json_string)
```

```
print(parsed_data)
```

Class Activity:

- Create a phone book application using dictionaries
- Build a word frequency counter for any text
- Implement a simple in-memory database with CRUD operations

Module 16: List Comprehensions & Generators

WHAT are List Comprehensions and Generators?

- **List comprehensions:** Concise way to create lists from iterables
- **Generators:** Lazy iterators that yield values one at a time

WHEN to use each?

- **List comprehensions:** When you need a complete list
- **Generators:** When dealing with large datasets or infinite sequences

WHERE to use them?

- Data transformation pipelines
- Processing large files
- Streaming data
- Memory-efficient operations

HOW to use comprehensions and generators?

```
# LIST COMPREHENSIONS
```

```
# Basic syntax: [expression for item in iterable if condition]
```

```
# Simple list comprehension
```

```
squares = [x**2 for x in range(10)]
```

```
print(squares) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```



```

# With condition
evens = [x for x in range(20) if x % 2 == 0]
print(evens) # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Nested list comprehension
matrix = [[j for j in range(5)] for i in range(3)]
print(matrix) # [[0,1,2,3,4], [0,1,2,3,4], [0,1,2,3,4]]

# Flatten a matrix
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Multiple conditions
numbers = [x for x in range(100) if x % 2 == 0 and x % 5 == 0]
print(numbers) # [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

# SET COMPREHENSIONS
unique_squares = {x**2 for x in range(10)}
print(unique_squares) # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}

# DICTIONARY COMPREHENSIONS
square_dict = {x: x**2 for x in range(5)}
print(square_dict) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# GENERATOR EXPRESSIONS
# Syntax: (expression for item in iterable if condition)

# Generator expression (uses parentheses)
gen = (x**2 for x in range(10))
print(type(gen)) # <class 'generator'>

# Lazy evaluation - values generated on demand
for value in gen:
    print(value, end=' ') # 0 1 4 9 16 25 36 49 64 81

# Memory efficiency comparison
import sys

list_comp = [x**2 for x in range(1000000)]
gen_expr = (x**2 for x in range(1000000))

print(f"List size: {sys.getsizeof(list_comp)} bytes")
print(f"Generator size: {sys.getsizeof(gen_expr)} bytes")

```

GENERATOR FUNCTIONS

```
def countdown(n):  
    """Generator that counts down from n to 0"""  
    while n >= 0:  
        yield n  
        n -= 1
```

Using generator

```
for num in countdown(5):  
    print(num, end=' ') # 5 4 3 2 1 0
```

Fibonacci generator

```
def fibonacci():  
    """Infinite Fibonacci generator"""  
    a, b = 0, 1  
    while True:  
        yield a  
        a, b = b, a + b
```

Take first 10 Fibonacci numbers

```
fib = fibonacci()  
first_10 = [next(fib) for _ in range(10)]  
print(first_10) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Reading large files line by line

```
def read_large_file(file_path):  
    """Generator that yields lines from a large file"""  
    with open(file_path, 'r') as file:  
        for line in file:  
            yield line.strip()
```

Process file without loading entire file into memory

```
# for line in read_large_file('large_file.txt'):  
#     process(line)
```

ADVANCED GENERATOR PATTERNS

Generator with send()

```
def accumulator():  
    """Generator that accumulates values sent to it"""  
    total = 0  
    while True:  
        value = yield total  
        if value is not None:
```

```

        total += value

acc = accumulator()
next(acc) # Initialize generator
print(acc.send(10)) # 10
print(acc.send(20)) # 30

# Generator with throw() and close()
def gen_with_exception():
    try:
        yield 1
        yield 2
    except Exception as e:
        yield f"Caught: {e}"
    finally:
        yield "Done"

g = gen_with_exception()
print(next(g)) # 1
print(g.throw(ValueError("Oops"))) # "Caught: Oops"
print(next(g)) # "Done"

# PIPELINES WITH GENERATORS
def read_numbers(file_path):
    """Read numbers from file"""
    with open(file_path, 'r') as f:
        for line in f:
            yield int(line.strip())

def filter_even(numbers):
    """Filter even numbers"""
    for num in numbers:
        if num % 2 == 0:
            yield num

def square(numbers):
    """Square numbers"""
    for num in numbers:
        yield num ** 2

def sum_all(numbers):
    """Sum all numbers"""
    total = 0
    for num in numbers:

```

```

        total += num
    return total

# Create pipeline
# numbers = read_numbers('data.txt')
# evens = filter_even(numbers)
# squares = square(evens)
# result = sum_all(squares)

# PERFORMANCE COMPARISON
import timeit

def test_list_comprehension():
    return [x**2 for x in range(10000) if x % 2 == 0]

def test_generator_expression():
    return (x**2 for x in range(10000) if x % 2 == 0)

# Time creation
lc_time = timeit.timeit('test_list_comprehension()', globals=globals(), number=1000)
gen_time = timeit.timeit('test_generator_expression()', globals=globals(), number=1000)

print(f"List comprehension: {lc_time:.4f}s")
print(f"Generator expression: {gen_time:.4f}s")

# PRACTICAL EXAMPLES
# 1. CSV data processing
def process_csv(file_path):
    """Process CSV file line by line"""
    import csv
    with open(file_path, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            # Transform data
            yield {
                'name': row['name'].strip().upper(),
                'age': int(row['age']),
                'city': row['city'].strip().title()
            }

# 2. Infinite sequences
def prime_numbers():
    """Generate prime numbers infinitely"""
    def is_prime(n):

```

```

    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

n = 2
while True:
    if is_prime(n):
        yield n
    n += 1

# Get first 10 primes
primes = prime_numbers()
first_10_primes = [next(primes) for _ in range(10)]
print(first_10_primes) # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

# 3. Pagination using generators
def paginate(data, page_size):
    """Generator that yields pages of data"""
    for i in range(0, len(data), page_size):
        yield data[i:i + page_size]

data = list(range(100))
pages = paginate(data, 10)
for page_num, page in enumerate(pages, 1):
    print(f"Page {page_num}: {page[:5]}...")

# 4. Generator for data streaming
def data_stream(sample_rate=1.0):
    """Simulate streaming data"""
    import random
    import time

    while True:
        # Generate random data point
        value = {
            'timestamp': time.time(),
            'value': random.random() * 100,
            'status': random.choice(['normal', 'warning', 'error'])
        }
        yield value
        time.sleep(1 / sample_rate)

```

```
# Process stream
# for data_point in data_stream(10):
#     if data_point['status'] == 'error':
#         print(f"Alert! {data_point}")
```

Class Activity:

- Create a data processing pipeline for a large CSV file using generators
- Build an infinite sequence generator (primes, Fibonacci, etc.)
- Implement a pagination system using generators

Module 17: Modules & Importing Libraries

WHAT are Modules?

Modules are Python files containing reusable code (functions, classes, variables).
Libraries are collections of modules.

WHEN to create modules?

- Organizing related code
- Reusing code across projects
- Creating packages for distribution
- Separating concerns in large projects

WHERE to use modules?

- Standard library
- Third-party packages
- Custom project organization
- Plugin systems

HOW to create and use modules?

```
# MODULE CREATION
# Create a file called 'math_utils.py':

"""
Math Utilities Module

This module provides various mathematical utility functions.
"""

def add(a, b):
```

```

    """Add two numbers"""
    return a + b

def subtract(a, b):
    """Subtract b from a"""
    return a - b

def multiply(a, b):
    """Multiply two numbers"""
    return a * b

def divide(a, b):
    """Divide a by b"""
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

# Module-level variables
PI = 3.14159
E = 2.71828

# This runs when module is imported
print("math_utils module loaded")

if __name__ == "__main__":
    # This code only runs when executed directly
    print("Testing math_utils")
    print(add(5, 3))
    print(multiply(5, 3))

# USING MODULES
# Import entire module
import math_utils

# Use functions
print(math_utils.add(5, 3))      # 8
print(math_utils.multiply(5, 3)) # 15
print(math_utils.PI)             # 3.14159

# Import specific items
from math_utils import add, subtract, PI

print(add(10, 5))    # 15
print(subtract(10, 5)) # 5

```

```

print(Pi)          # 3.14159

# Import with alias
import math_utils as mu

print(mu.add(5, 3))
print(mu.divide(10, 2))

# Import all items (not recommended)
from math_utils import *

# STANDARD LIBRARY MODULES
import math
import random
import datetime
import os
import sys
import json
import re

# Math operations
print(math.sqrt(16))    # 4.0
print(math.pow(2, 3))   # 8.0
print(math.factorial(5)) # 120
print(math.sin(math.pi/2)) # 1.0

# Random numbers
print(random.random())    # Random float 0-1
print(random.randint(1, 10)) # Random integer 1-10
print(random.choice(['a', 'b', 'c'])) # Random choice
random.shuffle([1, 2, 3, 4, 5]) # Shuffle list

# Date and time
now = datetime.datetime.now()
print(now)          # Current date and time
print(now.year, now.month) # Year and month
print(now.strftime("%Y-%m-%d")) # Format date

# Operating system
print(os.getcwd())    # Current working directory
print(os.listdir('.')) # List directory contents
os.makedirs('new_dir', exist_ok=True) # Create directory

# System information

```



```
print(sys.version)      # Python version
print(sys.platform)     # Operating system
print(sys.path)         # Module search path
```

```
# Regular expressions
pattern = re.compile(r'\d+')
result = pattern.findall('There are 123 numbers 456 here')
print(result) # ['123', '456']
```

THIRD-PARTY MODULES

Install with pip: `pip install requests numpy pandas`

Requests for HTTP requests
`import requests`

```
response = requests.get('https://api.github.com')
if response.status_code == 200:
    data = response.json()
    print(data['message'])
```

CREATE YOUR OWN PACKAGE

Directory structure:

```
# mypackage/
# __init__.py
# module1.py
# module2.py
# subpackage/
#   __init__.py
#   module3.py
```

`__init__.py` can contain package initialization code
It can also define what gets imported with `"from package import *"`

```
# __all__ = ['module1', 'module2'] # In __init__.py
```

Importing from packages
`from mypackage import module1`
`from mypackage.subpackage import module3`

MODULE SEARCH PATH

`import sys`

Python looks for modules in:
1. Current directory

```
# 2. PYTHONPATH environment variable
# 3. Standard library directories
# 4. Site-packages directories
```

```
# Add custom path
sys.path.append('/path/to/your/modules')
```

```
# RELOADING MODULES
import importlib
import math_utils
```

```
# If module changes, reload it
importlib.reload(math_utils)
```

```
# CREATING EXECUTABLE MODULES
# In your module:
if __name__ == '__main__':
    # Code that runs when executed directly
    # Not when imported
    main()
```

```
# This allows modules to be both imported and executed
# python math_utils.py # Runs the test code
# import math_utils    # Just imports, doesn't run test code
```

```
# PRACTICAL EXAMPLES
```

```
# 1. Configuration module (config.py)
"""
```

```
Configuration settings for the application
"""
```

```
import os
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
class Config:
    DEBUG = os.getenv('DEBUG', 'False') == 'True'
    DATABASE_URL = os.getenv('DATABASE_URL', 'sqlite:///app.db')
    SECRET_KEY = os.getenv('SECRET_KEY', 'dev-key-123')
    API_KEY = os.getenv('API_KEY')
```

```
# 2. Utility module (string_utils.py)
def capitalize_words(text):
```

```

"""Capitalize each word in a string"""
return ' '.join(word.capitalize() for word in text.split())

def reverse_string(text):
    """Reverse a string"""
    return text[::-1]

def is_palindrome(text):
    """Check if string is palindrome"""
    cleaned = ''.join(c.lower() for c in text if c.isalnum())
    return cleaned == cleaned[::-1]

# 3. Package structure for a data processing library
# data_processor/
#   __init__.py
#   core.py
#   transforms.py
#   validators.py
#   utils.py

# In __init__.py:
"""
Data Processing Library
"""

from .core import DataProcessor
from .transforms import normalize, standardize
from .validators import validate_data

__version__ = '1.0.0'
__all__ = ['DataProcessor', 'normalize', 'standardize', 'validate_data']

# 4. Dynamic imports
def load_processor(processor_name):
    """Dynamically load a processor module"""
    try:
        module = __import__(f'processors.{processor_name}', fromlist=['process'])
        return module.process
    except ImportError:
        print(f"Processor {processor_name} not found")
        return None

```

Class Activity:

- Create a utility module with common helper functions
- Build a package for data validation

- Write a script that uses multiple standard library modules

Module 18: File Handling (Read/Write)

WHAT is File Handling?

File handling allows Python programs to read from and write to files on the filesystem.

WHEN to use file operations?

- Reading configuration files
- Processing log files
- Saving user data
- Exporting/importing data

WHERE to use file handling?

- Data persistence
- Logging systems
- Data import/export
- Backup systems

HOW to handle files?

BASIC FILE OPERATIONS

Opening files

Modes: r (read), w (write), a (append), x (create)

Add b for binary mode (rb, wb), + for read/write (r+)

Reading files

with open('example.txt', 'r') as file:

content = file.read()

print(content)

Reading line by line

with open('example.txt', 'r') as file:

for line in file:

print(line.strip())

Reading specific number of characters

with open('example.txt', 'r') as file:

first_10 = file.read(10) # Read first 10 characters

print(first_10)

Writing files

with open('output.txt', 'w') as file:

file.write('Hello, World!\n')

file.write('Second line\n')

Appending to files

with open('output.txt', 'a') as file:

file.write('Appended line\n')

Reading all lines into a list

with open('example.txt', 'r') as file:

lines = file.readlines()

print(lines)

FILE POSITION

with open('example.txt', 'r') as file:

print(file.tell()) # Current position

content = file.read(5)

print(content)

print(file.tell()) # New position

file.seek(0) # Go back to beginning

print(file.read(5))

WORKING WITH DIFFERENT FILE TYPES

CSV Files

import csv

Write CSV

data = [

['Name', 'Age', 'City'],

['Alice', 25, 'New York'],

['Bob', 30, 'Los Angeles'],

['Charlie', 35, 'Chicago']

]

with open('people.csv', 'w', newline='') as file:

writer = csv.writer(file)

writer.writerows(data)

Read CSV

with open('people.csv', 'r') as file:

reader = csv.reader(file)

for row in reader:

```

        print(row)

# CSV with dictionary
with open('people.csv', 'w', newline='') as file:
    fieldnames = ['Name', 'Age', 'City']
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({'Name': 'Alice', 'Age': 25, 'City': 'New York'})
    writer.writerow({'Name': 'Bob', 'Age': 30, 'City': 'Los Angeles'})

with open('people.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row['Name'], row['Age'])

# JSON Files
import json

# Write JSON
data = {
    'name': 'Alice',
    'age': 25,
    'hobbies': ['reading', 'coding'],
    'address': {
        'city': 'New York',
        'zip': '10001'
    }
}

with open('data.json', 'w') as file:
    json.dump(data, file, indent=2)

# Read JSON
with open('data.json', 'r') as file:
    loaded_data = json.load(file)
    print(loaded_data)

# Pretty print JSON
json_string = json.dumps(data, indent=2, sort_keys=True)
print(json_string)

# Text Files with Encoding
with open('utf8_file.txt', 'w', encoding='utf-8') as file:
    file.write('Hello World')

```

```
with open('utf8_file.txt', 'r', encoding='utf-8') as file:
    content = file.read()
    print(content)
```

Binary Files

```
with open('image.jpg', 'rb') as file:
    binary_data = file.read()
    print(f"Read {len(binary_data)} bytes")
```

```
with open('copy.jpg', 'wb') as file:
    file.write(binary_data)
```

PRACTICAL EXAMPLES

1. File copy utility

```
def copy_file(source, destination):
    """Copy file from source to destination"""
    try:
        with open(source, 'rb') as src:
            with open(destination, 'wb') as dst:
                # Copy in chunks to handle large files
                while True:
                    chunk = src.read(8192) # 8KB chunks
                    if not chunk:
                        break
                    dst.write(chunk)
        print(f"Copied {source} to {destination}")
    except FileNotFoundError:
        print(f"Source file {source} not found")
    except PermissionError:
        print("Permission denied")
```

2. Log file analyzer

```
def analyze_log(log_file):
    """Analyze log file for errors and warnings"""
    error_count = 0
    warning_count = 0

    with open(log_file, 'r') as file:
        for line in file:
            if 'ERROR' in line:
                error_count += 1
            print(f"Error: {line.strip()}")
```

```

        elif 'WARNING' in line:
            warning_count += 1

    print(f"Total errors: {error_count}")
    print(f"Total warnings: {warning_count}")

# 3. Configuration file reader
def read_config(config_file):
    """Read key=value configuration file"""
    config = {}

    with open(config_file, 'r') as file:
        for line in file:
            line = line.strip()
            if line and not line.startswith('#'):
                key, value = line.split('=', 1)
                config[key.strip()] = value.strip()

    return config

# 4. Large file processing
def process_large_file(file_path, chunk_size=1024):
    """Process large file in chunks"""
    with open(file_path, 'r') as file:
        while True:
            chunk = file.read(chunk_size)
            if not chunk:
                break
            # Process chunk
            yield chunk

# 5. File statistics
def file_stats(file_path):
    """Get file statistics"""
    import os
    import time

    stats = os.stat(file_path)
    return {
        'size': stats.st_size,
        'created': time.ctime(stats.st_ctime),
        'modified': time.ctime(stats.st_mtime),
        'is_file': os.path.isfile(file_path),
        'is_dir': os.path.isdir(file_path)
    }

```



```
}
```

6. Directory traversal

```
import os
```

```
def walk_directory(directory, pattern=None):  
    """Walk through directory and yield file paths"""  
    for root, dirs, files in os.walk(directory):  
        for file in files:  
            if pattern is None or pattern in file:  
                yield os.path.join(root, file)
```

7. Safe file writing (atomic)

```
def safe_write(file_path, content):  
    """Write file safely using temporary file"""  
    import tempfile  
    import shutil  
  
    # Create temporary file  
    with tempfile.NamedTemporaryFile(mode='w', delete=False) as tmp:  
        tmp.write(content)  
        tmp_path = tmp.name  
  
    # Move temporary file to destination  
    shutil.move(tmp_path, file_path)
```

8. File locking for concurrent access

```
import fcntl
```

```
def locked_write(file_path, content):  
    """Write file with exclusive lock"""  
    with open(file_path, 'w') as file:  
        fcntl.flock(file, fcntl.LOCK_EX) # Acquire lock  
        try:  
            file.write(content)  
        finally:  
            fcntl.flock(file, fcntl.LOCK_UN) # Release lock
```

ERROR HANDLING

```
try:  
    with open('nonexistent.txt', 'r') as file:  
        content = file.read()  
except FileNotFoundError:  
    print("File not found!")
```

```

except PermissionError:
    print("Permission denied!")
except IOError as e:
    print(f"I/O error: {e}")
finally:
    print("File operation attempted")

```

Class Activity:

- Create a simple note-taking application with file storage
- Build a CSV data analyzer that reads and processes data
- Implement a file organizer that sorts files by type/date

Module 19: Error Handling with Try/Except

WHAT is Error Handling?

Error handling allows programs to gracefully manage and recover from runtime errors without crashing.

WHEN to use error handling?

- User input validation
- File operations
- Network requests
- Database operations
- External API calls

WHERE to use error handling?

- Input validation
- Resource management
- Graceful degradation
- Debugging and logging

HOW to handle errors?

```

# BASIC EXCEPTION HANDLING
# Try-except block
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")

```

```
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

```
# Catching multiple exceptions
```

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
except (ValueError, ZeroDivisionError) as e:
    print(f"Error: {e}")
```

```
# Catching any exception
```

```
try:
    risky_operation()
except Exception as e:
    print(f"An error occurred: {type(e).__name__}: {e}")
```

```
# Try-except-else
```

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
except ValueError:
    print("Invalid input!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    # Executes if no exception occurred
    print(f"Result: {result}")
```

```
# Try-except-else-finally
```

```
try:
    file = open('data.txt', 'r')
    content = file.read()
except FileNotFoundError:
    print("File not found!")
else:
    print(f"Read {len(content)} characters")
finally:
    # Always executes (cleanup code)
    if 'file' in locals():
        file.close()
    print("File closed")
```

```
# CUSTOM EXCEPTIONS
```

```
class ValidationError(Exception):
```

```

    """Custom exception for validation errors"""
    pass

class DatabaseError(Exception):
    """Custom exception for database errors"""
    def __init__(self, message, original_error=None):
        super().__init__(message)
        self.original_error = original_error

# Raising custom exceptions
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    if age > 150:
        raise ValueError("Age cannot exceed 150")
    return True

try:
    validate_age(-5)
except ValueError as e:
    print(f"Validation failed: {e}")

# EXCEPTION CHAINING
try:
    int('not a number')
except ValueError as e:
    raise RuntimeError("Conversion failed") from e

# RERAISING EXCEPTIONS
def process_data(data):
    try:
        # Some processing
        result = int(data)
    except ValueError:
        print("Invalid data, using default")
        result = 0
        raise # Re-raise for higher-level handling
    return result

# CONTEXT MANAGERS (with statement)
# Automatically handles cleanup
class ManagedFile:
    def __init__(self, filename, mode):
        self.filename = filename

```

```

        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        # Return True to suppress exception
        return False

```

Usage

```

with ManagedFile('test.txt', 'w') as f:
    f.write('Hello, World!')

```

Built-in context managers
import contextlib

```

@contextlib.contextmanager
def temporary_file(filename, content):
    """Context manager for temporary file"""
    try:
        with open(filename, 'w') as f:
            f.write(content)
        yield filename
    finally:
        import os
        if os.path.exists(filename):
            os.remove(filename)

```

PRACTICAL EXAMPLES

1. Retry mechanism
import time
import random

```

def retry(max_attempts=3, delay=1):
    """Decorator for retrying failed operations"""
    def decorator(func):
        def wrapper(*args, **kwargs):
            attempts = 0
            while attempts < max_attempts:

```

```

        try:
            return func(*args, **kwargs)
        except Exception as e:
            attempts += 1
            if attempts == max_attempts:
                raise
            print(f"Attempt {attempts} failed: {e}")
            time.sleep(delay * attempts)
    return None
return wrapper
return decorator

```

```

@retry(max_attempts=5, delay=2)
def unstable_network_call():
    """Simulate an unstable network call"""
    if random.random() < 0.7:
        raise ConnectionError("Network error")
    return "Success!"

```

2. Input validation with retry

```

def get_valid_number(prompt, min_val=None, max_val=None):
    """Get a valid number from user"""
    while True:
        try:
            value = float(input(prompt))
            if min_val is not None and value < min_val:
                print(f"Value must be >= {min_val}")
                continue
            if max_val is not None and value > max_val:
                print(f"Value must be <= {max_val}")
                continue
            return value
        except ValueError:
            print("Please enter a valid number")

```

3. Safe dictionary access

```

class SafeDict:
    """Dictionary that handles missing keys gracefully"""
    def __init__(self, data):
        self.data = data

    def get(self, key, default=None):
        try:
            return self.data[key]

```

```
except KeyError:
    return default
```

4. Resource management with contextlib

```
from contextlib import closing
import urllib.request
```

```
with closing(urllib.request.urlopen('http://example.com')) as response:
    for line in response:
        print(line)
```

5. Exception grouping

```
class ErrorHandler:
    """Handle multiple exceptions with custom responses"""
    def __init__(self):
        self.handlers = {}

    def register(self, exception_type, handler):
        self.handlers[exception_type] = handler

    def handle(self, func, *args, **kwargs):
        try:
            return func(*args, **kwargs)
        except Exception as e:
            for exc_type, handler in self.handlers.items():
                if isinstance(e, exc_type):
                    return handler(e)
            raise
```

Usage

```
handler = ErrorHandler()
handler.register(ValueError, lambda e: f"Value error: {e}")
handler.register(ZeroDivisionError, lambda e: "Cannot divide by zero")
```

6. Logging exceptions

```
import logging
```

```
logging.basicConfig(level=logging.ERROR)
```

```
def log_exceptions(func):
    """Decorator to log exceptions"""
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)
```

```

        except Exception as e:
            logging.exception(f"Error in {func.__name__}: {e}")
            raise
    return wrapper

@log_exceptions
def risky_function():
    raise ValueError("Something went wrong")

# 7. Graceful degradation
def fetch_data(api_url):
    """Fetch data with fallback to cached data"""
    try:
        import requests
        response = requests.get(api_url, timeout=5)
        response.raise_for_status()
        return response.json()
    except (requests.RequestException, ConnectionError):
        print("Network error, using cached data")
        try:
            with open('cache.json', 'r') as f:
                import json
                return json.load(f)
        except FileNotFoundError:
            return None

# 8. Cleanup with try-finally
def process_with_cleanup():
    """Ensure cleanup even if errors occur"""
    resource = acquire_resource()
    try:
        # Use resource
        result = process_resource(resource)
        return result
    finally:
        # Always cleanup
        release_resource(resource)

```

Class Activity:

- Create a robust calculator with error handling for all edge cases
- Build a file parser with validation and error recovery
- Implement a retry mechanism for API calls

Module 20: Classes & Objects (OOP Basics)

WHAT are Classes and Objects?

- **Classes:** Blueprints for creating objects
- **Objects:** Instances of classes with their own data and behavior

WHEN to use OOP?

- Modeling real-world entities
- Encapsulating data and behavior
- Creating reusable components
- Building complex systems

WHERE to use OOP?

- GUI applications
- Game development
- Large-scale applications
- Framework development

HOW to create classes and objects?

BASIC CLASS DEFINITION

class Dog:

"""A simple Dog class"""

Class variable (shared by all instances)

species = "Canis familiaris"

Constructor (initializer)

def __init__(self, name, age):

Instance variables

self.name = name

self.age = age

Instance methods

def bark(self):

return f"{self.name} says Woof!"

def get_info(self):

return f"{self.name} is {self.age} years old"

def __str__(self):

return f"Dog(name={self.name}, age={self.age})"

```

def __repr__(self):
    return f"Dog('{self.name}', {self.age})"

# Creating objects
my_dog = Dog("Buddy", 3)
your_dog = Dog("Max", 5)

# Accessing attributes
print(my_dog.name)      # Buddy
print(my_dog.age)       # 3
print(my_dog.species)   # Canis familiaris (class variable)

# Calling methods
print(my_dog.bark())    # Buddy says Woof!
print(my_dog.get_info()) # Buddy is 3 years old

# String representations
print(my_dog)           # Dog(name=Buddy, age=3)
print(repr(my_dog))     # Dog('Buddy', 3)

# CLASS METHODS AND STATIC METHODS
class Calculator:
    """Calculator with class and static methods"""

    # Class variable
    calculation_count = 0

    def __init__(self):
        self.memory = 0

    # Instance method (has access to self)
    def add_to_memory(self, value):
        self.memory += value
        Calculator.calculation_count += 1

    # Class method (receives class, not instance)
    @classmethod
    def get_count(cls):
        return cls.calculation_count

    # Static method (no self or cls)
    @staticmethod
    def add(a, b):
        return a + b

```

```

    @staticmethod
    def multiply(a, b):
        return a * b

# Using class and static methods
print(Calculator.add(5, 3))      # 8
print(Calculator.multiply(5, 3)) # 15
print(Calculator.get_count())  # 0

calc = Calculator()
calc.add_to_memory(10)
calc.add_to_memory(20)
print(Calculator.get_count())  # 2

# PROPERTIES (Getters and Setters)
class Person:
    """Person with property validation"""

    def __init__(self, name, age):
        self._name = name
        self._age = age

    # Property getter
    @property
    def name(self):
        return self._name

    # Property setter
    @name.setter
    def name(self, value):
        if not isinstance(value, str):
            raise TypeError("Name must be string")
        if len(value) < 2:
            raise ValueError("Name too short")
        self._name = value

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if not isinstance(value, (int, float)):

```

```

        raise TypeError("Age must be number")
    if value < 0 or value > 150:
        raise ValueError("Invalid age")
    self._age = value

# Computed property
@property
def is_adult(self):
    return self._age >= 18

person = Person("Alice", 25)
print(person.name)      # Alice
print(person.age)       # 25
print(person.is_adult)  # True

person.name = "Bob"     # Works
# person.age = -5       # Raises ValueError

# PRIVATE AND PROTECTED ATTRIBUTES
class BankAccount:
    """Bank account with encapsulation"""

    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance      # Protected (single underscore)
        self.__account_number = self._generate_account() # Private (double underscore)

    def _generate_account(self):
        """Protected method"""
        import random
        return f"ACC-{random.randint(10000, 99999)}"

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            return True
        return False

    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
            return True
        return False

```

```

def get_balance(self):
    return self._balance

# Property for read-only access
@property
def account_number(self):
    return self.__account_number

# Name mangling for private attributes
account = BankAccount("Alice", 1000)
print(account.owner)          # Alice
print(account.get_balance())  # 1000
# print(account.__account_number) # AttributeError
print(account._BankAccount__account_number) # Works but not recommended

# MAGIC METHODS (Dunder methods)
class Vector:
    """Vector class with magic methods"""

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        """Addition: v1 + v2"""
        if isinstance(other, Vector):
            return Vector(self.x + other.x, self.y + other.y)
        return NotImplemented

    def __sub__(self, other):
        """Subtraction: v1 - v2"""
        if isinstance(other, Vector):
            return Vector(self.x - other.x, self.y - other.y)
        return NotImplemented

    def __mul__(self, scalar):
        """Multiplication: v * scalar"""
        if isinstance(scalar, (int, float)):
            return Vector(self.x * scalar, self.y * scalar)
        return NotImplemented

    def __eq__(self, other):
        """Equality: v1 == v2"""
        if isinstance(other, Vector):

```

```
        return self.x == other.x and self.y == other.y
    return False
```

```
def __len__(self):
    """Length: len(v)"""
    return int((self.x**2 + self.y**2)**0.5)
```

```
def __repr__(self):
    return f"Vector({self.x}, {self.y})"
```

```
v1 = Vector(2, 3)
v2 = Vector(1, 4)
```

```
print(v1 + v2)    # Vector(3, 7)
print(v1 - v2)    # Vector(1, -1)
print(v1 * 3)     # Vector(6, 9)
print(v1 == v2)   # False
print(len(v1))    # 3 (sqrt(13) ≈ 3.6, int = 3)
```

```
# CLASS COMPOSITION
```

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        return "Engine starting..."
```

```
class Wheels:
    def __init__(self, count=4):
        self.count = count

    def rotate(self):
        return f"Wheels rotating ({self.count} wheels)"
```

```
class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model
        self.engine = Engine(200)
        self.wheels = Wheels(4)

    def drive(self):
        return f"{self.make} {self.model}: {self.engine.start()} {self.wheels.rotate()}"
```

```
car = Car("Toyota", "Camry")
print(car.drive())
```

PRACTICAL EXAMPLES

1. Simple To-Do List class

```
class TodoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append({'task': task, 'completed': False})
        return f"Added: {task}"

    def complete_task(self, index):
        if 0 <= index < len(self.tasks):
            self.tasks[index]['completed'] = True
            return f"Completed: {self.tasks[index]['task']}"
        return "Invalid task index"

    def __len__(self):
        return len(self.tasks)

    def __getitem__(self, index):
        return self.tasks[index]

    def __iter__(self):
        return iter(self.tasks)
```

2. Singleton Pattern

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        self.data = []
```

Both variables reference same instance

```
s1 = Singleton()
s2 = Singleton()
```

```
s1.data.append(1)
print(s2.data) # [1]
```

3. Factory Pattern

```
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type, name):
        if animal_type == "dog":
            return Dog(name, 1)
        elif animal_type == "cat":
            return Cat(name, 1)
        else:
            raise ValueError(f"Unknown animal type: {animal_type}")
```

4. Context Manager with class

```
class ManagedResource:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        print(f"Acquiring {self.name}")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Releasing {self.name}")
        return False

with ManagedResource("database") as resource:
    print("Using resource")
```

Class Activity:

- Create a Student class with properties and methods
- Build a BankAccount system with deposits and withdrawals
- Design a simple game using OOP (e.g., RPG character)

Day 21: Inheritance & Polymorphism

WHAT are Inheritance and Polymorphism?

- **Inheritance:** Creating new classes based on existing classes
- **Polymorphism:** Objects of different types responding to the same interface

WHEN to use inheritance?

- Code reuse
- Creating hierarchies
- Implementing interfaces
- Extending functionality

WHERE to use inheritance?

- Framework development
- GUI components
- Game object hierarchies
- Abstract base classes

HOW to use inheritance and polymorphism?

BASIC INHERITANCE

class Animal:

"""Base class"""

def __init__(self, name):

self.name = name

def speak(self):

return f"{self.name} makes a sound"

def move(self):

return f"{self.name} moves"

def __str__(self):

return f"Animal(name={self.name})"

Single inheritance

class Dog(Animal):

"""Dog class inheriting from Animal"""

def __init__(self, name, breed):

Call parent constructor

super().__init__(name)

self.breed = breed

Override method

def speak(self):

return f"{self.name} barks: Woof!"

New method

```

def fetch(self):
    return f"{self.name} fetches the ball"

def __str__(self):
    return f"Dog(name={self.name}, breed={self.breed})"

class Cat(Animal):
    """Cat class inheriting from Animal"""

    def speak(self):
        return f"{self.name} meows: Meow!"

    def scratch(self):
        return f"{self.name} scratches"

# Using inheritance
dog = Dog("Buddy", "Golden Retriever")
cat = Cat("Whiskers")

print(dog.speak())    # Buddy barks: Woof!
print(dog.move())     # Buddy moves
print(dog.fetch())    # Buddy fetches the ball

print(cat.speak())    # Whiskers meows: Meow!
print(cat.move())     # Whiskers moves

# MULTIPLE INHERITANCE
class Flyable:
    """Mixin class"""
    def fly(self):
        return "Flying through the air"

class Swimmable:
    """Mixin class"""
    def swim(self):
        return "Swimming in water"

class Duck(Animal, Flyable, Swimmable):
    """Duck inherits from multiple classes"""

    def speak(self):
        return f"{self.name} quacks: Quack!"

    def move(self):

```

```
# Override and use parent methods
return f"{self.name} can both {self.swim().lower()} and {self.fly().lower()}"
```

```
duck = Duck("Donald")
print(duck.speak())    # Donald quacks: Quack!
print(duck.fly())      # Flying through the air
print(duck.swim())     # Swimming in water
print(duck.move())     # Donald can both swimming in water and flying through the air
```

```
# METHOD RESOLUTION ORDER (MRO)
print(Duck.__mro__)
# (<class '__main__.Duck'>, <class '__main__.Animal'>,
# <class '__main__.Flyable'>, <class '__main__.Swimmable'>,
# <class 'object'>)
```

```
# ABSTRACT BASE CLASSES
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
    """Abstract base class"""

    @abstractmethod
    def area(self):
        """Calculate area"""
        pass

    @abstractmethod
    def perimeter(self):
        """Calculate perimeter"""
        pass

    def describe(self):
        """Concrete method"""
        return f"This is a {self.__class__.__name__}"
```

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14159 * self.radius
```

```

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# Cannot instantiate abstract class
# shape = Shape() # TypeError

circle = Circle(5)
rectangle = Rectangle(4, 6)

print(circle.area())      # 78.53975
print(rectangle.area())   # 24
print(circle.describe())  # This is a Circle

# POLYMORPHISM
def print_shape_info(shape):
    """Polymorphic function works with any Shape"""
    print(f"Area: {shape.area()}")
    print(f"Perimeter: {shape.perimeter()}")
    print(shape.describe())

# Works with both Circle and Rectangle
print_shape_info(circle)
print_shape_info(rectangle)

# SUPER() ADVANCED USAGE
class Logger:
    def __init__(self, level="INFO"):
        self.level = level

    def log(self, message):
        print(f"[{self.level}] {message}")

class Database:
    def __init__(self, connection_string):
        self.connection_string = connection_string

```

```

def connect(self):
    return f"Connected to {self.connection_string}"

class Service(Logger, Database):
    def __init__(self, name, level="INFO", connection_string="localhost"):
        # Call parent constructors
        Logger.__init__(self, level)
        Database.__init__(self, connection_string)
        self.name = name

    def process(self):
        self.log(f"Processing with {self.name}")
        self.connect()
        return "Processing complete"

service = Service("API Service")
print(service.process())

# COMPOSITION OVER INHERITANCE
class Engine:
    def start(self):
        return "Engine started"

class Transmission:
    def shift(self):
        return "Gear shifted"

class Vehicle:
    def __init__(self):
        self.engine = Engine()
        self.transmission = Transmission()

    def start(self):
        return f"{self.engine.start()} - {self.transmission.shift()}"

# Mixins for reusable functionality
class TimestampMixin:
    def get_timestamp(self):
        from datetime import datetime
        return datetime.now().isoformat()

class SerializableMixin:
    def to_dict(self):

```

```
    return {k: v for k, v in self.__dict__.items()
            if not k.startswith('_')}
```

```
class User(TimestampMixin, SerializableMixin):
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.created_at = self.get_timestamp()
```

```
user = User("Alice", "alice@example.com")
print(user.to_dict())
print(user.created_at)
```

PRACTICAL EXAMPLES

1. Plugin System

```
class Plugin(ABC):
    @abstractmethod
    def execute(self, data):
        pass
```

```
class UpperCasePlugin(Plugin):
    def execute(self, data):
        return data.upper()
```

```
class ReversePlugin(Plugin):
    def execute(self, data):
        return data[::-1]
```

```
class PluginManager:
    def __init__(self):
        self.plugins = []

    def register(self, plugin):
        if isinstance(plugin, Plugin):
            self.plugins.append(plugin)

    def process(self, data):
        for plugin in self.plugins:
            data = plugin.execute(data)
        return data
```

```
manager = PluginManager()
manager.register(UpperCasePlugin())
```

```
manager.register(ReversePlugin())
print(manager.process("hello")) # OLLEH
```

2. Template Method Pattern

```
class DataProcessor:
```

```
    def process(self, data):
        """Template method"""
        data = self.validate(data)
        data = self.transform(data)
        data = self.save(data)
        return data
```

```
    def validate(self, data):
        raise NotImplementedError
```

```
    def transform(self, data):
        raise NotImplementedError
```

```
    def save(self, data):
        raise NotImplementedError
```

```
class JSONProcessor(DataProcessor):
```

```
    def validate(self, data):
        import json
        try:
            json.loads(data)
            return data
        except:
            raise ValueError("Invalid JSON")
```

```
    def transform(self, data):
        import json
        parsed = json.loads(data)
        return json.dumps(parsed, indent=2)
```

```
    def save(self, data):
        with open('output.json', 'w') as f:
            f.write(data)
        return "Saved to output.json"
```

3. Multiple Inheritance with Diamond Problem

```
class A:
```

```
    def method(self):
        return "A"
```

```

class B(A):
    def method(self):
        return "B"

class C(A):
    def method(self):
        return "C"

class D(B, C):
    pass

d = D()
print(d.method()) # B (MRO: D -> B -> C -> A)
print(D.__mro__)

```

Class Activity:

- Create a vehicle hierarchy with different types (Car, Motorcycle, Truck)
- Build a plugin system for a text processor
- Implement an abstract base class for a payment system

STAGE 4: ADVANCED CONCEPTS (Modules 22-28)

Module 22: Iterators & Generators Deep Dive

WHAT are Iterators and Generators?

- **Iterators:** Objects that implement **iter()** and **next()** methods
- **Generators:** Functions that use **yield** to produce values lazily

WHEN to use advanced iterators?

- Custom iteration logic
- Lazy evaluation
- Infinite sequences
- Memory-efficient data processing

WHERE to use them?

- Data streaming
- Custom collections

- Lazy loading
- Pipeline processing

HOW to create custom iterators and generators?

CUSTOM ITERATORS

class Countdown:

"""Custom iterator that counts down"""

def __init__(self, start):

self.start = start

def __iter__(self):

return self

def __next__(self):

if self.start < 0:

raise StopIteration

current = self.start

self.start -= 1

return current

Using custom iterator

countdown = Countdown(5)

for num in countdown:

print(num, end=' ') # 5 4 3 2 1 0

Iterator for custom data structure

class FibonacciIterator:

"""Iterator for Fibonacci numbers"""

def __init__(self, limit):

self.limit = limit

self.a, self.b = 0, 1

self.count = 0

def __iter__(self):

return self

def __next__(self):

if self.count >= self.limit:

raise StopIteration

if self.count == 0:

self.count += 1

```

        return self.a

    self.count += 1
    current = self.b
    self.a, self.b = self.b, self.a + self.b
    return current

fib = Fibonaccilterator(10)
print(list(fib)) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# ADVANCED GENERATORS
# Generator with state
def fibonacci_generator():
    """Generator for infinite Fibonacci sequence"""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Using generator
fib = fibonacci_generator()
first_10 = [next(fib) for _ in range(10)]
print(first_10) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# Generator with send()
def accumulator():
    """Generator that receives values and accumulates"""
    total = 0
    while True:
        value = yield total
        if value is not None:
            total += value

acc = accumulator()
next(acc) # Initialize
print(acc.send(10)) # 10
print(acc.send(20)) # 30
print(acc.send(5)) # 35

# Generator with throw()
def generator_with_exception():
    try:
        yield 1
        yield 2

```

```

        yield 3
    except ValueError as e:
        yield f"Caught: {e}"
    finally:
        yield "Done"

gen = generator_with_exception()
print(next(gen)) # 1
print(next(gen)) # 2
print(gen.throw(ValueError("Oops"))) # "Caught: Oops"
print(next(gen)) # "Done"

# Generator with close()
def infinite_generator():
    count = 0
    try:
        while True:
            yield count
            count += 1
    except GeneratorExit:
        print("Generator closed")

gen = infinite_generator()
print(next(gen)) # 0
print(next(gen)) # 1
gen.close() # "Generator closed"

# GENERATOR EXPRESSIONS
# Memory efficient
large_range = (x**2 for x in range(1000000))
print(f"Generator size: {large_range.__sizeof__()} bytes")

# Compare with list
large_list = [x**2 for x in range(1000000)]
print(f"List size: {large_list.__sizeof__()} bytes")

# Chaining generators
def read_lines(filename):
    with open(filename, 'r') as f:
        for line in f:
            yield line.strip()

def filter_comments(lines):
    for line in lines:

```

```

        if not line.startswith('#'):
            yield line

def split_fields(lines):
    for line in lines:
        yield line.split(',')

# Pipeline
lines = read_lines('data.txt')
clean_lines = filter_comments(lines)
fields = split_fields(clean_lines)

# ITERATOR TOOLS (itertools)
import itertools

# Infinite iterators
counter = itertools.count(start=10, step=2)
first_5 = [next(counter) for _ in range(5)]
print(first_5) # [10, 12, 14, 16, 18]

# Cycle through iterable
colors = itertools.cycle(['red', 'green', 'blue'])
first_7 = [next(colors) for _ in range(7)]
print(first_7) # ['red', 'green', 'blue', 'red', 'green', 'blue', 'red']

# Repeat value
repeater = itertools.repeat('Hello', 3)
print(list(repeater)) # ['Hello', 'Hello', 'Hello']

# Combinatoric iterators
# Permutations
items = ['A', 'B', 'C']
perms = list(itertools.permutations(items, 2))
print(perms) # [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]

# Combinations
combs = list(itertools.combinations(items, 2))
print(combs) # [('A', 'B'), ('A', 'C'), ('B', 'C')]

# Combinations with replacement
combs_wr = list(itertools.combinations_with_replacement(items, 2))
print(combs_wr) # [('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'B'), ('B', 'C'), ('C', 'C')]

# Product (Cartesian product)

```

```
colors = ['red', 'blue']
sizes = ['S', 'M', 'L']
products = list(itertools.product(colors, sizes))
print(products) # [('red','S'), ('red','M'), ...]
```

```
# Advanced itertools functions
# Chain multiple iterables
list1 = [1, 2, 3]
list2 = [4, 5, 6]
chained = list(itertools.chain(list1, list2))
print(chained) # [1, 2, 3, 4, 5, 6]
```

```
# Group data
data = [('A', 1), ('A', 2), ('B', 3), ('B', 4)]
grouped = itertools.groupby(data, key=lambda x: x[0])
for key, group in grouped:
    print(key, list(group))
```

```
# Zip longest
names = ['Alice', 'Bob']
ages = [25, 30, 35]
zipped = list(itertools.zip_longest(names, ages, fillvalue='Unknown'))
print(zipped) # [('Alice',25), ('Bob',30), ('Unknown',35)]
```

```
# Slice iterators
squares = (x**2 for x in range(100))
first_10 = list(itertools.islice(squares, 10))
print(first_10) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

PRACTICAL EXAMPLES

1. Custom range with step

```
class StepRange:
    """Iterator that behaves like range but with floating steps"""

    def __init__(self, start, stop, step=1):
        self.current = start
        self.stop = stop
        self.step = step

    def __iter__(self):
        return self

    def __next__(self):
```

```

        if self.step > 0 and self.current >= self.stop:
            raise StopIteration
        if self.step < 0 and self.current <= self.stop:
            raise StopIteration

        current = self.current
        self.current += self.step
        return current

for num in StepRange(0, 1, 0.1):
    print(f"{num:.1f}", end=' ') # 0.0 0.1 0.2 ... 0.9

# 2. Generator for paginated API calls
def paginated_api_call(url, page_size=100):
    """Generator that yields pages from an API"""
    import requests

    page = 1
    while True:
        response = requests.get(url, params={'page': page, 'size': page_size})
        if response.status_code != 200:
            break

        data = response.json()
        if not data:
            break

        yield data
        page += 1

# 3. Generator for reading large files
def read_large_file(file_path, chunk_size=1024):
    """Read large file in chunks"""
    with open(file_path, 'r') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk

# 4. Iterator for tree traversal
class TreeNode:
    def __init__(self, value, children=None):
        self.value = value

```

```

        self.children = children or []

    def __iter__(self):
        """DFS iterator"""
        yield self.value
        for child in self.children:
            yield from child

# 5. Generator with context
import contextlib

@contextlib.contextmanager
def generator_context():
    """Context manager for generators"""
    try:
        yield
    finally:
        print("Cleaning up")

# 6. Parallel iteration with zip and generators
def parallel_process(generators):
    """Process multiple generators in parallel"""
    for values in zip(*generators):
        yield values

# 7. Generator for infinite data stream
def data_stream():
    """Simulate continuous data stream"""
    import random
    import time

    while True:
        yield {
            'timestamp': time.time(),
            'value': random.random(),
            'status': random.choice(['OK', 'WARNING', 'ERROR'])
        }
        time.sleep(0.1)

# 8. Lazy evaluation with generators
def lazy_map(func, iterable):
    """Lazy version of map"""
    for item in iterable:
        yield func(item)

```

```
def lazy_filter(predicate, iterable):
    """Lazy version of filter"""
    for item in iterable:
        if predicate(item):
            yield item

# Usage
numbers = range(1000000)
evens = lazy_filter(lambda x: x % 2 == 0, numbers)
squares = lazy_map(lambda x: x**2, evens)
first_10 = list(itertools.islice(squares, 10))
print(first_10) # [0, 4, 16, 36, 64, 100, 144, 196, 256, 324]
```

Class Activity:

- Create a custom iterator for a binary tree
- Build a generator for paginated data from an API
- Implement a lazy evaluation pipeline for data processing

Module 23: Decorators & Closures

WHAT are Decorators and Closures?

- **Closures:** Functions that capture variables from their enclosing scope
- **Decorators:** Functions that modify the behavior of other functions

WHEN to use decorators?

- Adding logging
- Measuring performance
- Access control
- Caching/memoization
- Validation

WHERE to use decorators?

- Web frameworks (Flask, Django)
- Authentication systems
- Caching layers
- Monitoring tools

HOW to create decorators and closures?

CLOSURES


```

def outer_function(x):
    """Closure example"""
    def inner_function(y):
        return x + y
    return inner_function

# Create closures
add_5 = outer_function(5)
add_10 = outer_function(10)

print(add_5(3)) # 8
print(add_10(3)) # 13

# Closure with state
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter1 = counter()
counter2 = counter()
print(counter1()) # 1
print(counter1()) # 2
print(counter2()) # 1

# BASIC DECORATORS
def my_decorator(func):
    """Simple decorator"""
    def wrapper():
        print("Something before function")
        func()
        print("Something after function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Something before function
# Hello!

```

```
# Something after function
```

```
# DECORATORS WITH ARGUMENTS
```

```
def repeat(n):  
    """Decorator that repeats function n times"""  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            results = []  
            for _ in range(n):  
                result = func(*args, **kwargs)  
                results.append(result)  
            return results  
        return wrapper  
    return decorator
```

```
@repeat(3)  
def greet(name):  
    return f"Hello, {name}!"
```

```
print(greet("Alice")) # ['Hello, Alice!', 'Hello, Alice!', 'Hello, Alice!']
```

```
# PRESERVING FUNCTION METADATA
```

```
import functools
```

```
def timer(func):  
    """Measure execution time"""  
    @functools.wraps(func) # Preserve function metadata  
    def wrapper(*args, **kwargs):  
        import time  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"{func.__name__} took {end - start:.4f}s")  
        return result  
    return wrapper
```

```
@timer  
def slow_function(duration):  
    import time  
    time.sleep(duration)  
    return "Done"
```

```
print(slow_function.__name__) # slow_function (preserved)
```

PRACTICAL DECORATORS

1. Logging decorator

```
def log(func):
    """Log function calls"""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper
```

```
@log
def add(a, b):
    return a + b
```

```
add(5, 3)
```

2. Authentication decorator

```
def require_auth(func):
    """Check authentication before executing"""
    @functools.wraps(func)
    def wrapper(user, *args, **kwargs):
        if not user.get('authenticated', False):
            raise PermissionError("User not authenticated")
        return func(user, *args, **kwargs)
    return wrapper
```

```
@require_auth
def view_profile(user):
    return f"Profile of {user['name']}"
```

3. Cache decorator (memoization)

```
def cache(func):
    """Cache function results"""
    memo = {}

    @functools.wraps(func)
    def wrapper(*args):
        if args in memo:
            print(f"Returning cached result for {args}")
            return memo[args]
        result = func(*args)
```

```

        memo[args] = result
    return result
return wrapper

@cache
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(30)) # Fast due to caching

# 4. Rate limiter decorator
import time

def rate_limit(calls_per_second):
    """Limit function calls per second"""
    def decorator(func):
        last_called = [0.0]

        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            if elapsed < 1.0 / calls_per_second:
                sleep_time = (1.0 / calls_per_second) - elapsed
                time.sleep(sleep_time)
            result = func(*args, **kwargs)
            last_called[0] = time.time()
            return result
        return wrapper
    return decorator

@rate_limit(2) # Max 2 calls per second
def api_call(url):
    return f"Calling {url}"

# 5. Retry decorator
def retry(max_attempts=3, delay=1):
    """Retry function on failure"""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            attempts = 0
            while attempts < max_attempts:

```

```

    try:
        return func(*args, **kwargs)
    except Exception as e:
        attempts += 1
        if attempts == max_attempts:
            raise
        print(f"Attempt {attempts} failed: {e}, retrying...")
        time.sleep(delay * attempts)
    return None
return wrapper
return decorator

```

```

@retry(max_attempts=5, delay=2)
def unstable_network_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("Network error")
    return "Success"

```

6. Validation decorator

```

def validate_types(**types):
    """Validate function argument types"""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Check positional args
            for (arg_name, arg_value), expected_type in zip(types.items(), args):
                if not isinstance(arg_value, expected_type):
                    raise TypeError(f"{arg_name} must be {expected_type.__name__}")

            # Check keyword args
            for arg_name, expected_type in types.items():
                if arg_name in kwargs:
                    if not isinstance(kwargs[arg_name], expected_type):
                        raise TypeError(f"{arg_name} must be {expected_type.__name__}")

            return func(*args, **kwargs)
        return wrapper
    return decorator

```

```

@validate_types(a=int, b=int)
def add_numbers(a, b):
    return a + b

```

7. Class decorators

```
def add_repr(cls):  
    """Add __repr__ method to class"""  
    def __repr__(self):  
        return f"{cls.__name__}({self.__dict__})"  
    cls.__repr__ = __repr__  
    return cls
```

```
@add_repr  
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
person = Person("Alice", 25)  
print(person) # Person({'name': 'Alice', 'age': 25})
```

8. Decorator with arguments

```
def prefix(prefix_str):  
    """Add prefix to function return"""  
    def decorator(func):  
        @functools.wraps(func)  
        def wrapper(*args, **kwargs):  
            return f"{prefix_str}{func(*args, **kwargs)}"  
        return wrapper  
    return decorator
```

```
@prefix("Result: ")  
def calculate(x, y):  
    return f"{x + y}"
```

```
print(calculate(5, 3)) # Result: 8
```

9. Stacking decorators

```
@log  
@timer  
@retry(max_attempts=3)  
def complex_operation():  
    import time  
    time.sleep(0.1)  
    return "Completed"
```

10. Property decorators

```
class Temperature:
```

```

def __init__(self, celsius):
    self._celsius = celsius

@property
def celsius(self):
    return self._celsius

@celsius.setter
def celsius(self, value):
    if value < -273.15:
        raise ValueError("Temperature below absolute zero")
    self._celsius = value

@property
def fahrenheit(self):
    return (self._celsius * 9/5) + 32

@fahrenheit.setter
def fahrenheit(self, value):
    self.celsius = (value - 32) * 5/9

temp = Temperature(25)
print(temp.celsius)    # 25
print(temp.fahrenheit) # 77.0
temp.fahrenheit = 86
print(temp.celsius)    # 30.0

```

11. Static and class methods as decorators

```

class MyClass:
    @staticmethod
    def static_method():
        return "Static method"

    @classmethod
    def class_method(cls):
        return f"Class method from {cls.__name__}"

    @property
    def property_method(self):
        return "Property method"

```

ADVANCED DECORATOR PATTERNS

1. Decorator with state

```

class CountCalls:
    """Class-based decorator with state"""
    def __init__(self, func):
        self.func = func
        self.count = 0
        functools.update_wrapper(self, func)

    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"Call {self.count} of {self.func.__name__}")
        return self.func(*args, **kwargs)

```

```

@CountCalls
def say_hello():
    print("Hello!")

```

```

say_hello()
say_hello()
print(f"Total calls: {say_hello.count}")

```

2. Singleton decorator

```

def singleton(cls):
    """Make class singleton"""
    instances = {}

    @functools.wraps(cls)
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

```

```

@singleton
class Database:
    def __init__(self):
        self.connection = "Connected"

```

```

db1 = Database()
db2 = Database()
print(db1 is db2) # True

```

3. Lazy property decorator

```

def lazy_property(func):
    """Property that caches result after first access"""

```



```

attr_name = "_lazy_" + func.__name__

@property
@functools.wraps(func)
def _lazy_property(self):
    if not hasattr(self, attr_name):
        setattr(self, attr_name, func(self))
    return getattr(self, attr_name)
return _lazy_property

class DataLoader:
    @lazy_property
    def data(self):
        print("Loading data...")
        return [i for i in range(1000000)]

loader = DataLoader()
print(loader.data) # Loads data
print(loader.data) # Uses cached data

```

Class Activity:

- Create a logging decorator with different log levels
- Build a caching decorator with expiration
- Implement an access control decorator for functions

Module 24: Context Managers & With Statement

WHAT are Context Managers?

Context managers handle resource setup and teardown automatically using the `with` statement.

WHEN to use context managers?

- File operations
- Database connections
- Network connections
- Lock management
- Temporary changes

WHERE to use them?

- Resource management
- Transaction handling
- Performance measurement
- Exception handling

HOW to create context managers?

USING CONTEXT MANAGERS

File handling

with open('file.txt', 'r') as file:

content = file.read()

File automatically closed

Multiple context managers

with open('input.txt', 'r') as input_file, open('output.txt', 'w') as output_file:

output_file.write(input_file.read())

Database connection (example)

import sqlite3

with sqlite3.connect('database.db') as conn:

cursor = conn.cursor()

cursor.execute('SELECT * FROM users')

Connection automatically commits/rolls back and closes

Lock management

import threading

lock = threading.Lock()

with lock:

Critical section

pass

CLASS-BASED CONTEXT MANAGERS

class FileManager:

"""Custom file context manager"""

def __init__(self, filename, mode):

self.filename = filename

self.mode = mode

self.file = None

def __enter__(self):

"""Setup resource"""

self.file = open(self.filename, self.mode)

return self.file

```

def __exit__(self, exc_type, exc_val, exc_tb):
    """Cleanup resource"""
    if self.file:
        self.file.close()

    # Return True to suppress exception
    # Return False to propagate exception
    if exc_type is not None:
        print(f"An error occurred: {exc_type.__name__}: {exc_val}")
        # return True # Would suppress exception
    return False # Propagate exception

# Usage
with FileManager('test.txt', 'w') as f:
    f.write('Hello, World!')

# With exception handling
try:
    with FileManager('test.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("File not found")

# CONTEXT MANAGER WITH TIMING
class Timer:
    """Context manager for measuring execution time"""

    def __init__(self, name="Operation"):
        self.name = name

    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        elapsed = time.time() - self.start
        print(f"{self.name} took {elapsed:.4f} seconds")
        return False

# Usage
with Timer("Processing") as timer:

```

```

# Do some work
import time
time.sleep(1)

# Nested context managers
with Timer("Outer"):
    with Timer("Inner"):
        import time
        time.sleep(0.5)

# CONTEXT MANAGER WITH STATE
class TemporaryDirectory:
    """Create temporary directory and clean up"""

    def __init__(self):
        import tempfile
        import shutil
        self.temp_dir = tempfile.mkdtemp()

    def __enter__(self):
        return self.temp_dir

    def __exit__(self, exc_type, exc_val, exc_tb):
        import shutil
        shutil.rmtree(self.temp_dir)
        return False

with TemporaryDirectory() as temp_dir:
    print(f"Working in {temp_dir}")
    # Create files in temp directory
    # Directory automatically deleted

# CONTEXT MANAGER WITH YIELD (contextlib)
import contextlib

@contextlib.contextmanager
def managed_file(filename, mode):
    """Context manager using generator"""
    file = None
    try:
        file = open(filename, mode)
        yield file
    finally:
        if file:

```

```

        file.close()

# Usage
with managed_file('test.txt', 'w') as f:
    f.write('Hello')

# @contextlib.contextmanager for database connection
@contextlib.contextmanager
def database_connection(db_path):
    """Context manager for database connections"""
    import sqlite3
    conn = sqlite3.connect(db_path)
    try:
        yield conn
        conn.commit() # Commit if no error
    except Exception:
        conn.rollback() # Rollback on error
        raise
    finally:
        conn.close()

with database_connection('test.db') as conn:
    cursor = conn.cursor()
    cursor.execute('CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)')

# CONTEXT MANAGER WITH EXCEPTION HANDLING
class ErrorHandler:
    """Context manager for handling specific exceptions"""

    def __init__(self, handle_exceptions=None):
        self.handle_exceptions = handle_exceptions or []

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None and exc_type in self.handle_exceptions:
            print(f"Handled {exc_type.__name__}: {exc_val}")
            return True # Suppress exception
        return False

# Usage
with ErrorHandler([ValueError, ZeroDivisionError]):
    x = int('not a number') # ValueError handled

```

PRACTICAL CONTEXT MANAGERS

1. Redirect stdout/stderr

```
import sys
from io import StringIO

@contextlib.contextmanager
def capture_output():
    """Capture stdout and stderr"""
    old_stdout = sys.stdout
    old_stderr = sys.stderr
    stdout_capture = StringIO()
    stderr_capture = StringIO()
    sys.stdout = stdout_capture
    sys.stderr = stderr_capture
    try:
        yield stdout_capture, stderr_capture
    finally:
        sys.stdout = old_stdout
        sys.stderr = old_stderr

with capture_output() as (out, err):
    print("Hello")
    print("Error", file=sys.stderr)

print(f"Captured stdout: {out.getvalue()}")
print(f"Captured stderr: {err.getvalue()}")
```

2. Change directory temporarily

```
@contextlib.contextmanager
def change_directory(path):
    """Temporarily change working directory"""
    import os
    old_cwd = os.getcwd()
    try:
        os.chdir(path)
        yield
    finally:
        os.chdir(old_cwd)

with change_directory('/tmp'):
    print(f"Current dir: {os.getcwd()}")
```

```
# 3. Suppress exceptions
@contextlib.contextmanager
def suppress(*exceptions):
    """Suppress specified exceptions"""
    try:
        yield
    except exceptions:
        pass

with suppress(FileNotFoundError, ValueError):
    open('nonexistent.txt').read() # No error
```

```
# 4. Context manager for timing with results
class Timed:
    def __init__(self, name=""):
        self.name = name

    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        self.duration = time.time() - self.start
        if self.name:
            print(f'{self.name}: {self.duration:.4f}s')
        return False

    @property
    def elapsed(self):
        return self.duration
```

```
with Timed("Sorting") as t:
    numbers = list(range(1000000))
    numbers.sort()
print(f"Took {t.elapsed:.4f} seconds")
```

```
# 5. Progress indicator context
class Progress:
    """Show progress during long operation"""

    def __init__(self, message="Processing"):
        self.message = message
```

```

def __enter__(self):
    import sys
    sys.stdout.write(f"{self.message}... ")
    sys.stdout.flush()
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    import sys
    if exc_type is None:
        sys.stdout.write("Done!\n")
    else:
        sys.stdout.write("Failed!\n")
    sys.stdout.flush()
    return False

with Progress("Loading data"):
    import time
    time.sleep(2)
    # Simulate work

# 6. Context manager for temporary file
@contextlib.contextmanager
def temporary_file(content=None, suffix='.txt'):
    """Create temporary file with optional content"""
    import tempfile
    import os

    fd, path = tempfile.mkstemp(suffix=suffix)
    os.close(fd)

    if content:
        with open(path, 'w') as f:
            f.write(content)

    try:
        yield path
    finally:
        os.unlink(path)

with temporary_file("Hello, World!") as filename:
    with open(filename, 'r') as f:
        print(f.read())

```


7. Database transaction context

class Transaction:

```
    """Database transaction context manager"""
```

```
    def __init__(self, connection):
```

```
        self.conn = connection
```

```
    def __enter__(self):
```

```
        self.conn.execute('BEGIN TRANSACTION')
```

```
        return self
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
```

```
        if exc_type is None:
```

```
            self.conn.commit()
```

```
        else:
```

```
            self.conn.rollback()
```

```
        return False
```

8. Chaining context managers

@contextlib.contextmanager

```
def chain_contexts(*managers):
```

```
    """Chain multiple context managers"""
```

```
    with contextlib.ExitStack() as stack:
```

```
        for manager in managers:
```

```
            stack.enter_context(manager)
```

```
    yield
```

9. Context manager for resource pooling

class ResourcePool:

```
    """Manage pool of resources"""
```

```
    def __init__(self, create_resource, max_size=10):
```

```
        self.create_resource = create_resource
```

```
        self.max_size = max_size
```

```
        self.pool = []
```

```
        self.active = 0
```

```
    def __enter__(self):
```

```
        if self.pool:
```

```
            resource = self.pool.pop()
```

```
        elif self.active < self.max_size:
```

```
            resource = self.create_resource()
```

```
            self.active += 1
```

```
        else:
```

```

        raise Exception("No resources available")
    return resource

def __exit__(self, exc_type, exc_val, exc_tb):
    self.pool.append(resource)
    return False

# 10. Context manager for profiling
class Profiler:
    """Profile code execution"""

    def __enter__(self):
        import cProfile
        self.pr = cProfile.Profile()
        self.pr.enable()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.pr.disable()
        self.pr.print_stats(sort='cumulative')
        return False

with Profiler():
    # Code to profile
    sum(range(1000000))

```

Class Activity:

- Create a context manager for database transactions
- Build a timer context manager with detailed metrics
- Implement a context manager for temporary file operations

Module 25: Virtual Environments & pip

WHAT are Virtual Environments and pip?

- **Virtual Environments:** Isolated Python environments for different projects
- **pip:** Python package installer

WHEN to use virtual environments?

- Every Python project
- When managing dependencies

- When working on multiple projects
- For reproducible environments

WHERE to use them?

- Development environments
- Deployment
- CI/CD pipelines
- Testing

HOW to manage virtual environments and packages?

VIRTUAL ENVIRONMENTS

Create virtual environment

python -m venv myenv

On Windows: python -m venv myenv

On Mac/Linux: python3 -m venv myenv

Activate virtual environment

Windows (Command Prompt)

myenv\Scripts\activate.bat

Windows (PowerShell)

myenv\Scripts\Activate.ps1

Mac/Linux

source myenv/bin/activate

Deactivate

deactivate

Check Python and pip location

where python # Windows

which python # Mac/Linux

pip --version

PACKAGE MANAGEMENT WITH PIP

Install package

pip install package_name

pip install requests==2.28.1 # Specific version

pip install "requests>=2.0,<3.0" # Version range

Install multiple packages

pip install requests numpy pandas matplotlib

Install from requirements file

pip install -r requirements.txt

Uninstall package

pip uninstall package_name

List installed packages

pip list

pip freeze # More detailed format

Show package information

pip show package_name

Search for packages

pip search search_term

Upgrade pip

pip install --upgrade pip

Install package in development mode

pip install -e .

Download packages without installing

pip download -d ./packages package_name

REQUIREMENTS FILE

requirements.txt format:

requests==2.28.1

numpy>=1.21.0

pandas<2.0.0

-r subrequirements.txt # Include another file

Create requirements file

pip freeze > requirements.txt

INSTALLING FROM DIFFERENT SOURCES

From GitHub

pip install git+https://github.com/user/repo.git

pip install git+https://github.com/user/repo.git@branch_name

From local directory

pip install /path/to/package

pip install ./package.whl # Wheel file

From URL

pip install https://example.com/package.whl

ADVANCED VENV TOOLS

virtualenv (more features)

pip install virtualenv

virtualenv myenv

virtualenv --python=python3.9 myenv # Specify Python version

virtualenvwrapper (management)

Install

pip install virtualenvwrapper

Setup in .bashrc/.zshrc

export WORKON_HOME=\$HOME/.virtualenvs

source /usr/local/bin/virtualenvwrapper.sh

Commands

mkvirtualenv projectname # Create

workon projectname # Activate

deactivate # Deactivate

rmvirtualenv projectname # Delete

lsvirtualenv # List

conda environments (if using Anaconda)

conda create -n myenv python=3.9

conda activate myenv

conda deactivate

conda env list

conda env export > environment.yml

conda env create -f environment.yml

PIPENV (Combines pip and virtualenv)

pip install pipenv

Create environment with Pipfile

pipenv install

pipenv install requests # Install package

pipenv shell # Activate

pipenv run python script.py # Run script in environment

pipenv lock # Create Pipfile.lock

pipenv install --dev # Install dev dependencies

POETRY (Modern packaging)

Install poetry

curl -sSL <https://install.python-poetry.org> | python3 -

```
# Create new project
poetry new myproject
cd myproject
```

```
# Add dependencies
poetry add requests numpy pandas
poetry add --dev pytest # Dev dependency
```

```
# Activate environment
poetry shell
```

```
# Install dependencies
poetry install
```

```
# Build package
poetry build
```

PRACTICAL EXAMPLES

1. Project structure with venv

```
"""
myproject/
├── myenv/          # Virtual environment (gitignored)
├── src/
│   ├── mypackage/
│   │   ├── __init__.py
│   │   ├── core.py
│   │   └── utils.py
│   ├── tests/
│   │   └── test_core.py
│   ├── requirements.txt # Production dependencies
│   ├── requirements-dev.txt # Development dependencies
│   ├── setup.py        # Package configuration
│   └── README.md
"""
```

2. requirements.txt with extras

```
"""
# requirements.txt
requests>=2.28,<3.0
numpy==1.24.3
pandas==2.0.0
matplotlib==3.7.1
scikit-learn==1.2.2
```

```
# Development dependencies
```

```
pytest==7.3.1
```

```
black==23.3.0
```

```
flake8==6.0.0
```

```
"""
```

```
# 3. requirements-dev.txt
```

```
"""
```

```
-r requirements.txt # Include production requirements
```

```
pytest
```

```
black
```

```
flake8
```

```
mypy
```

```
pre-commit
```

```
"""
```

```
# 4. setup.py for package distribution
```

```
"""
```

```
from setuptools import setup, find_packages
```

```
setup(
```

```
    name="mypackage",
```

```
    version="1.0.0",
```

```
    author="Your Name",
```

```
    description="Package description",
```

```
    packages=find_packages(),
```

```
    install_requires=[
```

```
        "requests>=2.28",
```

```
        "numpy>=1.21",
```

```
    ],
```

```
    extras_require={
```

```
        "dev": ["pytest", "black"],
```

```
        "docs": ["sphinx"],
```

```
    },
```

```
    classifiers=[
```

```
        "Programming Language :: Python :: 3",
```

```
        "License :: OSI Approved :: MIT License",
```

```
    ],
```

```
    python_requires=">=3.8",
```

```
)
```

```
"""
```

```
# 5. Docker environment
```

```
"""
# Dockerfile
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
"""
```

6. CI/CD with GitHub Actions

```
"""
# .github/workflows/python-app.yml
name: Python CI

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.9'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest
      - name: Test with pytest
        run: |
          pytest
"""
```

7. Environment variables in .env

```
"""
# .env file
DATABASE_URL=postgresql://localhost/mydb
"""
```



```
SECRET_KEY=your-secret-key
DEBUG=True
"""
```

```
# Python code to load .env
from dotenv import load_dotenv
import os
```

```
load_dotenv()
database_url = os.getenv('DATABASE_URL')
debug = os.getenv('DEBUG', 'False') == 'True'
```

```
# 8. Package versioning with pip-tools
# Install pip-tools
pip install pip-tools
```

```
# Create requirements.in
# requests>=2.28
# numpy>=1.21
```

```
# Compile requirements.txt
pip-compile requirements.in
```

```
# Sync environment with requirements.txt
pip-sync requirements.txt
```

```
# 9. Checking for security vulnerabilities
pip install safety
safety check # Check installed packages
safety check -r requirements.txt # Check requirements file
```

```
# 10. Freezing environment with all dependencies
pip freeze > requirements.txt
pip install -r requirements.txt --no-deps # Install without checking dependencies
```

BEST PRACTICES

- # 1. Always use virtual environments
- # 2. Pin exact versions in requirements.txt for production
- # 3. Separate production and development dependencies
- # 4. Use .gitignore to exclude venv, __pycache__, .env
- # 5. Document Python version and dependencies in README
- # 6. Use lock files (Pipfile.lock, poetry.lock) for reproducibility
- # 7. Regularly update dependencies

8. Test with minimum and maximum dependency versions

.gitignore example

```
"""  
  
# Virtual Environment  
venv/  
env/  
myenv/  
.env/  
.python-version
```

Python

```
__pycache__/  
*.py[cod]  
*.so  
.Python
```

Distribution

```
dist/  
build/  
*.egg-info/
```

IDE

```
.vscode/  
.idea/
```

Environment variables

```
.env  
.env.local  
"""
```

PYTHON PACKAGE MANAGERS COMPARISON

pip - Basic package manager

- # - Simple and ubiquitous*
- # - Requires separate virtualenv*

pipenv - Combines pip and virtualenv

- # - Uses Pipfile and Pipfile.lock*
- # - Automatic environment management*

poetry - Modern all-in-one tool

- # - Dependency management*
- # - Packaging and publishing*

- Build system

conda - For scientific Python

- Cross-platform

- Handles non-Python dependencies

- Large package ecosystem

Class Activity:

- Set up a project with proper virtual environment and dependencies
- Create a requirements.txt file for a small project
- Build a simple package and install it in development mode

Module 26: Working with Popular Libraries (NumPy, Pandas)

WHAT are NumPy and Pandas?

- **NumPy:** Numerical computing library with multi-dimensional arrays
- **Pandas:** Data manipulation and analysis library

WHEN to use these libraries?

- Data analysis
- Scientific computing
- Machine learning
- Financial modeling
- Statistical analysis

WHERE to use them?

- Data science projects
- Research
- Business intelligence
- Data pipelines

HOW to use NumPy and Pandas?

NUMPY BASICS

import numpy as np

Creating arrays

From lists

arr1 = np.array([1, 2, 3, 4, 5])

arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr1) # [1 2 3 4 5]

```
print(arr2) # [[1 2 3]
            # [4 5 6]]
```

Special arrays

```
zeros = np.zeros((3, 4))    # 3x4 array of zeros
ones = np.ones((2, 3))     # 2x3 array of ones
empty = np.empty((2, 2))   # Uninitialized
identity = np.eye(3)       # 3x3 identity matrix
range_arr = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
linspace = np.linspace(0, 1, 5) # [0, 0.25, 0.5, 0.75, 1]
```

Random arrays

```
random_arr = np.random.rand(3, 3)    # Uniform [0, 1)
random_int = np.random.randint(0, 10, (2, 3)) # Random integers
normal_dist = np.random.randn(100)    # Normal distribution
```

Array attributes

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape) # (2, 3) - dimensions
print(arr.size)  # 6 - total elements
print(arr.ndim)  # 2 - number of dimensions
print(arr.dtype) # int64 - data type
```

Array operations

Element-wise operations

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # [5 7 9]
print(a * b) # [4 10 18]
print(a ** 2) # [1 4 9]
print(np.sqrt(a)) # [1. 1.41 1.73]
```

Broadcasting

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr + 10) # Add scalar to all elements
print(arr * 2)  # Multiply all elements
```

Matrix operations

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B) # Matrix multiplication
print(np.dot(A, B)) # Same
print(A.T) # Transpose
```

```

# Indexing and slicing
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr[0, 1])    # 2
print(arr[:, 1])    # [2, 5, 8] (second column)
print(arr[1:, :2])  # [[4, 5], [7, 8]]
print(arr[arr > 5]) # [6, 7, 8, 9] (boolean indexing)

```

```

# Reshaping and stacking
arr = np.arange(12)
reshaped = arr.reshape(3, 4) # 3x4 array
print(reshaped)

```

```

# Stack arrays
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.vstack((a, b))) # Vertical stack
print(np.hstack((a, b))) # Horizontal stack

```

```

# Universal functions (ufuncs)
arr = np.array([1, 2, 3, 4, 5])
print(np.sum(arr))    # 15
print(np.mean(arr))   # 3.0
print(np.std(arr))     # 1.41
print(np.min(arr))     # 1
print(np.max(arr))     # 5
print(np.argmax(arr))  # 4 (index)

```

```

# PANDAS BASICS
import pandas as pd

```

```

# Series (1D labeled array)
s = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
print(s)
print(s['a'])    # 1
print(s[['a', 'c']]) # a:1, c:3
print(s[s > 3])  # d:4, e:5

```

```

# DataFrame (2D labeled structure)
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'Paris', 'London', 'Tokyo'],
    'Salary': [50000, 60000, 70000, 55000]
}

```

```

df = pd.DataFrame(data)
print(df)

# DataFrame properties
print(df.shape)      # (4, 4)
print(df.columns)     # Index(['Name', 'Age', 'City', 'Salary'])
print(df.index)       # RangeIndex(start=0, stop=4)
print(df.dtypes)      # Data types
print(df.info())      # Summary info

# Accessing data
# Column selection
print(df['Name'])      # Single column (Series)
print(df[['Name', 'Age']]) # Multiple columns

# Row selection
print(df.loc[0])       # By label
print(df.iloc[0])      # By integer position
print(df.loc[1:3])     # Slicing

# Conditional selection
young_people = df[df['Age'] < 30]
print(young_people)

# Multiple conditions
filtered = df[(df['Age'] > 25) & (df['Salary'] > 55000)]
print(filtered)

# Adding columns
df['Bonus'] = df['Salary'] * 0.1
df['Total'] = df['Salary'] + df['Bonus']

# Removing columns
df.drop('Bonus', axis=1, inplace=True)

# Basic operations
print(df.describe())   # Statistical summary
print(df['Age'].mean()) # Mean age
print(df['Salary'].max()) # Max salary
print(df.groupby('City')['Salary'].mean()) # Group by city

# Data cleaning
# Handle missing values

```

```

df_with_na = df.copy()
df_with_na.loc[2, 'Age'] = None

# Fill missing values
df_filled = df_with_na.fillna({'Age': df['Age'].mean()})

# Drop rows with missing values
df_dropped = df_with_na.dropna()

# Remove duplicates
df_no_dupes = df.drop_duplicates()

# PRACTICAL EXAMPLES

# 1. NumPy: Linear regression
class LinearRegression:
    def __init__(self):
        self.weights = None

    def fit(self, X, y):
        # Add bias term
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        # Normal equation:  $\theta = (X^T X)^{-1} X^T y$ 
        self.weights = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

    def predict(self, X):
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        return X_b @ self.weights

# 2. NumPy: Image processing
def image_brightness_adjust(image, factor):
    """Adjust image brightness"""
    return np.clip(image * factor, 0, 255).astype(np.uint8)

def image_blur(image, kernel_size=3):
    """Apply Gaussian blur"""
    kernel = np.ones((kernel_size, kernel_size)) / (kernel_size**2)
    from scipy import signal
    return signal.convolve2d(image, kernel, mode='same')

# 3. Pandas: Data analysis pipeline
def analyze_sales_data(df):
    """Analyze sales data"""
    # Summary statistics

```

```

summary = {
    'total_sales': df['amount'].sum(),
    'avg_sale': df['amount'].mean(),
    'max_sale': df['amount'].max(),
    'min_sale': df['amount'].min(),
    'total_transactions': len(df)
}

# Daily sales
daily_sales = df.groupby(df['date'].dt.date)['amount'].sum()

# Top products
top_products = df.groupby('product')['amount'].sum().nlargest(10)

# Customer segmentation
customer_value = df.groupby('customer_id')['amount'].sum()
segments = pd.cut(customer_value,
                    bins=[0, 100, 500, float('inf')],
                    labels=['Low', 'Medium', 'High'])

return summary, daily_sales, top_products, segments

```

4. Pandas: Data cleaning

```

def clean_dataset(df):
    """Clean dataset"""
    # Remove duplicates
    df = df.drop_duplicates()

    # Handle missing values
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    categorical_cols = df.select_dtypes(include=['object']).columns

    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
    df[categorical_cols] = df[categorical_cols].fillna('Unknown')

    # Remove outliers (IQR method)
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower) & (df[col] <= upper)]

```



```
return df
```

5. NumPy: Signal processing

```
def moving_average(signal, window_size):  
    """Calculate moving average"""  
    window = np.ones(window_size) / window_size  
    return np.convolve(signal, window, mode='valid')
```

```
def fft_analysis(signal, sample_rate):  
    """Perform FFT analysis"""  
    n = len(signal)  
    freq = np.fft.fftfreq(n, 1/sample_rate)  
    fft = np.fft.fft(signal)  
    magnitude = np.abs(fft)[:n//2]  
    return freq[:n//2], magnitude
```

6. Pandas: Time series analysis

```
def time_series_analysis(df, date_col, value_col):  
    """Analyze time series data"""  
    # Set datetime index  
    df[date_col] = pd.to_datetime(df[date_col])  
    df = df.set_index(date_col)  
  
    # Resample to different frequencies  
    daily = df[value_col].resample('D').mean()  
    weekly = df[value_col].resample('W').mean()  
    monthly = df[value_col].resample('M').mean()  
  
    # Rolling statistics  
    rolling_mean = df[value_col].rolling(window=7).mean()  
    rolling_std = df[value_col].rolling(window=7).std()  
  
    # Seasonal decomposition  
    from statsmodels.tsa.seasonal import seasonal_decompose  
    decomposition = seasonal_decompose(daily, model='additive', period=7)  
  
    return daily, weekly, monthly, rolling_mean, rolling_std, decomposition
```

7. NumPy: Monte Carlo simulation

```
def monte_carlo_pi(num_samples=10000):  
    """Estimate  $\pi$  using Monte Carlo method"""  
    points = np.random.rand(num_samples, 2)  
    inside_circle = np.sum(points[:, 0]**2 + points[:, 1]**2 <= 1)  
    return 4 * inside_circle / num_samples
```

8. Pandas: Pivot tables and crosstabs

```
def create_report(df):
    """Create pivot table report"""
    # Pivot table
    pivot = pd.pivot_table(df,
                            values='sales',
                            index='region',
                            columns='quarter',
                            aggfunc='sum',
                            fill_value=0)

    # Cross tabulation
    crosstab = pd.crosstab(df['category'],
                           df['status'],
                           margins=True,
                           normalize='index')

    return pivot, crosstab
```

9. NumPy: Neural network basics

```
class SimpleNeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.W1 = np.random.randn(input_size, hidden_size) * 0.01
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, output_size) * 0.01
        self.b2 = np.zeros((1, output_size))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2
```

10. Performance comparison: NumPy vs Pure Python

```
import timeit

# Python list
python_setup = """
numbers = list(range(1000000))
```

```

"""

python_code = """
result = [x**2 for x in numbers]
"""

# NumPy array
numpy_setup = """
import numpy as np
numbers = np.arange(1000000)
"""

numpy_code = """
result = numbers ** 2
"""

python_time = timeit.timeit(python_code, setup=python_setup, number=10)
numpy_time = timeit.timeit(numpy_code, setup=numpy_setup, number=10)

print(f"Python: {python_time:.4f}s")
print(f"NumPy: {numpy_time:.4f}s")
print(f"NumPy is {python_time/numpy_time:.1f}x faster")

```

Class Activity:

- Analyze a CSV dataset using Pandas (cleaning, analysis, visualization)
- Perform matrix operations and linear algebra with NumPy
- Create a simple data processing pipeline with NumPy and Pandas

Module 27: API Requests & JSON Handling

WHAT are APIs and JSON?

- **API:** Application Programming Interface for communication between systems
- **JSON:** JavaScript Object Notation for data exchange

WHEN to use APIs?

- Fetching external data
- Integrating with web services
- Building microservices
- Mobile app backends

WHERE to use APIs?

- Web applications
- Mobile apps
- Data collection
- Cloud services

HOW to work with APIs and JSON?

REQUESTS LIBRARY BASICS

import requests

GET request

```
response = requests.get('https://api.github.com')
print(response.status_code) # 200
print(response.headers)    # Response headers
print(response.text)       # Response as text
```

JSON response

```
data = response.json()    # Parse JSON
print(data['current_user'])
```

GET with parameters

```
params = {
    'q': 'python',
    'sort': 'stars',
    'order': 'desc'
}
response = requests.get('https://api.github.com/search/repositories', params=params)
print(response.url) # URL with parameters
```

POST request

```
data = {
    'title': 'My Post',
    'body': 'This is my post',
    'userId': 1
}
response = requests.post('https://jsonplaceholder.typicode.com/posts', json=data)
print(response.status_code) # 201
print(response.json())
```

PUT request (update)

```
data = {
    'id': 1,
    'title': 'Updated Title',
```

```

        'body': 'Updated content',
        'userId': 1
    }
    response = requests.put('https://jsonplaceholder.typicode.com/posts/1', json=data)

# PATCH request (partial update)
response = requests.patch('https://jsonplaceholder.typicode.com/posts/1',
                           json={'title': 'New Title'})

# DELETE request
response = requests.delete('https://jsonplaceholder.typicode.com/posts/1')

# HEAD request (headers only)
response = requests.head('https://api.github.com')
print(response.headers)

# OPTIONS request
response = requests.options('https://api.github.com')

# ERROR HANDLING
try:
    response = requests.get('https://api.github.com', timeout=5)
    response.raise_for_status() # Raises exception for 4xx/5xx responses
except requests.exceptions.RequestException as e:
    print(f"Request failed: {e}")

# AUTHENTICATION
# Basic Auth
response = requests.get('https://api.github.com/user',
                        auth=('username', 'password'))

# Token Auth (Bearer)
headers = {'Authorization': 'Bearer YOUR_TOKEN'}
response = requests.get('https://api.github.com/user', headers=headers)

# OAuth
from requests_oauthlib import OAuth1
auth = OAuth1('YOUR_APP_KEY', 'YOUR_APP_SECRET',
              'USER_OAUTH_TOKEN', 'USER_OAUTH_TOKEN_SECRET')
response = requests.get('https://api.twitter.com/1.1/account/verify_credentials.json',
                        auth=auth)

# SESSIONS (Persistent connections)
session = requests.Session()

```

```

session.headers.update({'User-Agent': 'MyApp/1.0'})
session.auth = ('username', 'password')

# Make multiple requests with same session
response1 = session.get('https://api.github.com/user')
response2 = session.get('https://api.github.com/user/repos')
session.close()

# FILE UPLOAD
files = {'file': open('data.txt', 'rb')}
response = requests.post('https://httpbin.org/post', files=files)

# JSON HANDLING
import json

# Parsing JSON
json_string = '{"name": "Alice", "age": 25, "city": "New York"}'
data = json.loads(json_string)
print(data['name']) # Alice

# Converting Python to JSON
python_dict = {
    'name': 'Bob',
    'age': 30,
    'hobbies': ['reading', 'coding'],
    'is_student': False,
    'score': 95.5,
    'address': None
}

json_string = json.dumps(python_dict, indent=2, sort_keys=True)
print(json_string)

# Reading/writing JSON files
# Write
with open('data.json', 'w') as f:
    json.dump(python_dict, f, indent=2)

# Read
with open('data.json', 'r') as f:
    data = json.load(f)

# Custom JSON serialization
class User:

```

```

def __init__(self, name, age):
    self.name = name
    self.age = age

def to_json(self):
    return {'name': self.name, 'age': self.age}

def user_encoder(obj):
    if isinstance(obj, User):
        return obj.to_json()
    raise TypeError(f"Object of type {type(obj)} is not JSON serializable")

user = User("Alice", 25)
json_string = json.dumps(user, default=user_encoder)

```

PRACTICAL API EXAMPLES

1. Weather API

```

class WeatherAPI:
    def __init__(self, api_key):
        self.api_key = api_key
        self.base_url = 'http://api.openweathermap.org/data/2.5'

    def get_current_weather(self, city):
        """Get current weather for city"""
        params = {
            'q': city,
            'appid': self.api_key,
            'units': 'metric'
        }
        response = requests.get(f'{self.base_url}/weather', params=params)
        response.raise_for_status()
        return response.json()

    def get_forecast(self, city, days=5):
        """Get weather forecast"""
        params = {
            'q': city,
            'appid': self.api_key,
            'units': 'metric',
            'cnt': days * 8 # 3-hour intervals
        }
        response = requests.get(f'{self.base_url}/forecast', params=params)
        response.raise_for_status()

```

```
    return response.json()
```

2. GitHub API

```
class GitHubAPI:
```

```
    def __init__(self, token=None):
        self.session = requests.Session()
        self.session.headers.update({'User-Agent': 'Python-API-Client'})
        if token:
            self.session.headers.update({'Authorization': f'token {token}'})
```

```
    def get_user(self, username):
        """Get user information"""
        response = self.session.get(f'https://api.github.com/users/{username}')
        response.raise_for_status()
        return response.json()
```

```
    def get_repos(self, username):
        """Get user repositories"""
        response = self.session.get(f'https://api.github.com/users/{username}/repos')
        response.raise_for_status()
        return response.json()
```

```
    def search_repos(self, query, sort='stars', order='desc'):
        """Search repositories"""
        params = {
            'q': query,
            'sort': sort,
            'order': order
        }
        response = self.session.get('https://api.github.com/search/repositories',
                                    params=params)
        response.raise_for_status()
        return response.json()
```

3. REST API Client with retry and error handling

```
class APIClient:
```

```
    def __init__(self, base_url, timeout=30, max_retries=3):
        self.base_url = base_url.rstrip('/')
        self.timeout = timeout
        self.max_retries = max_retries
        self.session = requests.Session()
```

```
    def request(self, method, endpoint, **kwargs):
        """Make API request with retry logic"""
```



```

url = f"{self.base_url}/{endpoint.lstrip('/')}"

for attempt in range(self.max_retries):
    try:
        response = self.session.request(
            method, url,
            timeout=self.timeout,
            **kwargs
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        if attempt == self.max_retries - 1:
            raise
        print(f"Attempt {attempt + 1} failed: {e}")
        import time
        time.sleep(2 ** attempt) # Exponential backoff

def get(self, endpoint, **kwargs):
    return self.request('GET', endpoint, **kwargs)

def post(self, endpoint, **kwargs):
    return self.request('POST', endpoint, **kwargs)

def put(self, endpoint, **kwargs):
    return self.request('PUT', endpoint, **kwargs)

def delete(self, endpoint, **kwargs):
    return self.request('DELETE', endpoint, **kwargs)

```

4. Async API requests

```

import asyncio
import aiohttp

```

```

async def fetch_url(session, url):
    """Fetch URL asynchronously"""
    try:
        async with session.get(url) as response:
            return await response.json()
    except Exception as e:
        return {'error': str(e)}

async def fetch_multiple_urls(urls):
    """Fetch multiple URLs concurrently"""

```

```
async with aiohttp.ClientSession() as session:
    tasks = [fetch_url(session, url) for url in urls]
    results = await asyncio.gather(*tasks)
    return results
```

5. Webhook receiver (using Flask)

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

```
@app.route('/webhook', methods=['POST'])
```

```
def webhook():
```

```
    """Receive webhook data"""
```

```
    data = request.json
```

```
    print(f"Received webhook: {data}")
```

```
    # Process webhook data
```

```
    event_type = request.headers.get('X-Event-Type')
```

```
    # Return response
```

```
    return jsonify({'status': 'success', 'message': 'Webhook received'})
```

6. API pagination

```
def get_all_pages(base_url, params=None, per_page=100):
```

```
    """Fetch all pages from paginated API"""
```

```
    all_data = []
```

```
    page = 1
```

```
    while True:
```

```
        params = params or {}
```

```
        params['page'] = page
```

```
        params['per_page'] = per_page
```

```
        response = requests.get(base_url, params=params)
```

```
        response.raise_for_status()
```

```
        data = response.json()
```

```
        if not data:
```

```
            break
```

```
        all_data.extend(data)
```

```
        page += 1
```

```
    # Check if we've reached the last page
```

```

        if len(data) < per_page:
            break

    return all_data

# 7. Rate limiting
import time
from functools import wraps

def rate_limit(calls_per_second):
    """Decorator to rate limit API calls"""
    def decorator(func):
        last_called = [0.0]

        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            if elapsed < 1.0 / calls_per_second:
                time.sleep((1.0 / calls_per_second) - elapsed)
            result = func(*args, **kwargs)
            last_called[0] = time.time()
            return result
        return wrapper
    return decorator

@rate_limit(5) # Max 5 calls per second
def api_call():
    response = requests.get('https://api.github.com')
    return response.status_code

# 8. Caching API responses
import functools
import hashlib
import pickle

class APICache:
    def __init__(self, cache_dir='.api_cache'):
        self.cache_dir = cache_dir
        import os
        os.makedirs(cache_dir, exist_ok=True)

    def _get_cache_key(self, url, params):
        """Generate cache key from URL and params"""
        key_data = f"{url}{sorted(params.items())}" if params else url

```

```

        return hashlib.md5(key_data.encode()).hexdigest()

def get(self, url, params=None, ttl=3600):
    """Get cached response"""
    cache_key = self._get_cache_key(url, params)
    cache_file = f"{self.cache_dir}/{cache_key}.pickle"

    import os
    import time
    if os.path.exists(cache_file):
        with open(cache_file, 'rb') as f:
            cached_data, timestamp = pickle.load(f)
            if time.time() - timestamp < ttl:
                return cached_data
    return None

def set(self, url, params, data):
    """Cache response"""
    cache_key = self._get_cache_key(url, params)
    cache_file = f"{self.cache_dir}/{cache_key}.pickle"

    import time
    with open(cache_file, 'wb') as f:
        pickle.dump((data, time.time()), f)

def clear(self):
    """Clear cache"""
    import os
    import glob
    for f in glob.glob(f"{self.cache_dir}/*.pickle"):
        os.remove(f)

# 9. API client with circuit breaker
class CircuitBreaker:
    def __init__(self, failure_threshold=5, recovery_timeout=60):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failure_count = 0
        self.last_failure_time = None
        self.state = 'CLOSED' # CLOSED, OPEN, HALF_OPEN

    def call(self, func, *args, **kwargs):
        """Call function with circuit breaker"""
        if self.state == 'OPEN':

```

```

import time
if time.time() - self.last_failure_time > self.recovery_timeout:
    self.state = 'HALF_OPEN'
else:
    raise Exception("Circuit breaker is OPEN")

try:
    result = func(*args, **kwargs)
    if self.state == 'HALF_OPEN':
        self.state = 'CLOSED'
        self.failure_count = 0
    return result
except Exception as e:
    self.failure_count += 1
    self.last_failure_time = time.time()

    if self.failure_count >= self.failure_threshold:
        self.state = 'OPEN'
        raise e

```

Class Activity:

- Build a weather app using OpenWeatherMap API
- Create a GitHub repository analyzer with the GitHub API
- Implement a REST API client with caching and rate limiting

Module 28: Working with Databases in Python

WHAT are Databases?

Databases store and manage structured data. SQLite and MySQL are popular relational databases. Databases like MongoDB are NoSQL databases or document databases.

WHEN to use databases?

- Persistent data storage
- Multi-user applications
- Complex queries
- Transaction management

WHERE to use databases?

- Web applications
- Desktop applications

- Data analysis
- Enterprise systems

HOW to work with databases?

SQLITE BASICS

```
import sqlite3
```

```
# Connect to database (creates if not exists)
```

```
conn = sqlite3.connect('example.db')
```

```
cursor = conn.cursor()
```

```
# Create table
```

```
cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS users (
```

```
        id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
        name TEXT NOT NULL,
```

```
        email TEXT UNIQUE NOT NULL,
```

```
        age INTEGER,
```

```
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
    )
```

```
""")
```

```
# Insert data
```

```
# Single row
```

```
cursor.execute("""
```

```
    INSERT INTO users (name, email, age)
```

```
    VALUES (?, ?, ?)
```

```
""", ('Alice', 'alice@example.com', 25))
```

```
# Multiple rows
```

```
users = [
```

```
    ('Bob', 'bob@example.com', 30),
```

```
    ('Charlie', 'charlie@example.com', 35),
```

```
    ('David', 'david@example.com', 28)
```

```
]
```

```
cursor.executemany('INSERT INTO users (name, email, age) VALUES (?, ?, ?)', users)
```

```
# Commit changes
```

```
conn.commit()
```

```
# Query data
```

```
# Fetch all
```

```
cursor.execute('SELECT * FROM users')
```

```
all_users = cursor.fetchall()
```

```

print(all_users)

# Fetch one
cursor.execute('SELECT * FROM users WHERE name = ?', ('Alice',))
user = cursor.fetchone()
print(user)

# Fetch many
cursor.execute('SELECT * FROM users')
first_two = cursor.fetchmany(2)
print(first_two)

# Query with conditions
cursor.execute("""
    SELECT * FROM users
    WHERE age > ?
    ORDER BY age DESC
""", (25,))
older_users = cursor.fetchall()
print(older_users)

# Update data
cursor.execute("""
    UPDATE users
    SET age = ?
    WHERE name = ?
""", (26, 'Alice'))
conn.commit()

# Delete data
cursor.execute('DELETE FROM users WHERE name = ?', ('David',))
conn.commit()

# Using with statement (auto-commit/rollback)
with sqlite3.connect('example.db') as conn:
    cursor = conn.cursor()
    cursor.execute('INSERT INTO users (name, email, age) VALUES (?, ?, ?)',
                   ('Eve', 'eve@example.com', 32))

# ADVANCED SQLITE
# Row factory for dictionary results
conn = sqlite3.connect('example.db')
conn.row_factory = sqlite3.Row

```

```

cursor = conn.cursor()
cursor.execute('SELECT * FROM users')
for row in cursor:
    print(f"Name: {row['name']}, Email: {row['email']}")

# Transactions
try:
    conn.execute('BEGIN TRANSACTION')
    conn.execute('INSERT INTO users (name, email, age) VALUES (?, ?, ?)',
                  ('Frank', 'frank@example.com', 40))
    conn.execute('UPDATE users SET age = age + 1 WHERE name = ?', ('Bob',))
    conn.commit()
except Exception as e:
    conn.rollback()
    print(f"Transaction failed: {e}")

# Using parameters safely (prevents SQL injection)
# Good: parameterized queries
cursor.execute('SELECT * FROM users WHERE name = ?', (user_input,))

# Bad: string formatting (SQL injection risk!)
# cursor.execute(f'SELECT * FROM users WHERE name = "{user_input}"')

# MYSQL WITH PYTHON
import mysql.connector
from mysql.connector import Error

try:
    connection = mysql.connector.connect(
        host='localhost',
        database='mydb',
        user='root',
        password='password'
    )

    cursor = connection.cursor(dictionary=True) # Returns dicts

    # Create table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS products (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            price DECIMAL(10,2),
            stock INT DEFAULT 0
    """)

```



```

    )
    """

# Insert with auto-commit
cursor.execute("""
    INSERT INTO products (name, price, stock)
    VALUES (%s, %s, %s)
    """, ('Laptop', 999.99, 10))
connection.commit()

# Query
cursor.execute('SELECT * FROM products')
products = cursor.fetchall()

except Error as e:
    print(f"MySQL error: {e}")
finally:
    if connection.is_connected():
        cursor.close()
        connection.close()

# ORM WITH SQLALCHEMY
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship
from datetime import datetime

# Create engine
engine = create_engine('sqlite:///orm_example.db', echo=True)
Base = declarative_base()

# Define models
class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    email = Column(String(100), unique=True)
    created_at = Column(DateTime, default=datetime.now)

# Relationship
orders = relationship('Order', back_populates='user')

def __repr__(self):

```

```

        return f"<User(name='{self.name}', email='{self.email}')>"

class Product(Base):
    __tablename__ = 'products'

    id = Column(Integer, primary_key=True)
    name = Column(String(100))
    price = Column(Float)
    stock = Column(Integer, default=0)

    def __repr__(self):
        return f"<Product(name='{self.name}', price={self.price})>"

class Order(Base):
    __tablename__ = 'orders'

    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'))
    product_id = Column(Integer, ForeignKey('products.id'))
    quantity = Column(Integer)
    order_date = Column(DateTime, default=datetime.now)

    user = relationship('User', back_populates='orders')
    product = relationship('Product')

# Create tables
Base.metadata.create_all(engine)

# Create session
Session = sessionmaker(bind=engine)
session = Session()

# CRUD operations
# Create
new_user = User(name='Alice', email='alice@example.com')
session.add(new_user)
session.commit()

# Read
users = session.query(User).all()
user = session.query(User).filter_by(name='Alice').first()
user_by_id = session.get(User, 1)

# Query with conditions

```

```

older_users = session.query(User).filter(User.id > 1).all()
sorted_users = session.query(User).order_by(User.name.desc()).all()

# Update
user = session.query(User).filter_by(name='Alice').first()
user.age = 26
session.commit()

# Delete
user = session.query(User).filter_by(name='Alice').first()
session.delete(user)
session.commit()

# Relationships
# Create product
product = Product(name='Laptop', price=999.99, stock=10)
session.add(product)
session.commit()

# Create order
order = Order(user_id=1, product_id=1, quantity=1)
session.add(order)
session.commit()

# Query with joins
orders = session.query(Order).join(User).filter(User.name == 'Alice').all()

# Aggregations
from sqlalchemy import func
total_orders = session.query(func.count(Order.id)).scalar()
avg_price = session.query(func.avg(Product.price)).scalar()

# Connection pooling
from sqlalchemy.pool import QueuePool

engine = create_engine(
    'sqlite:///example.db',
    poolclass=QueuePool,
    pool_size=5,
    max_overflow=10,
    pool_pre_ping=True
)

```

PRACTICAL EXAMPLES

1. Database connection manager

class DatabaseManager:

```
def __init__(self, db_path='database.db'):  
    self.db_path = db_path
```

```
@contextlib.contextmanager
```

```
def connection(self):
```

```
    conn = sqlite3.connect(self.db_path)  
    conn.row_factory = sqlite3.Row  
    try:  
        yield conn  
        conn.commit()  
    except Exception:  
        conn.rollback()  
        raise  
    finally:  
        conn.close()
```

```
def execute_query(self, query, params=None):
```

```
    with self.connection() as conn:  
        cursor = conn.cursor()  
        cursor.execute(query, params or [])  
        return cursor.fetchall()
```

```
def execute_many(self, query, params_list):
```

```
    with self.connection() as conn:  
        cursor = conn.cursor()  
        cursor.executemany(query, params_list)
```

2. Data migration tool

class DataMigration:

```
def __init__(self, source_db, target_db):  
    self.source = DatabaseManager(source_db)  
    self.target = DatabaseManager(target_db)
```

```
def migrate_users(self):
```

```
    """Migrate users from source to target"""
```

```
    users = self.source.execute_query('SELECT * FROM users')
```

```
    for user in users:
```

```
        self.target.execute_query("""  
            INSERT OR IGNORE INTO users (name, email, age)  
            VALUES (?, ?, ?)
```

```

        "", (user['name'], user['email'], user['age']))

    print(f"Migrated {len(users)} users")

# 3. Database backup utility
def backup_database(db_path, backup_path):
    """Create database backup"""
    import shutil
    import datetime

    timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
    backup_file = f"{backup_path}/backup_{timestamp}.db"
    shutil.copy2(db_path, backup_file)
    return backup_file

# 4. Query builder
class QueryBuilder:
    def __init__(self, table):
        self.table = table
        self.select_fields = ["*"]
        self.where_clauses = []
        self.order_by = None
        self.limit_val = None

    def select(self, *fields):
        self.select_fields = fields
        return self

    def where(self, condition, params):
        self.where_clauses.append((condition, params))
        return self

    def order(self, field, direction='ASC'):
        self.order_by = f"{field} {direction}"
        return self

    def limit(self, limit):
        self.limit_val = limit
        return self

    def build(self):
        query = f"SELECT {', '.join(self.select_fields)} FROM {self.table}"

        if self.where_clauses:

```

```
conditions = [cond for cond, _ in self.where_clauses]
query += " WHERE " + " AND ".join(conditions)
```

```
if self.order_by:
    query += f" ORDER BY {self.order_by}"
```

```
if self.limit_val:
    query += f" LIMIT {self.limit_val}"
```

```
params = [param for _, param in self.where_clauses]
```

```
return query, params
```

Usage

```
query, params = (QueryBuilder('users')
    .select('name', 'email')
    .where('age > ?', 18)
    .where('city = ?', 'New York')
    .order('name')
    .limit(10)
    .build())
```

5. Connection pool for MySQL

```
from mysql.connector import pooling
```

```
dbconfig = {
    'host': 'localhost',
    'user': 'root',
    'password': 'password',
    'database': 'mydb'
}
```

```
connection_pool = pooling.MySQLConnectionPool(
    pool_name='mypool',
    pool_size=5,
    **dbconfig
)
```

```
def get_connection():
    return connection_pool.get_connection()
```

6. Database model with validation

```
class Model:
    def __init__(self, **kwargs):
```

```

        for key, value in kwargs.items():
            setattr(self, key, value)

    @classmethod
    def create_table(cls, conn):
        raise NotImplementedError

    def save(self, conn):
        raise NotImplementedError

    @classmethod
    def find(cls, conn, **conditions):
        raise NotImplementedError

class UserModel(Model):
    table = 'users'

    @classmethod
    def create_table(cls, conn):
        conn.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                email TEXT UNIQUE NOT NULL,
                age INTEGER CHECK(age >= 0 AND age <= 150)
            )
        """)

    def save(self, conn):
        cursor = conn.cursor()
        if hasattr(self, 'id'):
            cursor.execute("""
                UPDATE users
                SET name=?, email=?, age=?
                WHERE id=?
            """, (self.name, self.email, self.age, self.id))
        else:
            cursor.execute("""
                INSERT INTO users (name, email, age)
                VALUES (?, ?, ?)
            """, (self.name, self.email, self.age))
            self.id = cursor.lastrowid
        conn.commit()

```

```

@classmethod
def find(cls, conn, **conditions):
    where = " AND ".join([f"{k}=?" for k in conditions.keys()])
    query = f"SELECT * FROM {cls.table} WHERE {where}"
    cursor = conn.cursor()
    cursor.execute(query, list(conditions.values()))
    return [cls(**dict(zip([desc[0] for desc in cursor.description], row)))
            for row in cursor.fetchall()]

```

7. Database migration system

class Migration:

```

def __init__(self, conn):
    self.conn = conn
    self.create_migrations_table()

```

```

def create_migrations_table(self):
    self.conn.execute("""
        CREATE TABLE IF NOT EXISTS migrations (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            version INTEGER NOT NULL,
            applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    self.conn.commit()

```

```

def get_current_version(self):
    cursor = self.conn.execute('SELECT MAX(version) FROM migrations')
    result = cursor.fetchone()[0]
    return result or 0

```

```

def migrate(self, migrations):
    current = self.get_current_version()
    for version, (up, down) in sorted(migrations.items()):
        if version > current:
            print(f"Applying migration {version}")
            up(self.conn)
            self.conn.execute('INSERT INTO migrations (version) VALUES (?)', (version,))
            self.conn.commit()

```

```

def rollback(self, migrations, target_version):
    current = self.get_current_version()
    for version in range(current, target_version, -1):
        if version in migrations:
            print(f"Rolling back migration {version}")

```



```
_, down = migrations[version]
down(self.conn)
self.conn.execute('DELETE FROM migrations WHERE version = ?', (version,))
self.conn.commit()
```

Class Activity:

- Build a simple CRUD application using SQLite and tkinter
- Create a database migration system
- Implement a connection pool for MySQL

STAGE 5:

PYTHON FOR PROJECTS (Modules 29-30)

Module 29: Build a Mini Project

Project Options

Choose one of these projects to build:

1. **Calculator Application**
2. **To-Do List Application**
3. **API Caller Application**
4. **Weather App**
5. **Currency Converter**
6. **CRUD Application - Student Database**

Let's build a **Comprehensive To-Do List Application** with all the features!

```
"""
To-Do List Application with:
- Task management (add, edit, delete, complete)
- Categories and priorities
- Due dates and reminders
- Search and filter
- Data persistence (SQLite)
- GUI (Tkinter) or CLI
"""
```

```
import sqlite3
import datetime
```

```

import json
import os
from typing import List, Dict, Optional
from dataclasses import dataclass, asdict
from enum import Enum

# ENUMS
class Priority(Enum):
    LOW = 1
    MEDIUM = 2
    HIGH = 3

    def __str__(self):
        return self.name.title()

class Category(Enum):
    PERSONAL = "Personal"
    WORK = "Work"
    STUDY = "Study"
    SHOPPING = "Shopping"
    HEALTH = "Health"
    OTHER = "Other"

# DATA MODEL
@dataclass
class Task:
    """Task data model"""
    id: Optional[int] = None
    title: str = ""
    description: str = ""
    priority: Priority = Priority.MEDIUM
    category: Category = Category.OTHER
    due_date: Optional[datetime.date] = None
    completed: bool = False
    created_at: datetime.datetime = datetime.datetime.now()

    def to_dict(self):
        """Convert to dictionary for JSON"""
        data = asdict(self)
        data['priority'] = self.priority.value
        data['category'] = self.category.value
        data['due_date'] = self.due_date.isoformat() if self.due_date else None
        data['created_at'] = self.created_at.isoformat()
        return data

```

```

    @classmethod
    def from_dict(cls, data):
        """Create Task from dictionary"""
        return cls(
            id=data.get('id'),
            title=data['title'],
            description=data.get('description', ''),
            priority=Priority(data['priority']),
            category=Category(data['category']),
            due_date=datetime.date.fromisoformat(data['due_date']) if data.get('due_date')
        else None,
            completed=data.get('completed', False),
            created_at=datetime.datetime.fromisoformat(data['created_at'])
        )

```

DATABASE MANAGER

```

class DatabaseManager:
    """Handle database operations"""

    def __init__(self, db_path='todo.db'):
        self.db_path = db_path
        self.init_database()

    def get_connection(self):
        """Get database connection"""
        return sqlite3.connect(self.db_path)

    def init_database(self):
        """Create tables if they don't exist"""
        with self.get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS tasks (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    title TEXT NOT NULL,
                    description TEXT,
                    priority INTEGER NOT NULL,
                    category TEXT NOT NULL,
                    due_date TEXT,
                    completed INTEGER DEFAULT 0,
                    created_at TEXT NOT NULL
                )
            """)

```

```
conn.commit()
```

```
def add_task(self, task: Task) -> int:
    """Add new task to database"""
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO tasks
            (title, description, priority, category, due_date, completed, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (
            task.title,
            task.description,
            task.priority.value,
            task.category.value,
            task.due_date.isoformat() if task.due_date else None,
            1 if task.completed else 0,
            task.created_at.isoformat()
        ))
        conn.commit()
        return cursor.lastrowid
```

```
def get_tasks(self, completed: bool = None,
               category: Category = None,
               priority: Priority = None,
               search: str = None) -> List[Task]:
    """Get tasks with filters"""
    query = "SELECT * FROM tasks WHERE 1=1"
    params = []
```

```
    if completed is not None:
        query += " AND completed = ?"
        params.append(1 if completed else 0)
```

```
    if category:
        query += " AND category = ?"
        params.append(category.value)
```

```
    if priority:
        query += " AND priority = ?"
        params.append(priority.value)
```

```
    if search:
        query += " AND (title LIKE ? OR description LIKE ?)"
```

```
params.extend([f'%{search}%', f'%{search}%'])
```

```
query += " ORDER BY priority DESC, due_date ASC"
```

```
with self.get_connection() as conn:
```

```
    cursor = conn.cursor()
```

```
    cursor.execute(query, params)
```

```
    rows = cursor.fetchall()
```

```
tasks = []
```

```
for row in rows:
```

```
    task = Task(
```

```
        id=row[0],
```

```
        title=row[1],
```

```
        description=row[2],
```

```
        priority=Priority(row[3]),
```

```
        category=Category(row[4]),
```

```
        due_date=datetime.date.fromisoformat(row[5]) if row[5] else None,
```

```
        completed=bool(row[6]),
```

```
        created_at=datetime.datetime.fromisoformat(row[7])
```

```
    )
```

```
    tasks.append(task)
```

```
return tasks
```

```
def update_task(self, task: Task):
```

```
    """Update existing task"""
```

```
    with self.get_connection() as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            UPDATE tasks
```

```
            SET title=?, description=?, priority=?, category=?,
```

```
                due_date=?, completed=?
```

```
            WHERE id=?
```

```
        """, (
```

```
            task.title,
```

```
            task.description,
```

```
            task.priority.value,
```

```
            task.category.value,
```

```
            task.due_date.isoformat() if task.due_date else None,
```

```
            1 if task.completed else 0,
```

```
            task.id
```

```
        ))
```

```
        conn.commit()
```

```

def delete_task(self, task_id: int):
    """Delete task by ID"""
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('DELETE FROM tasks WHERE id=?', (task_id,))
        conn.commit()

def toggle_complete(self, task_id: int):
    """Toggle task completion status"""
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("
            UPDATE tasks
            SET completed = NOT completed
            WHERE id=?
        ", (task_id,))
        conn.commit()

# TASK MANAGER (Business Logic)
class TaskManager:
    """Manage tasks with business logic"""

    def __init__(self, db_manager: DatabaseManager):
        self.db = db_manager
        self.tasks = []
        self.refresh()

    def refresh(self):
        """Refresh tasks from database"""
        self.tasks = self.db.get_tasks()

    def add_task(self, title: str, description: str = "",
                 priority: Priority = Priority.MEDIUM,
                 category: Category = Category.OTHER,
                 due_date: Optional[datetime.date] = None) -> Task:
        """Add new task"""
        task = Task(
            title=title,
            description=description,
            priority=priority,
            category=category,
            due_date=due_date
        )
        task.id = self.db.add_task(task)

```

```

        self.refresh()
        return task

def update_task(self, task_id: int, **kwargs):
    """Update task attributes"""
    task = self.get_task(task_id)
    if task:
        for key, value in kwargs.items():
            if hasattr(task, key):
                setattr(task, key, value)
        self.db.update_task(task)
        self.refresh()

def delete_task(self, task_id: int):
    """Delete task"""
    self.db.delete_task(task_id)
    self.refresh()

def complete_task(self, task_id: int):
    """Mark task as complete"""
    self.db.toggle_complete(task_id)
    self.refresh()

def get_task(self, task_id: int) -> Optional[Task]:
    """Get task by ID"""
    for task in self.tasks:
        if task.id == task_id:
            return task
    return None

def get_tasks_by_status(self, completed: bool = False) -> List[Task]:
    """Get tasks by completion status"""
    return [t for t in self.tasks if t.completed == completed]

def get_overdue_tasks(self) -> List[Task]:
    """Get overdue tasks"""
    today = datetime.date.today()
    return [t for t in self.tasks
            if not t.completed and t.due_date and t.due_date < today]

def get_tasks_by_category(self, category: Category) -> List[Task]:
    """Get tasks by category"""
    return [t for t in self.tasks if t.category == category]

```

```

def get_tasks_by_priority(self, priority: Priority) -> List[Task]:
    """Get tasks by priority"""
    return [t for t in self.tasks if t.priority == priority]

def search_tasks(self, query: str) -> List[Task]:
    """Search tasks by title or description"""
    query_lower = query.lower()
    return [t for t in self.tasks
            if query_lower in t.title.lower()
            or query_lower in t.description.lower()]

def get_statistics(self) -> Dict:
    """Get task statistics"""
    total = len(self.tasks)
    completed = len([t for t in self.tasks if t.completed])
    pending = total - completed
    overdue = len(self.get_overdue_tasks())

    by_priority = {
        priority: len([t for t in self.tasks if t.priority == priority])
        for priority in Priority
    }

    by_category = {
        category: len([t for t in self.tasks if t.category == category])
        for category in Category
    }

    return {
        'total': total,
        'completed': completed,
        'pending': pending,
        'overdue': overdue,
        'completion_rate': (completed / total * 100) if total > 0 else 0,
        'by_priority': by_priority,
        'by_category': by_category
    }

```

CLI INTERFACE

```

class TodoCLI:
    """Command-line interface for Todo app"""

    def __init__(self):
        self.db = DatabaseManager()

```



```

        self.manager = TaskManager(self.db)

def run(self):
    """Main CLI loop"""
    print("=" * 50)
    print("Welcome to To-Do List Manager!")
    print("=" * 50)

    while True:
        self.show_menu()
        choice = input("\nChoose an option: ").strip()

        if choice == '1':
            self.list_tasks()
        elif choice == '2':
            self.add_task()
        elif choice == '3':
            self.edit_task()
        elif choice == '4':
            self.delete_task()
        elif choice == '5':
            self.complete_task()
        elif choice == '6':
            self.search_tasks()
        elif choice == '7':
            self.show_statistics()
        elif choice == '8':
            print("Goodbye!")
            break
        else:
            print("Invalid option. Please try again.")

def show_menu(self):
    """Display menu"""
    print("\n" + "-" * 30)
    print("1. List all tasks")
    print("2. Add new task")
    print("3. Edit task")
    print("4. Delete task")
    print("5. Complete task")
    print("6. Search tasks")
    print("7. Statistics")
    print("8. Exit")

```

```

def list_tasks(self, tasks: List[Task] = None):
    """Display tasks"""
    tasks = tasks or self.manager.get_tasks_by_status(completed=False)

    if not tasks:
        print("\nNo pending tasks!")
        return

    print("\nPending Tasks:")
    print("-" * 80)
    for task in tasks:
        status = "[✓]" if task.completed else "[ ]"
        priority_icon = {
            Priority.HIGH: "🔴",
            Priority.MEDIUM: "🟡",
            Priority.LOW: "🟢"
        }[task.priority]

        due_info = f"Due: {task.due_date}" if task.due_date else "No due date"

        print(f"{status} {priority_icon} [{task.id}] {task.title}")
        print(f"  Category: {task.category}")
        print(f"  Priority: {task.priority}")
        print(f"  {due_info}")
        if task.description:
            print(f"  Description: {task.description}")
        print("-" * 80)

def add_task(self):
    """Add new task"""
    print("\nAdd New Task")
    print("-" * 30)

    title = input("Title: ").strip()
    if not title:
        print("Title cannot be empty!")
        return

    description = input("Description (optional): ").strip()

    print("\nPriority:")
    for p in Priority:
        print(f"  {p.value}. {p.name.title()}")
    priority_choice = input("Priority (1-3) [2]: ").strip()

```

```

priority = Priority.MEDIUM
if priority_choice:
    try:
        priority = Priority(int(priority_choice))
    except (ValueError, KeyError):
        pass

print("\nCategory:")
for c in Category:
    print(f" {c.value}")
category_choice = input("Category [Other]: ").strip()
category = Category.OTHER
if category_choice:
    try:
        category = Category(category_choice)
    except ValueError:
        pass

due_date_str = input("Due date (YYYY-MM-DD, optional): ").strip()
due_date = None
if due_date_str:
    try:
        due_date = datetime.date.fromisoformat(due_date_str)
    except ValueError:
        print("Invalid date format. Skipping due date.")

task = self.manager.add_task(title, description, priority, category, due_date)
print(f"\n✓ Task added with ID: {task.id}")

def edit_task(self):
    """Edit existing task"""
    task_id = self.get_task_id()
    if not task_id:
        return

    task = self.manager.get_task(task_id)
    if not task:
        print(f"Task {task_id} not found!")
        return

    print(f"\nEditing task: {task.title}")
    print("Leave blank to keep current value")

    title = input(f"Title [{task.title}]: ").strip()

```

```

if title:
    task.title = title

description = input(f"Description [{task.description}]: ").strip()
if description:
    task.description = description

print(f"\nPriority ({task.priority}):")
for p in Priority:
    print(f" {p.value}. {p.name.title()}")
priority_choice = input("Priority (1-3): ").strip()
if priority_choice:
    try:
        task.priority = Priority(int(priority_choice))
    except (ValueError, KeyError):
        pass

print(f"\nCategory ({task.category}):")
for c in Category:
    print(f" {c.value}")
category_choice = input("Category: ").strip()
if category_choice:
    try:
        task.category = Category(category_choice)
    except ValueError:
        pass

due_date_str = input(f"Due date ({task.due_date or 'none'}): ").strip()
if due_date_str:
    try:
        task.due_date = datetime.date.fromisoformat(due_date_str)
    except ValueError:
        print("Invalid date format. Keeping current.")

self.manager.update_task(task_id,
                          title=task.title,
                          description=task.description,
                          priority=task.priority,
                          category=task.category,
                          due_date=task.due_date)
print(f"\n✓ Task {task_id} updated!")

def delete_task(self):
    """Delete task"""

```

```

        task_id = self.get_task_id()
        if not task_id:
            return

        task = self.manager.get_task(task_id)
        if not task:
            print(f"Task {task_id} not found!")
            return

        confirm = input(f"Delete '{task.title}'? (y/n): ").lower()
        if confirm == 'y':
            self.manager.delete_task(task_id)
            print(f"✓ Task {task_id} deleted!")

    def complete_task(self):
        """Mark task as complete"""
        task_id = self.get_task_id()
        if not task_id:
            return

        task = self.manager.get_task(task_id)
        if not task:
            print(f"Task {task_id} not found!")
            return

        self.manager.complete_task(task_id)
        print(f"✓ Task '{task.title}' marked as complete!")

    def search_tasks(self):
        """Search for tasks"""
        query = input("Search query: ").strip()
        if not query:
            return

        results = self.manager.search_tasks(query)
        if results:
            print(f"\nFound {len(results)} tasks:")
            self.list_tasks(results)
        else:
            print("No tasks found.")

    def show_statistics(self):
        """Display statistics"""
        stats = self.manager.get_statistics()

```

```

print("\n📊 Task Statistics")
print("=" * 40)
print(f"Total tasks: {stats['total']}")
print(f"Completed: {stats['completed']}")
print(f"Pending: {stats['pending']}")
print(f"Overdue: {stats['overdue']}")
print(f"Completion rate: {stats['completion_rate']:.1f}%")

print("\nBy Priority:")
for priority, count in stats['by_priority'].items():
    if count > 0:
        print(f" {priority.name.title()}: {count}")

print("\nBy Category:")
for category, count in stats['by_category'].items():
    if count > 0:
        print(f" {category.value}: {count}")

def get_task_id(self) -> Optional[int]:
    """Get task ID from user input"""
    try:
        task_id = int(input("Task ID: "))
        return task_id
    except ValueError:
        print("Invalid task ID!")
        return None

# GUI INTERFACE (Tkinter)
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
from datetime import datetime

class TodoGUI:
    """Graphical interface for Todo app"""

    def __init__(self):
        self.db = DatabaseManager()
        self.manager = TaskManager(self.db)

        self.root = tk.Tk()
        self.root.title("To-Do List Manager")
        self.root.geometry("1000x600")

```

```

self.setup_ui()
self.refresh_task_list()

def setup_ui(self):
    """Setup UI components"""
    # Main frame
    main_frame = ttk.Frame(self.root, padding="10")
    main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    # Toolbar
    toolbar = ttk.Frame(main_frame)
    toolbar.grid(row=0, column=0, columnspan=2, sticky=(tk.W, tk.E), pady=5)

    ttk.Button(toolbar, text="Add Task",
command=self.add_task_dialog).pack(side=tk.LEFT, padx=5)
    ttk.Button(toolbar, text="Edit Task",
command=self.edit_task_dialog).pack(side=tk.LEFT, padx=5)
    ttk.Button(toolbar, text="Delete Task",
command=self.delete_task).pack(side=tk.LEFT, padx=5)
    ttk.Button(toolbar, text="Complete Task",
command=self.complete_task).pack(side=tk.LEFT, padx=5)

    ttk.Separator(toolbar, orient=tk.VERTICAL).pack(side=tk.LEFT, padx=5, fill=tk.Y)

    # Filter
    ttk.Label(toolbar, text="Show:").pack(side=tk.LEFT, padx=5)
    self.filter_var = tk.StringVar(value="pending")
    ttk.Radiobutton(toolbar, text="Pending", variable=self.filter_var,
value="pending", command=self.refresh_task_list).pack(side=tk.LEFT)
    ttk.Radiobutton(toolbar, text="Completed", variable=self.filter_var,
value="completed", command=self.refresh_task_list).pack(side=tk.LEFT)
    ttk.Radiobutton(toolbar, text="All", variable=self.filter_var,
value="all", command=self.refresh_task_list).pack(side=tk.LEFT)

    ttk.Separator(toolbar, orient=tk.VERTICAL).pack(side=tk.LEFT, padx=5, fill=tk.Y)

    # Search
    ttk.Label(toolbar, text="Search:").pack(side=tk.LEFT, padx=5)
    self.search_var = tk.StringVar()
    self.search_var.trace('w', lambda *args: self.refresh_task_list())
    ttk.Entry(toolbar, textvariable=self.search_var, width=30).pack(side=tk.LEFT)

    # Task list
    columns = ('ID', 'Status', 'Priority', 'Title', 'Category', 'Due Date')

```

```
self.task_tree = ttk.Treeview(main_frame, columns=columns, show='headings',
height=20)
```

```
# Define headings
self.task_tree.heading('ID', text='ID')
self.task_tree.heading('Status', text='✓')
self.task_tree.heading('Priority', text='Priority')
self.task_tree.heading('Title', text='Title')
self.task_tree.heading('Category', text='Category')
self.task_tree.heading('Due Date', text='Due Date')
```

```
# Set column widths
self.task_tree.column('ID', width=50)
self.task_tree.column('Status', width=50)
self.task_tree.column('Priority', width=80)
self.task_tree.column('Title', width=400)
self.task_tree.column('Category', width=100)
self.task_tree.column('Due Date', width=100)
```

```
# Scrollbar
scrollbar = ttk.Scrollbar(main_frame, orient=tk.VERTICAL,
command=self.task_tree.yview)
self.task_tree.configure(yscrollcommand=scrollbar.set)
```

```
self.task_tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
```

```
# Details area
details_frame = ttk.LabelFrame(main_frame, text="Task Details", padding="10")
details_frame.grid(row=2, column=0, columnspan=2, sticky=(tk.W, tk.E), pady=10)
```

```
self.details_text = scrolledtext.ScrolledText(details_frame, height=6, width=80)
self.details_text.pack(fill=tk.BOTH, expand=True)
```

```
# Bind selection event
self.task_tree.bind('<<TreeviewSelect>>', self.show_task_details)
```

```
# Configure grid weights
main_frame.columnconfigure(0, weight=1)
main_frame.rowconfigure(1, weight=1)
self.root.columnconfigure(0, weight=1)
self.root.rowconfigure(0, weight=1)
```

```
def refresh_task_list(self):
```



```

"""Refresh task list based on filters"""
# Clear existing items
for item in self.task_tree.get_children():
    self.task_tree.delete(item)

# Get tasks based on filter
filter_val = self.filter_var.get()
search_query = self.search_var.get()

if filter_val == 'pending':
    tasks = self.manager.get_tasks_by_status(completed=False)
elif filter_val == 'completed':
    tasks = self.manager.get_tasks_by_status(completed=True)
else:
    tasks = self.manager.tasks

# Apply search
if search_query:
    tasks = [t for t in tasks if search_query.lower() in t.title.lower()
              or search_query.lower() in t.description.lower()]

# Add tasks to tree
priority_icons = {
    Priority.HIGH: "🔴",
    Priority.MEDIUM: "🟡",
    Priority.LOW: "🟢"
}

for task in tasks:
    status = "✓" if task.completed else "□"
    priority_icon = priority_icons.get(task.priority, "")
    due_date = task.due_date.isoformat() if task.due_date else ""

    self.task_tree.insert("", tk.END, values=(
        task.id,
        status,
        f"{priority_icon} {task.priority.name.title()}",
        task.title,
        task.category.value,
        due_date
    ), tags=('completed' if task.completed else 'pending',))

# Configure tags
self.task_tree.tag_configure('completed', foreground='gray')

```

```

def show_task_details(self, event):
    """Show task details when selected"""
    selection = self.task_tree.selection()
    if not selection:
        return

    item = self.task_tree.item(selection[0])
    task_id = int(item['values'][0])
    task = self.manager.get_task(task_id)

    if task:
        details = f"Title: {task.title}\n\n"
        details += f"Description: {task.description}\n\n"
        details += f"Priority: {task.priority.name.title()}\n"
        details += f"Category: {task.category.value}\n"
        if task.due_date:
            details += f"Due Date: {task.due_date}\n"
        details += f"Created: {task.created_at.strftime('%Y-%m-%d %H:%M')}\n"
        details += f>Status: {'Completed' if task.completed else 'Pending'}"

        self.details_text.delete(1.0, tk.END)
        self.details_text.insert(1.0, details)

def add_task_dialog(self):
    """Dialog to add new task"""
    dialog = tk.Toplevel(self.root)
    dialog.title("Add Task")
    dialog.geometry("400x500")
    dialog.transient(self.root)
    dialog.grab_set()

    # Form fields
    ttk.Label(dialog, text="Title:").grid(row=0, column=0, sticky=tk.W, padx=10, pady=5)
    title_entry = ttk.Entry(dialog, width=40)
    title_entry.grid(row=0, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Description:").grid(row=1, column=0, sticky=tk.W, padx=10,
pady=5)
    desc_text = tk.Text(dialog, height=5, width=30)
    desc_text.grid(row=1, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Priority:").grid(row=2, column=0, sticky=tk.W, padx=10,
pady=5)

```

```

priority_var = tk.StringVar(value="MEDIUM")
priority_combo = ttk.Combobox(dialog, textvariable=priority_var,
                              values=['LOW', 'MEDIUM', 'HIGH'], width=20)
priority_combo.grid(row=2, column=1, padx=10, pady=5)

ttk.Label(dialog, text="Category:").grid(row=3, column=0, sticky=tk.W, padx=10,
pady=5)
category_var = tk.StringVar(value="OTHER")
category_combo = ttk.Combobox(dialog, textvariable=category_var,
                              values=[c.value for c in Category], width=20)
category_combo.grid(row=3, column=1, padx=10, pady=5)

ttk.Label(dialog, text="Due Date (YYYY-MM-DD):").grid(row=4, column=0,
sticky=tk.W, padx=10, pady=5)
date_entry = ttk.Entry(dialog, width=20)
date_entry.grid(row=4, column=1, padx=10, pady=5)

def save_task():
    title = title_entry.get().strip()
    if not title:
        messagebox.showerror("Error", "Title cannot be empty!")
        return

    description = desc_text.get(1.0, tk.END).strip()
    priority = Priority[priority_var.get()]
    category = Category(category_var.get())

    due_date = None
    date_str = date_entry.get().strip()
    if date_str:
        try:
            due_date = datetime.strptime(date_str, '%Y-%m-%d').date()
        except ValueError:
            messagebox.showerror("Error", "Invalid date format!")
            return

    self.manager.add_task(title, description, priority, category, due_date)
    self.refresh_task_list()
    dialog.destroy()
    messagebox.showinfo("Success", "Task added successfully!")

ttk.Button(dialog, text="Save", command=save_task).grid(row=5, column=0,
columnspan=2, pady=20)

```

```

def edit_task_dialog(self):
    """Dialog to edit task"""
    selection = self.task_tree.selection()
    if not selection:
        messagebox.showwarning("Warning", "Please select a task to edit!")
        return

    item = self.task_tree.item(selection[0])
    task_id = int(item['values'][0])
    task = self.manager.get_task(task_id)

    if not task:
        messagebox.showerror("Error", "Task not found!")
        return

    dialog = tk.Toplevel(self.root)
    dialog.title(f"Edit Task {task_id}")
    dialog.geometry("400x500")
    dialog.transient(self.root)
    dialog.grab_set()

    # Form fields with current values
    ttk.Label(dialog, text="Title:").grid(row=0, column=0, sticky=tk.W, padx=10, pady=5)
    title_entry = ttk.Entry(dialog, width=40)
    title_entry.insert(0, task.title)
    title_entry.grid(row=0, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Description:").grid(row=1, column=0, sticky=tk.W, padx=10,
pady=5)
    desc_text = tk.Text(dialog, height=5, width=30)
    desc_text.insert(1.0, task.description)
    desc_text.grid(row=1, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Priority:").grid(row=2, column=0, sticky=tk.W, padx=10,
pady=5)
    priority_var = tk.StringVar(value=task.priority.name)
    priority_combo = ttk.Combobox(dialog, textvariable=priority_var,
                                values=['LOW', 'MEDIUM', 'HIGH'], width=20)
    priority_combo.grid(row=2, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Category:").grid(row=3, column=0, sticky=tk.W, padx=10,
pady=5)
    category_var = tk.StringVar(value=task.category.value)
    category_combo = ttk.Combobox(dialog, textvariable=category_var,

```

```

        values=[c.value for c in Category], width=20)
category_combo.grid(row=3, column=1, padx=10, pady=5)

    ttk.Label(dialog, text="Due Date (YYYY-MM-DD):").grid(row=4, column=0,
sticky=tk.W, padx=10, pady=5)
    date_entry = ttk.Entry(dialog, width=20)
    if task.due_date:
        date_entry.insert(0, task.due_date.isoformat())
    date_entry.grid(row=4, column=1, padx=10, pady=5)

def save_task():
    title = title_entry.get().strip()
    if not title:
        messagebox.showerror("Error", "Title cannot be empty!")
        return

    description = desc_text.get(1.0, tk.END).strip()
    priority = Priority[priority_var.get()]
    category = Category(category_var.get())

    due_date = None
    date_str = date_entry.get().strip()
    if date_str:
        try:
            due_date = datetime.strptime(date_str, '%Y-%m-%d').date()
        except ValueError:
            messagebox.showerror("Error", "Invalid date format!")
            return

    self.manager.update_task(task_id, title=title, description=description,
                             priority=priority, category=category, due_date=due_date)
    self.refresh_task_list()
    dialog.destroy()
    messagebox.showinfo("Success", "Task updated successfully!")

    ttk.Button(dialog, text="Save", command=save_task).grid(row=5, column=0,
columnspan=2, pady=20)

def delete_task(self):
    """Delete selected task"""
    selection = self.task_tree.selection()
    if not selection:
        messagebox.showwarning("Warning", "Please select a task to delete!")
        return

```

```

        if messagebox.askyesno("Confirm", "Are you sure you want to delete this task?"):
            item = self.task_tree.item(selection[0])
            task_id = int(item['values'][0])
            self.manager.delete_task(task_id)
            self.refresh_task_list()
            messagebox.showinfo("Success", "Task deleted!")

def complete_task(self):
    """Complete selected task"""
    selection = self.task_tree.selection()
    if not selection:
        messagebox.showwarning("Warning", "Please select a task to complete!")
        return

    item = self.task_tree.item(selection[0])
    task_id = int(item['values'][0])
    task = self.manager.get_task(task_id)

    if task and not task.completed:
        self.manager.complete_task(task_id)
        self.refresh_task_list()
        messagebox.showinfo("Success", f"Task '{task.title}' completed!")

def run(self):
    """Run the GUI application"""
    self.root.mainloop()

# MAIN ENTRY POINT
def main():
    """Main application entry point"""
    print("Choose interface:")
    print("1. Command Line Interface (CLI)")
    print("2. Graphical User Interface (GUI)")

    choice = input("Choice (1/2): ").strip()

    if choice == '1':
        app = TodoCLI()
        app.run()
    elif choice == '2':
        app = TodoGUI()
        app.run()
    else:

```

```

        print("Invalid choice. Exiting.")

if __name__ == "__main__":
    main()

```

Class Activity:

- Complete the project by testing all features
- Add additional features like:
 - Task categories with colors
 - Export/import tasks
 - Recurring tasks
 - Notifications/reminders
 - Task tags and filtering
- Create a simple web version using Flask

Day 30: Build a Data Project

Project: Data Analysis & Web Scraping Application

```

"""
    Data Analysis & Web Scraping Project
    - Scrape data from websites
    - Clean and analyze data
    - Visualize results
    - Export reports
"""

import requests
from bs4 import BeautifulSoup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import json
import time
import random
from typing import List, Dict, Any
import logging

# Setup logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

```

```

logger = logging.getLogger(__name__)

# WEB SCRAPER CLASS
class WebScraper:
    """Base web scraper with common functionality"""

    def __init__(self, base_url, delay=1):
        self.base_url = base_url
        self.delay = delay
        self.session = requests.Session()
        self.session.headers.update({
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
        })

    def _request(self, url):
        """Make request with delay"""
        time.sleep(self.delay + random.uniform(0, 1))
        try:
            response = self.session.get(url)
            response.raise_for_status()
            return response
        except requests.exceptions.RequestException as e:
            logger.error(f"Request failed: {e}")
            return None

    def parse_html(self, html):
        """Parse HTML content"""
        return BeautifulSoup(html, 'html.parser')

    def scrape_page(self, url):
        """Scrape single page"""
        response = self._request(url)
        if response:
            return self.parse_html(response.text)
        return None

# PRODUCT SCRAPER (Example: Books to Scrape)
class BookScraper(WebScraper):
    """Scrape book information from books.toscrape.com"""

    def __init__(self):
        super().__init__('http://books.toscrape.com')
        self.books = []

```



```

def scrape_all_books(self):
    """Scrape all books from the site"""
    url = f"{self.base_url}/catalogue/page-1.html"
    page = 1

    while url:
        logger.info(f"Scraping page {page}")
        soup = self.scrape_page(url)
        if not soup:
            break

        # Extract books from page
        books = self.extract_books(soup)
        self.books.extend(books)
        logger.info(f"Found {len(books)} books on page {page}")

        # Find next page
        next_button = soup.find('li', class_='next')
        if next_button and next_button.a:
            next_url = next_button.a['href']
            url = f"{self.base_url}/catalogue/{next_url}"
            page += 1
        else:
            break

    logger.info(f"Total books scraped: {len(self.books)}")
    return self.books

def extract_books(self, soup):
    """Extract book information from page"""
    books = []
    book_articles = soup.find_all('article', class_='product_pod')

    for article in book_articles:
        try:
            # Get book URL
            title_elem = article.h3.a
            book_url = f"{self.base_url}/catalogue/{title_elem['href']}"

            # Get detailed book info
            book_data = self.scrape_book_details(book_url)
            if book_data:
                books.append(book_data)
        except Exception as e:

```

```

        logger.error(f"Error extracting book: {e}")
        continue

    return books

def scrape_book_details(self, book_url):
    """Scrape detailed information about a book"""
    soup = self.scrape_page(book_url)
    if not soup:
        return None

    try:
        # Extract book details
        title = soup.h1.text.strip()

        # Price
        price_elem = soup.find('p', class_='price_color')
        price = float(price_elem.text.replace('£', '').replace('Â', ''))

        # Availability
        avail_elem = soup.find('p', class_='instock availability')
        availability = avail_elem.text.strip() if avail_elem else 'Out of stock'
        in_stock = 'In stock' in availability

        # Rating
        rating_elem = soup.find('p', class_='star-rating')
        rating_classes = rating_elem.get('class', [])
        rating = [r for r in rating_classes if r != 'star-rating'][0] if len(rating_classes) > 1
    else 'No rating'
        rating_map = {'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5}
        rating_value = rating_map.get(rating, 0)

        # Description
        desc_elem = soup.find('meta', {'name': 'description'})
        description = desc_elem['content'].strip() if desc_elem else ""

        # Category
        breadcrumb = soup.find('ul', class_='breadcrumb')
        if breadcrumb:
            category = breadcrumb.find_all('li')[2].a.text.strip()
        else:
            category = 'Unknown'

        # UPC

```

```
upc_elem = soup.find('th', text='UPC')
upc = upc_elem.find_next_sibling('td').text if upc_elem else 'N/A'
```

```
return {
    'title': title,
    'price': price,
    'availability': availability,
    'in_stock': in_stock,
    'rating': rating_value,
    'description': description,
    'category': category,
    'upc': upc,
    'url': book_url
}
```

```
except Exception as e:
    logger.error(f"Error scraping book details: {e}")
    return None
```

DATA ANALYZER

```
class DataAnalyzer:
```

```
    """Analyze scraped data"""
```

```
    def __init__(self, data):
        self.data = data
        self.df = pd.DataFrame(data)
```

```
    def clean_data(self):
        """Clean and prepare data for analysis"""
        # Remove duplicates
        self.df = self.df.drop_duplicates(subset=['title'])
```

```
    # Convert price to float
    self.df['price'] = pd.to_numeric(self.df['price'], errors='coerce')
```

```
    # Fill missing values
    self.df['description'] = self.df['description'].fillna('No description')
```

```
    # Add derived columns
    self.df['price_category'] = pd.cut(self.df['price'],
                                       bins=[0, 10, 20, 50, 100, float('inf')],
                                       labels=['Cheap', 'Affordable', 'Moderate', 'Expensive',
'Premium'])
```

```

        return self.df

    def get_statistics(self):
        """Calculate basic statistics"""
        stats = {
            'total_books': len(self.df),
            'average_price': self.df['price'].mean(),
            'median_price': self.df['price'].median(),
            'min_price': self.df['price'].min(),
            'max_price': self.df['price'].max(),
            'std_price': self.df['price'].std(),
            'books_in_stock': self.df['in_stock'].sum(),
            'average_rating': self.df['rating'].mean(),
            'categories': self.df['category'].nunique()
        }
        return stats

    def analyze_by_category(self):
        """Analyze books by category"""
        category_stats = self.df.groupby('category').agg({
            'title': 'count',
            'price': ['mean', 'min', 'max'],
            'rating': 'mean',
            'in_stock': 'sum'
        }).round(2)

        category_stats.columns = ['count', 'avg_price', 'min_price', 'max_price', 'avg_rating',
                                   'in_stock']
        return category_stats.sort_values('count', ascending=False)

    def analyze_by_rating(self):
        """Analyze books by rating"""
        rating_stats = self.df.groupby('rating').agg({
            'title': 'count',
            'price': 'mean',
            'in_stock': 'sum'
        }).round(2)

        rating_stats.columns = ['count', 'avg_price', 'in_stock']
        return rating_stats

    def find_best_deals(self, min_rating=4):
        """Find best value books (high rating, low price)"""
        best_deals = self.df[

```

```
(self.df['rating'] >= min_rating) &
(self.df['in_stock'] == True)
].nsmallest(10, 'price')

return best_deals[['title', 'price', 'rating', 'category']]
```

DATA VISUALIZER

```
class DataVisualizer:
```

```
    """Create visualizations from data"""
```

```
    def __init__(self, df):
        self.df = df
        self.setup_style()
```

```
    def setup_style(self):
        """Setup plotting style"""
        plt.style.use('seaborn-v0_8-darkgrid')
        sns.set_palette("husl")
```

```
    def plot_price_distribution(self):
        """Plot price distribution"""
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
        # Histogram
        self.df['price'].hist(bins=30, ax=axes[0], edgecolor='black')
        axes[0].set_title('Price Distribution')
        axes[0].set_xlabel('Price (£)')
        axes[0].set_ylabel('Number of Books')
```

```
        # Box plot
        self.df.boxplot(column='price', ax=axes[1])
        axes[1].set_title('Price Box Plot')
        axes[1].set_ylabel('Price (£)')
```

```
        plt.tight_layout()
        plt.savefig('price_distribution.png', dpi=100)
        plt.show()
```

```
    def plot_category_analysis(self, category_stats):
        """Plot category analysis"""
        fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```
        # Top categories by count
        top_categories = category_stats.head(10)
```

```

top_categories['count'].plot(kind='bar', ax=axes[0])
axes[0].set_title('Top 10 Categories by Book Count')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Number of Books')
axes[0].tick_params(axis='x', rotation=45)

# Average price by category
top_categories['avg_price'].plot(kind='bar', ax=axes[1], color='green')
axes[1].set_title('Average Price by Category (Top 10)')
axes[1].set_xlabel('Category')
axes[1].set_ylabel('Average Price (£)')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.savefig('category_analysis.png', dpi=100)
plt.show()

def plot_rating_analysis(self):
    """Plot rating analysis"""
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Rating distribution
    rating_counts = self.df['rating'].value_counts().sort_index()
    rating_counts.plot(kind='bar', ax=axes[0], color='orange')
    axes[0].set_title('Book Ratings Distribution')
    axes[0].set_xlabel('Rating (stars)')
    axes[0].set_ylabel('Number of Books')

    # Price vs Rating
    self.df.boxplot(column='price', by='rating', ax=axes[1])
    axes[1].set_title('Price Distribution by Rating')
    axes[1].set_xlabel('Rating (stars)')
    axes[1].set_ylabel('Price (£)')

    plt.suptitle("")
    plt.tight_layout()
    plt.savefig('rating_analysis.png', dpi=100)
    plt.show()

def plot_price_vs_rating(self):
    """Scatter plot of price vs rating"""
    plt.figure(figsize=(10, 6))

    # Scatter plot with trend line

```

```

sns.regplot(x='rating', y='price', data=self.df, scatter_kws={'alpha':0.5})
plt.title('Price vs Rating Relationship')
plt.xlabel('Rating (stars)')
plt.ylabel('Price (£)')

plt.tight_layout()
plt.savefig('price_vs_rating.png', dpi=100)
plt.show()

def create_dashboard(self, stats, category_stats):
    """Create comprehensive dashboard"""
    fig = plt.figure(figsize=(16, 10))

    # Title
    fig.suptitle('Book Sales Analysis Dashboard', fontsize=16, fontweight='bold')

    # 1. Key metrics (text)
    ax1 = plt.subplot(2, 3, 1)
    ax1.axis('off')
    metrics_text = (
        f"Key Metrics\n\n"
        f"Total Books: {stats['total_books']:.}\n"
        f"Avg Price: £{stats['average_price']:.2f}\n"
        f"Median Price: £{stats['median_price']:.2f}\n"
        f"Avg Rating: {stats['average_rating']:.2f}/5\n"
        f"Books in Stock: {stats['books_in_stock']:.}\n"
        f"Categories: {stats['categories']}\n"
        f"Price Range: £{stats['min_price']:.2f} - £{stats['max_price']:.2f}"
    )
    ax1.text(0.1, 0.5, metrics_text, fontsize=10, verticalalignment='center')

    # 2. Price distribution
    ax2 = plt.subplot(2, 3, 2)
    self.df['price'].hist(bins=30, ax=ax2, edgecolor='black', alpha=0.7)
    ax2.set_title('Price Distribution')
    ax2.set_xlabel('Price (£)')

    # 3. Top categories
    ax3 = plt.subplot(2, 3, 3)
    top_cats = category_stats.head(8)
    top_cats['count'].plot(kind='bar', ax=ax3, color='skyblue')
    ax3.set_title('Top Categories')
    ax3.set_xlabel("")
    ax3.tick_params(axis='x', rotation=45)

```

```

# 4. Rating distribution
ax4 = plt.subplot(2, 3, 4)
rating_counts = self.df['rating'].value_counts().sort_index()
rating_counts.plot(kind='bar', ax=ax4, color='orange')
ax4.set_title('Rating Distribution')
ax4.set_xlabel('Stars')

# 5. Price by category
ax5 = plt.subplot(2, 3, 5)
top_cats['avg_price'].plot(kind='bar', ax=ax5, color='lightgreen')
ax5.set_title('Avg Price by Category')
ax5.set_xlabel("")
ax5.tick_params(axis='x', rotation=45)

# 6. Price vs Rating
ax6 = plt.subplot(2, 3, 6)
sns.boxplot(x='rating', y='price', data=self.df, ax=ax6)
ax6.set_title('Price Distribution by Rating')

plt.tight_layout()
plt.savefig('dashboard.png', dpi=150)
plt.show()

```

REPORT GENERATOR

```
class ReportGenerator:
```

```
    """Generate reports from analysis"""
```

```

    def __init__(self, df, stats, category_stats, rating_stats, best_deals):
        self.df = df
        self.stats = stats
        self.category_stats = category_stats
        self.rating_stats = rating_stats
        self.best_deals = best_deals

```

```

    def generate_markdown_report(self, filename='report.md'):
        """Generate markdown report"""
        with open(filename, 'w', encoding='utf-8') as f:
            f.write("# Book Sales Analysis Report\n\n")
            f.write(f"""Generated on {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}*\n\n")

```

```

        # Executive Summary
        f.write("## Executive Summary\n\n")

```



```

f.write(f"- Total Books Analyzed: {self.stats['total_books']}\n")
f.write(f"- Average Price: £{self.stats['average_price']:.2f}\n")
f.write(f"- Average Rating: {self.stats['average_rating']:.2f} / 5\n")
f.write(f"- Books in Stock: {self.stats['books_in_stock']}\n")
f.write(f"- Categories Found: {self.stats['categories']}\n\n")

# Price Analysis
f.write("## Price Analysis\n\n")
f.write("| Metric | Value |\n")
f.write("|-----|-----|\n")
f.write(f"| Minimum Price | £{self.stats['min_price']:.2f} |\n")
f.write(f"| Maximum Price | £{self.stats['max_price']:.2f} |\n")
f.write(f"| Median Price | £{self.stats['median_price']:.2f} |\n")
f.write(f"| Standard Deviation | £{self.stats['std_price']:.2f} |\n\n")

# Category Analysis
f.write("## Top Categories\n\n")
f.write("| Category | Books | Avg Price (£) | Avg Rating |\n")
f.write("|-----|-----|-----|-----|\n")
for idx, row in self.category_stats.head(10).iterrows():
    f.write(f"| {idx} | {row['count']} | {row['avg_price']:.2f} | {row['avg_rating']:.2f} |\n")
f.write("\n")

# Best Deals
f.write("## Best Value Books\n\n")
f.write("| Title | Price (£) | Rating | Category |\n")
f.write("|-----|-----|-----|-----|\n")
for _, row in self.best_deals.iterrows():
    f.write(f"| {row['title']} | {row['price']:.2f} | {row['rating']} | {row['category']} |\n")
f.write("\n")

# Recommendations
f.write("## Recommendations\n\n")
f.write("1. Inventory Management: Focus on categories with high demand\n")
f.write("2. Pricing Strategy: Adjust prices for categories with high price  
variance\n")
f.write("3. Quality Focus: Maintain high ratings in top-performing  
categories\n")
f.write("4. Stock Optimization: Ensure availability of best-rated books\n")

logger.info(f"Markdown report saved to {filename}")

def generate_csv_export(self, filename='books_data.csv'):
    """Export data to CSV"""

```

```

        self.df.to_csv(filename, index=False, encoding='utf-8')
        logger.info(f"CSV data exported to {filename}")

    def generate_json_export(self, filename='analysis.json'):
        """Export analysis to JSON"""
        export_data = {
            'metadata': {
                'generated_at': datetime.now().isoformat(),
                'total_books': len(self.df)
            },
            'statistics': self.stats,
            'category_analysis': self.category_stats.to_dict(),
            'rating_analysis': self.rating_stats.to_dict(),
            'best_deals': self.best_deals.to_dict('records')
        }

        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(export_data, f, indent=2, default=str)

        logger.info(f"JSON analysis saved to {filename}")

# MAIN APPLICATION
def main():
    """Main application entry point"""
    print("=" * 60)
    print("Book Sales Analysis & Web Scraping Application")
    print("=" * 60)

    # Step 1: Scrape data
    print("\nStep 1: Scraping book data...")
    scraper = BookScraper()
    books = scraper.scrape_all_books()

    if not books:
        print("No data scraped. Exiting.")
        return

    print(f"✓ Scraped {len(books)} books")

    # Step 2: Analyze data
    print("\nStep 2: Analyzing data...")
    analyzer = DataAnalyzer(books)
    df = analyzer.clean_data()
    stats = analyzer.get_statistics()

```

```

category_stats = analyzer.analyze_by_category()
rating_stats = analyzer.analyze_by_rating()
best_deals = analyzer.find_best_deals()

print("✓ Analysis complete")

# Step 3: Visualize data
print("\nStep 3: Creating visualizations...")
visualizer = DataVisualizer(df)

visualizer.plot_price_distribution()
visualizer.plot_category_analysis(category_stats)
visualizer.plot_rating_analysis()
visualizer.plot_price_vs_rating()
visualizer.create_dashboard(stats, category_stats)

print("✓ Visualizations saved")

# Step 4: Generate reports
print("\nStep 4: Generating reports...")
reporter = ReportGenerator(df, stats, category_stats, rating_stats, best_deals)

reporter.generate_markdown_report()
reporter.generate_csv_export()
reporter.generate_json_export()

print("✓ Reports generated")

# Step 5: Display summary
print("\n" + "=" * 60)
print("ANALYSIS SUMMARY")
print("=" * 60)
print(f"Total Books: {stats['total_books']}")
print(f"Average Price: £{stats['average_price']:.2f}")
print(f"Average Rating: {stats['average_rating']:.2f}/5")
print(f"Categories Found: {stats['categories']}")
print(f"Books in Stock: {stats['books_in_stock']}")

print("\nTop 5 Best Value Books:")
for idx, (_, row) in enumerate(best_deals.head(5).iterrows(), 1):
    print(f" {idx}. {row['title'][:50]}... - £{row['price']:.2f} ({row['rating']}★)")

print("\nGenerated Files:")
print(" - price_distribution.png")

```

```

print(" - category_analysis.png")
print(" - rating_analysis.png")
print(" - price_vs_rating.png")
print(" - dashboard.png")
print(" - report.md")
print(" - books_data.csv")
print(" - analysis.json")

print("\nAnalysis complete!")

if __name__ == "__main__":
    main()

```

Class Activity:

- Run the complete data analysis pipeline
- Add additional data sources (e.g., news articles, social media)
- Create interactive dashboards using Plotly or Dash
- Implement scheduled scraping with cron jobs

Course Conclusion & Next Steps

Congratulations! You've completed the 30-day Python mastery course. Here's what you've learned:

Skills Acquired:

1. **Python Fundamentals:** Variables, data types, control flow
2. **Data Structures:** Lists, tuples, dictionaries, sets
3. **Functions:** Regular, lambda, decorators, generators
4. **Object-Oriented Programming:** Classes, inheritance, polymorphism
5. **Advanced Concepts:** Context managers, iterators, decorators
6. **Libraries:** NumPy, Pandas, Requests, BeautifulSoup
7. **Tools:** Virtual environments, pip, Git
8. **Projects:** CLI apps, GUI apps, data analysis, web scraping

Next Steps:

1. **Practice Daily:** Code every day to reinforce concepts
2. **Build Portfolio:** Create 5-10 projects to showcase on Github or netlify
3. **Learn Frameworks:** Django (web), Flask (API), FastAPI (modern API)
4. **Explore Specializations:**
 - Data Science: Scikit-learn, TensorFlow

- Web Development: Django, Flask
 - Automation: Selenium, PyAutoGUI
 - Game Development: Pygame
5. **Contribute to Open Source:** Start with small contributions
 6. **Prepare for Interviews:** Practice coding challenges on LeetCode
 7. **Join Community:** Python Discord, Reddit r/learnpython, local meetups

Resources:

- **Documentation:** docs.python.org
- **Practice:** LeetCode, HackerRank, Codewars
- **Books:** "Fluent Python", "Python Crash Course"
- **Blogs:** Real Python, PyCoder's Weekly
- **Videos:** PyCon talks, Corey Schafer, Sentdex